Understanding ML: the teacher's edition

Lectures 1–5 explained the way a teacher explains them in class: what each method is and why, the experiment behind it narrated step by step, what students do wrong and why it is tempting, how to tell neighbouring methods apart, the derivations the professor asks with every step justified, and his questions answered at the board. Chapters 3, 4 and 5 in depth.

Fundamentals of Machine Learning, midterm

2026-11-03

How to read this

Six parts, in this order: the Experiments first, because that is where the "what is the issue, what would you do" questions come from; then chapters 3, 4 and 5, which carry most of the exam; then chapters 1 and 2. Each concept is one card with five parts: **What it is** and why it is defined that way, **Manipulate / measure** where I run the experiment in front of you and read the curve with you, **Goes wrong** where I tell you what you will be tempted to do and how to catch it, **Versus** where I clear up the confusion with the neighbouring method, and an **Example** with numbers. After the cards of a chapter come one table that separates its methods, the derivations with each step justified, the professor's questions answered the way I would answer them at the board, and five mistakes. Formulas marked [on your sheet] are on your two-page sheet, so do not memorise them; learn what each symbol means and when the formula applies. [not on your sheet: know it] marks the few you must carry in your head.

Contents

1 The Experiments: split, test set, cross-validation, tuning, learning curves, bias–variance **2**

2 Chapter 3: From Fitting to Generalization **10**

3 Chapter 4: Discriminative and Generative Classification **20**

4 Chapter 5: Learning by Combining Models **31**

5 Chapter 1: What Does It Mean to Learn? **43**

6 Chapter 2: From Representations to Decisions **47**

1 The Experiments: split, test set, cross-validation, tuning, learning curves, bias–variance

Let me start with the one idea that the whole course rests on, because every method you will meet tomorrow is judged by it. When we train a model, we let the data choose it. If we then ask the same data how good the model is, the data will answer kindly, because the model was shaped to please them. So the only honest judge is a case the model has never seen. Everything in this part is a consequence of that sentence: why we keep three sets of rows, why the test set is used once and never touched again, why cross-validation holds out each fold in turn, why every hyperparameter is chosen on validation rows and never on training rows, and why a learning curve or a bias–variance experiment has to refit the model many times rather than look at one fit. Read this part first. The professor's "what is the issue here, what would you do" questions nearly all come from it. Formulas marked [on your sheet] are on your sheet; what you must carry in your head is why they are true and when they apply.

The generic learning experiment, and the four curves you will be shown

What it is. Everything we measure in this course is a risk, that is an average loss over a set of examples. On a set S it is $\widehat{R}_S(f) = \frac{1}{|S|}\sum_{i\in S} \ell(y^{(i)}, f(x^{(i)}))$ [on your sheet], and on the whole population it is $R(f) = \mathbb{E}[\ell(Y, f(X))]$ [on your sheet]. The population risk is the number we actually care about, because it describes the next case, and it is also the one number we can never compute, because we do not have the population. So every experiment in this course is a way of estimating R with some $\widehat{R}_S$, and the whole art lies in choosing the set S so that the estimate is not flattering.

Manipulate / measure. Here is how I would run any learning experiment in front of you. I fix everything: the rows in each subset, the preprocessing, the random seed, the metric. Then I change exactly one knob, which could be the polynomial degree, the tree depth, the regularisation strength λ, the number of neighbours k, the number of training epochs, the number of trees, or the amount of training data N. At each setting of the knob I record two numbers: the risk on the rows that fitted the model, and the risk on rows the model never saw. Then I draw both against the knob. You read the two curves with three questions, in this order. Are both curves high? Then the family is too restricted or the features are poor, and we call that underfitting. Is the gap between them large? Then the fit depends on the particular sample, and we call that overfitting or high variance. Are the curves still moving as I turn the knob further? Then I have not yet reached the setting where the knob stops mattering, and I should extend the grid.

Goes wrong. A student will tell me "the error goes down when we make the model bigger", and I will ask: which error? The training error always goes down when capacity increases, because a bigger family contains the smaller one, so it can at least reuse the smaller family's best fit. That sentence therefore carries no information until you tell me what the held-out error did.

Versus. Students mix up four plots that look alike, because all four show a curve that falls. The way to keep them apart is to look at the horizontal axis. On the validation curve the axis is capacity (degree, depth, $1/k$, $1/\lambda$): training error falls monotonically and held-out error makes a U. On the training curve the axis is epochs or boosting rounds: the same U, but over time, and the minimum is where early stopping stops. On the learning curve the axis is the amount of data N: training error rises and held-out error falls, both toward the same plateau. On the ensemble-size curve the axis is the number of models M: the held-out error falls and then flattens on a floor that no number of models can lower.

The repeated-sampling thought experiment

What it is. Imagine that I draw a training set from the world, run the whole learning procedure, and obtain a fitted model $\hat{f}_{\mathcal{D}}$. Now imagine I throw that training set away, draw a fresh one of the same size, and run the procedure again. I get a different model, $\hat{f}_{\mathcal{D}'}$. Every quantity in this course that wears a hat is random in this sense: it depends on which sample happened to be drawn. When we say "bias", "variance", "optimistic" or "honest", we are always making a statement about what happens on average over these imaginary repetitions, which we write $\mathbb{E}_{\mathcal{D}}[\cdot]$.

Manipulate / measure. You will never see this experiment in real life with real data, because you only have one sample. But you can run it in simulation, where you know the truth and can draw as many training sets as you like, and you can approximate it on real data by refitting with different random seeds, different folds or different bootstrap samples. Each refit is one draw. When you see the predictions scatter from refit to refit, you are looking at variance with your own eyes.

Goes wrong. The phrase "the variance of this trained model" is a mistake students make constantly. One fitted model is one draw; it has no variance, any more than a single coin flip has a variance. Variance belongs to the procedure, and to measure it you must refit.

Train, validation, test: three sets for three questions

What it is. We cut the labelled rows into three disjoint sets, and each set answers one question. The training rows fit the parameters θ: the weights, the coefficients, the split thresholds. The validation rows choose between alternatives that training cannot choose for itself: the regularisation strength, the number of neighbours, the depth, which features to keep, the decision threshold, when to stop training; we call all of these hyperparameters h. The test rows are used once, at the very end, to report how good the single chosen model is [on your sheet]. The reason we need three sets and not two is that each question would be answered dishonestly by the rows used for the previous question. The training rows grade the fitted model too kindly because they chose it. The validation rows grade each candidate honestly, but once you pick the candidate with the best validation score, that best score is flattering, because you selected the luckiest of several noisy numbers. Only the test set took no part in any choice, which is the only reason its number can be believed.

Manipulate / measure. Before fitting anything, I write down which rows go where, and I never move them. The rule for cutting is not arbitrary: it comes from the deployment question, "what kind of new case will this model meet?" If new cases are independent draws from the same population, a random split is fine. If one class is rare, I stratify, which means I keep the class proportions the same in each subset so that the rare class is not accidentally absent from the test rows. If the model will meet new users, devices, patients or hospitals, I split by group: every group lies entirely in one subset, so the model is tested on groups it has never seen. If the model predicts the future, I split by time: train on the past, validate on later rows, test on the latest rows [on your sheet]. Then I look at the table of results, one row per protocol for the same model. The random-split number is always the largest, the grouped number is smaller, the temporal number is the smallest, and only the protocol that mirrors deployment gives the number that will come true.

Goes wrong. The tempting mistake is a random split when deployment is in the future or on new entities, because it is the default in every library and it gives the nicest number. The number is not wrong arithmetic; it answers a question nobody asked. A second mistake is stratifying across groups: scattering one device's flows across train and test to balance the classes destroys the grouping, and the model can again recognise the device.

Example. Think of flows recorded from 50 devices over 6 months. With a random split, flows from every device and every week land on both sides, so the model can learn that device 17 is the compromised laptop, and it scores 99.5%. With a grouped split that holds out 10 devices, the model has to recognise attack behaviour, and it scores 96%. With a temporal split that trains on months 1 to 4 and tests on months 5 and 6, during which a new attack tool appeared, it scores 90%. Only the last number describes next month.

How big must the validation set be?

What it is. A validation score is an estimate, and estimates have noise. The standard error of an accuracy near 0.9 measured on n rows is $\sqrt{0.9 \cdot 0.1/n}$, which is 0.03 for $n = 100$, about 0.0095 for $n = 1{,}000$, and 0.003 for $n = 10{,}000$.

Goes wrong. So when a student compares two models that differ by half a point on 100 validation rows and declares a winner, they are reading noise: the difference is six times smaller than the standard error. With few rows, the remedy is cross-validation, which uses every row for validation once.

Leakage: using tomorrow's information today

What it is. Leakage is any use, during development, of information that will not exist at the moment a new decision has to be made [on your sheet]. It takes many forms, and they all have the same effect: the held-out number becomes unrealistically good, sometimes perfect. The forms you must recognise are preprocessing fitted on all rows before splitting; duplicates or near-duplicates that appear in two subsets; the same device, session or patient on both sides; features that were created after the outcome, such as "number of analyst comments" on an event, which are written during the investigation that follows the attack; a time series shuffled before splitting, so that the model sees the future of the series it is tested on; and hyperparameters chosen by looking at test results.

Manipulate / measure. The way I catch leakage is a question I ask of every feature: at what moment does this value become known, relative to the moment of the decision? If the answer is "after", the feature leaks. And the way I prevent the preprocessing form is a rule: the scaler, the imputer, the feature selector, the dimensionality reduction are all fitted on the training rows only, and the fitted numbers $\hat{\mu}_j$ and $\hat{\sigma}_j$ are reused unchanged on validation, test and deployment rows [on your sheet]. In cross-validation, each fold fits them on its own training portion.
Goes wrong. The classic case: a student standardises all 10,000 rows, then splits 80/20. For a scaler on that many rows the effect is small, because two sample means differ little. But the same habit with feature selection, where you pick the 20 most correlated features on all rows, or with imputation by a fitted model, inflates the estimate by several points, and with few rows and many features it can produce near-perfect scores on labels that are pure noise.

Why the test set must not be touched: the winner's curse

What it is. Suppose you evaluate many candidate configurations on the same validation set and you keep the one with the best score. Each score, taken alone, was an honest estimate of its candidate's true risk. But the best score is the minimum of several noisy numbers, and the minimum of noisy numbers is pulled below the truth, because part of what made it the minimum was luck. So the validation score of the winner is optimistic even though no single score was biased. On the sheet this reads $\mathbb{E}[\min_h \widehat{R}_{\text{val}}(\hat{f}_h)] \leq \min_h R(\hat{f}_h)$ [on your sheet], with strict inequality as soon as the estimates have noise. How large is the effect? For two candidates with the same true risk R and independent Gaussian noise of standard deviation σ, the expected minimum is $R - \sigma/\sqrt{\pi} \approx R - 0.56\sigma$ [on your sheet]. For n candidates the optimism grows like $\sigma\sqrt{2\ln n}$ [on your sheet], which is 1.18σ for $n = 2$, 2.15σ for $n = 10$, 3.0σ for $n = 100$ and 3.7σ for $n = 1{,}000$.
Manipulate / measure. Here is the experiment that shows it with nothing to learn. I take labels that are pure coin flips and split them 500/500. I then try n random rules that ignore the input entirely, and I keep whichever scores best on the validation half. As n goes from 1 to 10 to 100 to 1,000 to 10,000, the best validation accuracy climbs from about 50% to 53%, 56%, 57%, 59%. Nothing was learned; I merely selected the luckiest coin. Now I score the winner on the test half: it sits at 50% for every n. That flat line is what an untouched test set buys you. And notice what would happen if I now started choosing among test scores: the test line would begin to climb in exactly the same way, and I would have no honest number left anywhere.
Goes wrong. The ways the test set gets touched are rarely dramatic. Someone runs the test set "just to see" after each change and keeps the change when the number improved: that is selection on the test set. Someone chooses the decision threshold, or which metric to report, after looking at test results. Someone stops training when the test loss is lowest. Someone tunes by cross-validation over all rows and then reports the cross-validation score of the winner as if it were a test score; but those folds chose the hyperparameters, so that score is a validation score and it is optimistic.
Versus. Notice the asymmetry in the formula: the noise σ enters linearly, the number of candidates only through $\sqrt{2\ln n}$. A small validation set therefore does more damage than a long search. The fixes follow: a larger validation set or cross-validation attacks the linear factor; a coarser search attacks the logarithmic one; and the only way to *know* the deployment number is one evaluation on rows that took no part in the comparisons.
Example. On competition platforms the public leaderboard is scored on a fixed subset. Teams that submit hundreds of times and keep whatever scores best on it routinely drop many places on the private leaderboard, which is scored on rows they never saw. The public subset had become their validation set. The same happens inside a company when "the test set" is re-run after every change: the reported number drifts upward and deployment performance does not.

K-fold cross-validation

What it is. When the rows are too few to spare a large fixed validation set, we cut the development rows into K folds, we fit on $K-1$ of them and evaluate on the one we held out, and we do this K times so that every row is validated exactly once. The estimate is the average of the K held-out scores, $\widehat{R}_{\text{CV}}(h) = \frac{1}{K}\sum_{j=1}^{K} \widehat{R}^{(j)}_{\text{val}}(\hat{f}_h^{(-j)})$ [on your sheet]. I want you to be precise about what this number estimates: it is the risk of the *procedure*, run with configuration h on about $N(K-1)/K$ rows, not the risk of any one of the K models you fitted on the way.
Manipulate / measure. I draw one fold assignment and reuse it for every candidate, so that all candidates are judged on the same slices; if deployment demands it, the folds are grouped or forward-chaining in time. For each candidate I obtain K numbers; I plot their mean against the candidate with the fold-to-fold spread as error bars, which is the validation curve with its noise made visible. I pick the minimum, but

I treat candidates whose means lie within the error bars as tied, and among ties I take the simpler one. Then comes the step students forget: I refit the chosen configuration on all development rows, because the K fold models were only instruments, and I score the test set once.

Goes wrong. The mistakes are the same leaks as before, now hidden inside the folds. If feature selection or preprocessing runs once on all development rows before the folds are cut, every validation fold has already influenced the representation it is scored on. With 300 rows and 5,000 random features, selecting the top 20 on all rows and then cross-validating gives about 97% accuracy on labels that are noise; running the selection inside each fold gives about 50%. Other mistakes: random folds on grouped or temporal data; deploying one of the fold models; reporting the winner's cross-validation score as the deployment number. When you must have an honest number without a separate test set, use nested cross-validation: an outer loop whose held-out folds are never used for any choice, and an inner cross-validation inside each outer training portion that chooses h. The outer average is honest for the whole procedure, selection included.

Versus. Why not leave one out, with $K = N$? Because the N training sets then differ by a single row, so the N fitted models are nearly identical and their held-out losses are strongly correlated; the average of strongly correlated numbers does not become stable the way an average of independent numbers does, and you pay N fits. With $K = 5$ or 10 the folds differ more, the average is more stable, the cost is 5 or 10 fits per candidate, and the price is a slight pessimism because each model saw only 80 to 90% of the rows. The out-of-bag estimate of a bagged ensemble is a free cousin of cross-validation; it carries the same winner's-curse caveat if you use it to select.

Hyperparameter tuning and the validation curve

What it is. Let me define a hyperparameter by what it does rather than by a list: a hyperparameter is any knob that the training error, left to itself, would turn to maximal capacity. The regularisation strength λ, the number of neighbours k, the degree, the depth, the number of boosting rounds, the learning rate, the number of epochs, the number of candidate features per split, the feature subset, the decision threshold: all of these would be set to "as flexible as possible" if the training rows were allowed to decide. So we solve two nested problems: the inner one on training rows, $\hat{\theta}_h = \arg\min_\theta \widehat{R}_{\text{train}}(\theta; h)$, and the outer one on validation rows, $\hat{h} = \arg\min_h \widehat{R}_{\text{val}}(\hat{f}_h)$ [on your sheet].

Manipulate / measure. I fix every other knob and the preprocessing, which was fitted on the training rows. I sweep the one knob over a grid: for λ a logarithmic grid such as $10^{-3}, 10^{-2}, \ldots, 10^{2}$, because the useful range spans orders of magnitude; for k the odd integers; for depth the integers. At each point I record training and validation risk. The training risk falls monotonically as capacity grows, which means it rises with λ and with k. The validation risk makes a U. I read $\hat{h}$ at the bottom of the U, and if the bottom sits on the edge of my grid I extend the grid, because I have not seen the far side.

Goes wrong. Why can the training set not choose λ? Because regularised fitting is the same thing as ordinary fitting restricted to the family $\Omega(\theta) \le c$, and a larger λ means a smaller c, hence a smaller family; a minimum over a smaller family is never smaller, so the training risk is smallest at $\lambda = 0$ [on your sheet]. The training set will always answer "no regularisation". The same argument sends degree, depth and epochs to their maximum and k to one. Three further habits to avoid: penalising features that were not standardised, so that a feature in millimetres is penalised a thousand times more than the same feature in metres; penalising the intercept; and searching a fine grid on a tiny validation set, where the valley you see is noise.

Versus. Keep the direction of each knob straight, because the professor will test it. Capacity grows with degree, depth, epochs and boosting rounds, and it grows when λ *decreases* and when k *decreases*. The number of bagged trees is different in kind: it does not grow capacity at all, it only makes the average more stable.

Example. On four points with $\sum x^2 = 14$ and $\sum xy = 19$, the ridge slope is $19/(14 + 4\lambda)$, which is 1.357 at $\lambda = 0$, 1.056 at $\lambda = 1$ and 0.352 at $\lambda = 10$. The training error points to $\lambda = 0$ every time; only validation points can tell you whether $\lambda = 1$ predicts better.

Early stopping, and knobs that are coupled

What it is. When a model is fitted iteratively, the number of iterations is itself a capacity knob: early iterations learn the strong recurring patterns, later ones start fitting exceptions and noise. Early stopping keeps the checkpoint with the smallest validation loss, $t^* = \arg\min_t \widehat{R}_{\text{val}}(\hat{\theta}_t)$ [on your sheet], and a patience p means we continue p epochs past the current best before we give up, so that a temporary wobble of the validation curve does not stop us too soon. The parameter count never changes; the training time is what limits the effective capacity.

Manipulate / measure. I train on, save a checkpoint after each epoch, and record training and validation loss. The training loss falls throughout. The validation loss falls, reaches its minimum at t^*, and then rises. I roll back to the checkpoint at t^*. In gradient boosting the same curve runs over rounds, but now three knobs are coupled: the learning rate η, the number of rounds M and the depth. Halving η roughly doubles the best M, because each tree applies a smaller correction; deeper trees reach the minimum in fewer rounds. So I grid over (η, depth), let early stopping choose M for each pair, and compare the pairs at their own selected rounds.
Goes wrong. Keeping the last epoch instead of the best one. Stopping on the test loss, which is tuning on the test set. And the colleague who switches η from 0.1 to 0.01 and keeps $M = 400$: they now apply about one tenth of the total correction, the model is underfitted, and their worse validation loss says nothing about the smaller rate, only about the rounds they did not re-select.

Learning curves: vary the amount of data

What it is. A learning curve plots training and validation performance against the number of training rows N [on your sheet], with everything else fixed. It is the only experiment that separates "the family cannot express this relationship" from "the family could, but the sample is too small". The height of the plateau the two curves approach is bias plus noise; the gap between them at finite N is variance.
Manipulate / measure. I fix the configuration, the validation rows and the metric. I vary N by taking nested subsamples of the training rows, 10%, 20%, up to 100%, with several random subsamples at each size, averaged. The training risk *rises* with N, because more rows are harder to fit perfectly; the validation risk *falls*; the two approach each other. Now read the shape. If both curves are high and already flat, close together, you are limited by bias: more data will not help, change the representation or increase capacity. If the training curve is low and the validation curve high, with a wide gap that narrows only slowly, you are limited by variance: regularise, reduce capacity, or collect more representative data, and the gap will close as N grows. If both are still falling and converging, you are limited by data, and more rows are likely to help. A fourth situation does not show on a learning curve at all: validation performance that degrades over calendar time, which is distribution shift and calls for a time-aware evaluation.
Goes wrong. Students expect more data to remove bias; it removes variance, and the plateau is set by the family and the features. They also confuse the data-size axis with the capacity axis and the epoch axis. And here is a subtle one: a gap that stays constant across a fiftyfold increase in data is not variance, because variance shrinks with N; it is a systematic difference between training rows and validation rows, such as a shortcut present in the training collection and absent from a grouped or temporal validation set, duplicates, or label noise concentrated on one side.
Example. A security team has 10,000 labelled flows and trains a random forest on 1,000, 2,000, up to 8,000 rows, validating on the same 2,000. Validation recall goes $0.71, 0.78, 0.83, 0.86, 0.87, 0.875, 0.878, 0.88$ while training recall stays near 1. The gap is large but closing slowly and the validation curve has nearly flattened, so labelling 20,000 more flows would buy about one point; the remaining gap is variance that larger leaves or fewer features per split address more cheaply, and the plateau says the features carry no more information.

Bias–variance experiments

What it is. At a fixed input x, let $f^*(x) = \mathbb{E}[Y \mid X = x]$ be the best possible prediction and $\bar{f}(x) = \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(x)]$ the average of what the procedure predicts across training sets. The bias is $\bar{f}(x) - f^*(x)$, the variance is $\mathbb{E}_{\mathcal{D}}[(\hat{f}_{\mathcal{D}}(x) - \bar{f}(x))^2]$, and the expected squared error splits exactly into $\sigma^2 + \text{Bias}^2 + \text{Var}$ [on your sheet]. These are properties of the procedure, meaning the family together with the fitting rule and the sample size, never of a single fit.
Manipulate / measure. I build a world I control: a known f^*, a known noise level, a fixed N and a grid of test inputs. I draw 500 training sets, refit each time, and at each test input I record the cloud of 500 predictions. The centre of the cloud minus the truth is the bias; the spread of the cloud is the variance. Then I sweep capacity: the bias squared falls, the variance rises, and their sum plus the noise is U-shaped and matches the test error I measure directly. Then I sweep N instead: the variance curve drops everywhere, the bias curve does not move, and the minimum of the U shifts to a higher capacity. That last observation is the whole story of "more data lets you afford a bigger model".
Goes wrong. Students expect regularisation to reduce variance for free. It does not: it adds bias, and whether the total falls depends on where the truth is. Take the simplest case, estimating a mean from $N = 4$ draws with noise variance 1. The sample mean has bias zero and variance 0.25. Predicting zero regardless of the data has variance zero and bias squared μ^2. Halving the sample mean has bias squared

$\mu^2/4$ and variance 0.0625. At $\mu = 1$ the totals are 0.25, 1 and 0.3125, so the unbiased procedure wins; at $\mu = 0.2$ they are 0.25, 0.04 and 0.0725, so both biased procedures win, because they traded a little systematic error for a large reduction in sample sensitivity.

Versus. Here is the map of what each lever does. More capacity: bias down, variance up. More data: variance down only. Regularisation and shrinkage: bias up, variance down, worth it when the truth is near the shrinkage target. Early stopping: the same trade through training time. Bagging and random forests: variance down to a floor, bias untouched. Boosting: bias down, variance up with the number of rounds, which is why its rounds are early-stopped. Better features: bias down, which is the only lever that moves the plateau.

Example. Compare 1-nearest-neighbour and 50-nearest-neighbour regression at a fixed x. The 1-NN prediction is one noisy neighbour's y, so its cloud is as wide as the noise but centred close to the truth: low bias, high variance. The 50-NN prediction averages fifty neighbours, so its spread is about $\sigma^2/50$, but the neighbours reach far from x and their average follows a smoothed version of the truth: higher bias, low variance. On the sheet: Bias $= \frac{1}{k}\sum_{j\le k} f(x^{(j)}) - f(x)$ and Var $= \sigma^2/k$ [on your sheet].

Variance reduction by averaging: the ensemble-size curve

What it is. If M models have errors with the same variance σ^2 and pairwise correlation ρ, the error of their average has variance $\sigma^2\left(\rho + \frac{1-\rho}{M}\right)$ [on your sheet]. With independent errors this is σ^2/M; with identical errors it is σ^2, averaging changed nothing; and as M grows it tends to $\rho\sigma^2$, not to zero.

Manipulate / measure. I draw bootstrap samples of one training set, fit a deep tree on each, and plot validation error against the number of trees $M = 1, 5, 10, 25, 50, 100, 200$. The error falls steeply and then flattens. Then I repeat with only $\sqrt{d}$ random candidate features at each split instead of all d: the curve flattens lower, because feature randomisation lowered the correlation ρ between the trees, and ρ is the floor.

Goes wrong. Average M copies of the same model, or M models that all lack the same essential feature, and you gain nothing: their errors are identical, $\rho = 1$. With $\rho = 0.2$ and $M = 25$ the variance is $0.232\sigma^2$, not the $\sigma^2/25$ a student would hope for, and no number of trees takes it below $0.2\sigma^2$.

The diagnostic table: what the curves say and what to do

Observed pattern	Likely issue	Response
training and validation both high, flat in N	underfitting (bias)	richer representation, more capacity, less regularisation
training low, validation high, gap narrows slowly with N	high variance	regularise, reduce capacity, more representative data, ensembles
both decreasing and converging	data-limited	more representative examples
gap constant across a fiftyfold increase in N	systematic train/validation difference	audit the split, duplicates, shortcuts, label noise
fine on the split, degrades over calendar time	distribution shift	temporal evaluation, retraining, monitoring
best validation score rises as more configurations are tried, test flat	winner's curse	larger validation set or CV; coarser search; test once

[on your sheet]

Training risk is optimistic (the sandwich)

Let $\hat{f}_{\mathcal{D}}$ minimise $\widehat{R}_{\mathcal{D}}$ over the family $\mathcal{H}$, and let $f^*_{\mathcal{H}}$ minimise the population risk R over the same family.
(1) Because $\hat{f}_{\mathcal{D}}$ is the minimiser of the empirical risk, $\widehat{R}_{\mathcal{D}}(\hat{f}_{\mathcal{D}}) \le \widehat{R}_{\mathcal{D}}(f^*_{\mathcal{H}})$ on every sample; this is the only place where minimisation is used.
(2) Because $f^*_{\mathcal{H}}$ was fixed before the sample was drawn, $\mathbb{E}[\widehat{R}_{\mathcal{D}}(f^*_{\mathcal{H}})] = R(f^*_{\mathcal{H}})$: an average of i.i.d. losses of a fixed function is unbiased.
(3) Because $f^*_{\mathcal{H}}$ is the best member of the family, $R(\hat{f}_{\mathcal{D}}) \ge R(f^*_{\mathcal{H}})$ for every sample.
Chain the three: $\mathbb{E}[\widehat{R}_{\mathcal{D}}(\hat{f}_{\mathcal{D}})] \le R(f^*_{\mathcal{H}}) \le \mathbb{E}[R(\hat{f}_{\mathcal{D}})]$ [on your sheet], so the expected gap is nonnegative. The word that does the work is *fixed*: a validation set grades a model that was fixed before the validation rows

were looked at, so step (2) applies to it and its number is honest.

The best of many validation scores is optimistic

Take any one candidate h_0. The minimum over all candidates is at most the value at h_0: $\min_h \widehat{R}_{\text{val}}(\hat{f}_h) \le \widehat{R}_{\text{val}}(\hat{f}_{h_0})$, pointwise on every sample.
Take expectations; the right side is $R(\hat{f}_{h_0})$ because each candidate's score is honest.
This holds for every h_0, so it holds for the best one: $\mathbb{E}[\min_h \widehat{R}_{\text{val}}(\hat{f}_h)] \le \min_h R(\hat{f}_h)$ [on your sheet].
The inequality is strict as soon as the scores are noisy and no single candidate wins on every sample. For the size: two equal candidates give $\mathbb{E}\min = R - \sigma/\sqrt{\pi}$, and n candidates give roughly $R - \sigma\sqrt{2\ln n}$ [on your sheet].

Training risk is minimal at $\lambda = 0$

Let $\hat{\theta}_\lambda$ minimise $\widehat{R} + \lambda\Omega$ and set $c = \Omega(\hat{\theta}_\lambda)$.
Take any θ with $\Omega(\theta) \le c$. Optimality of $\hat{\theta}_\lambda$ gives $\widehat{R}(\hat{\theta}_\lambda) + \lambda c \le \widehat{R}(\theta) + \lambda\Omega(\theta) \le \widehat{R}(\theta) + \lambda c$, hence $\widehat{R}(\hat{\theta}_\lambda) \le \widehat{R}(\theta)$: the penalised solution also solves the constrained problem [on your sheet].
A larger λ gives a smaller c and a smaller family, and the families are nested; a minimum over a smaller family can only be larger.
Therefore $\widehat{R}_{\text{train}}(\hat{\theta}_\lambda)$ is nondecreasing in λ, smallest at $\lambda = 0$. The same argument applies to degree, depth, epochs and $1/k$: training error turns every capacity knob to its maximum, which is why validation exists.

Why the two curves of a learning curve meet at one limit

At every sample size N the sandwich holds: $\mathbb{E}[\widehat{R}_{\text{train}}(\hat{f}_N)] \le R(f^*_{\mathcal{H}}) \le \mathbb{E}[R(\hat{f}_N)]$.
As N grows, the empirical risk of every member of a reasonable family approaches its population risk, so the fitted model approaches $f^*_{\mathcal{H}}$ and both outer terms approach $R(f^*_{\mathcal{H}})$.
The lower term rises to the limit, which is the training curve; the upper term falls to it, which is the validation curve.
The common limit is the approximation error of the family plus the noise, and no amount of data changes it; the gap at finite N is estimation error, which shrinks with N and grows with capacity. At a fixed x this reads: plateau $= \sigma^2 + \text{Bias}^2$, gap $\approx$ Var.

The bias–variance decomposition

Assume $Y = f^*(x) + \varepsilon$ with $\mathbb{E}[\varepsilon \mid x] = 0$ and variance σ^2, and a training set independent of the test pair.
Step 1: write $Y - \hat{f} = \varepsilon + (f^* - \hat{f})$ and square. The expectation of ε^2 is σ^2; the cross term is $\mathbb{E}[\varepsilon]\cdot\mathbb{E}_{\mathcal{D}}[f^* - \hat{f}] = 0$, because ε is independent of the training set and has mean zero.
Step 2: write $f^* - \hat{f} = (f^* - \bar{f}) + (\bar{f} - \hat{f})$ and square. The first bracket is the constant $-$Bias, the second has mean zero over training sets by the definition of $\bar{f}$, so the cross term vanishes again, leaving $\text{Bias}^2 + \text{Var}$.
Total: $\sigma^2 + \text{Bias}^2 + \text{Var}$ [on your sheet]. It is the same move twice: add and subtract a mean, and the cross term dies because a mean-zero quantity multiplies a constant.

How he asks it

Q. *You standardise all 10,000 rows, split 80/20, train and report 94% on the 20%. Is the number trustworthy?*
A. Not fully, and I want you to say exactly why. The scaler's means and standard deviations were computed with the test rows included, so the test rows helped shape the representation on which they are then graded; that is leakage, even if mild. For a scaler on 10,000 rows the effect is small, because two sample means differ little. For feature selection or model-based imputation done the same way, the effect can be several points, and with few rows it can be enormous. The fix costs nothing: split first, fit the scaler on the 8,000, apply it unchanged to the 2,000.

Q. *You tried 50 configurations on 200 validation rows and the best scored 91%. Why should you expect less in deployment, and what would reduce the discrepancy?*
A. Because 91% is the minimum of 50 noisy estimates, and the expected minimum of honest estimates lies below the best true risk. Put numbers on it: the noise is about $\sqrt{0.09 \cdot 0.91/200} \approx 0.02$, and the optimism for 50 candidates is of the order of $\sigma\sqrt{2\ln 50} \approx 2.8\sigma \approx 0.06$, so several points are plausible. You reduce it by enlarging the validation set or using cross-validation, which attacks the linear factor, and by searching

less, which attacks the logarithmic factor. To know the deployment number, evaluate the winner once on a test set that took no part in the 50 comparisons.

Q. *Prove that training error cannot be used to choose λ.*
A. The claim is that training risk is minimised at $\lambda = 0$. Penalised fitting is equivalent to ordinary fitting restricted to the family $\Omega(\theta) \leq c_\lambda$, by the soft-constraint argument; as λ grows the family shrinks and the families are nested; a minimum over a smaller family is never smaller. So the training risk is nondecreasing in λ and the training set would always answer "no regularisation". That is why λ is chosen on validation rows, and why the test set stays untouched for the final number.

Q. *A learning curve shows training error at 2% and validation error at 15% at every size from 1,000 to 50,000 rows, with no narrowing. What is your diagnosis?*
A. The table would say high variance, because the gap is large. But variance shrinks with N, and a gap that does not move across a fiftyfold increase in data is not sampling variance. It is a systematic difference between the training rows and the validation rows: a shortcut present in the training collection and absent from a grouped or temporal validation set, duplicates in the training rows, or label noise concentrated on one side. The response is not more data or regularisation but an audit of the split and of the features.

Q. *Why does leave-one-out cross-validation have high variance although each of its models uses almost all the data?*
A. Its N training sets differ by one row, so the N fitted models are nearly identical and their N held-out losses are strongly correlated. The average of correlated quantities does not shrink its variance like $1/N$; it is the ensemble formula $\sigma^2(\rho + (1-\rho)/M)$ with ρ close to one. Five or ten folds differ more from one another, so their average is more stable, at the price of a slight pessimism because each model sees 80 to 90% of the rows.

Q. *Give a counterexample to "averaging many models always reduces error".*
A. Take M copies of the same fitted model, or M models that all lack the same essential feature. Their errors at any input are identical, so $\rho = 1$ and the variance of the average is $\sigma^2(1+0) = \sigma^2$: exactly the error of one model. Averaging removes only the part of the error that differs between models; the shared part, which is bias, survives, and the floor $\rho\sigma^2$ is untouched by M.

Five mistakes

- Grading a model on the rows that chose it: training rows for the parameters, validation rows for the winning configuration, and the test set after it has been used to decide anything.
- Fitting preprocessing or feature selection on all rows, or outside the cross-validation folds; cutting a random split when deployment is in the future or on new entities.
- Choosing λ, k, depth or epochs on training error or on the test set; keeping the last epoch; tuning boosting's rate, rounds and depth separately.
- Confusing the capacity axis, the epoch axis and the data-size axis; expecting more data to remove bias.
- Reporting the winner's validation or cross-validation score as the deployment number; comparing candidates on different folds or on a validation set too small to separate them.

2 Chapter 3: From Fitting to Generalization

In Chapter 2 you learned how to fit: you take the training rows, you choose the parameters that make those rows look as good as possible, and you call the result a model. This chapter asks the question that fitting cannot answer: will the model also be good on the next case, the one it has never seen? We will give that worry a name, the generalisation gap, and we will prove that the score a model gets on its own training rows is always too flattering, because those rows chose the model and then graded it. We will see that the culprit is capacity: a flexible family can bend itself around the accidents of one sample. We will then split the error at a single input into three pieces, noise, bias and variance, and you will see that each piece answers to a different lever. The second half of the chapter is about the dials that control how flexible a model is allowed to be (penalties, early stopping, feature selection), and the rules that keep an evaluation honest (a split that looks like deployment, metrics that cannot hide a failure on the rare class, and no leakage). Keep one question in mind all the way through: which of the regularities I see in my sample will still be there on the next case?

Generalisation gap; why training risk is optimistic

What it is. Start with two averages of the same loss. The empirical risk $\widehat{R}_{\mathcal{D}}(f) = \frac{1}{N}\sum_i \ell(y^{(i)}, f(x^{(i)}))$ is the average loss on the rows you have; the population risk $R(f) = \mathbb{E}[\ell(Y, f(X))]$ is the average loss over the whole world the rows came from. The generalisation gap of the fitted model is simply the difference, $G(\hat{f}_{\mathcal{D}}) = R(\hat{f}_{\mathcal{D}}) - \widehat{R}_{\mathcal{D}}(\hat{f}_{\mathcal{D}})$ [on your sheet]. Notice that both terms refer to the *same* fitted model; only the source of the examples changes. We define it this way because the sentence "the fit did not carry over" needs a number and a sign, and this is that number.

The reason the gap exists is that the data play two roles. Follow the arrows: $\mathcal{D} \to \hat{f}_{\mathcal{D}} \to \widehat{R}_{\mathcal{D}}(\hat{f}_{\mathcal{D}})$. The rows first decide which model wins, and then the same rows are used to score the winner. Any accidental regularity in the sample is rewarded twice, once when it helps the model win and once when it is counted as a success. That is why the training score cannot be trusted as evidence about new cases.

Manipulate / measure. Here is the experiment I would run in front of you. We hold the family and the training rows fixed and fit once. Then we measure the risk twice: once on the training rows and once on rows that were never used, either held out before fitting or collected next month. You get two bars. The held-out bar is higher, and the difference between them is the gap. If you repeat this with a more flexible family, the gap grows; if you repeat it with more training rows, the gap shrinks. Those two movements are the whole story of the chapter.

Goes wrong. The tempting mistake is to read a low training loss as proof that the model works. It feels right because the loss is a real number computed on real data. But a low training loss only proves that the optimiser did its job; it says nothing about whether learning carried over. It overstates performance in one direction only, upward. You catch it by insisting on a held-out set: if the held-out number is far above the training number, the training number was flattering.

Versus. Students confuse "training risk" with "empirical risk". The way to keep them apart is to ask whether the model was fixed before the rows were seen. For a *fixed* model, the average loss on any fresh rows is an unbiased estimate of R. For the *fitted* model, the average on the rows that fitted it sits below $\min_{\mathcal{H}} R$; the derivation below makes this precise.

Example. A spam filter makes 2% errors on the emails it was trained on and 9% on the emails of the following month. Its gap is 0.07. Nothing about the filter changed between the two measurements; only the rows did.

Stable versus spurious regularities; the shortcut probe

What it is. Every sample contains regularities of two kinds. Some describe the relationship you actually want: the shape of the animal says "cow". Others are artefacts of how the data were collected: the grass says "cow" because the cows happened to be photographed on grass and the camels on sand. Here is the uncomfortable fact: both regularities fit every training row perfectly, so training performance cannot tell them apart. The way the dataset was assembled is therefore part of the learning method, not a neutral background. This idea explains the puzzle of a model with zero training loss and a huge gap: it learned a cue that is perfect in the collection and absent in the world.

Manipulate / measure. We take the fitted model and we hold it fixed. Now we change only the nuisance factor: we show the model a cow standing on sand, or we feed it flows from a network segment it has never seen, or from a new week. We measure accuracy on these probe cases and compare it with accuracy on the original split. If the probe accuracy drops, the model was using the shortcut. I would run two controls

alongside. First, shuffle the labels and retrain: performance must collapse to chance, and if it does not, something in the pipeline leaks the label. Second, delete the metadata columns and retrain: performance must *not* collapse, and if it does, the model was reading identifiers rather than behaviour.
Goes wrong. The tempting thing is to trust a random split, because the test rows are, after all, rows the model did not train on. But a shortcut present in every part of the collection is present in the test rows too, so the random-split number is inflated in exactly the same way as the training number. You catch it by evaluating under deployment-like conditions: a grouped split that holds out whole sites or devices, or a temporal split that holds out later weeks.
Versus. Students mix up shortcuts and leakage because both produce a number that is too good. A shortcut is a cue that is genuinely there in the collection but will not hold at deployment, like the grass. Leakage is a feature that will not even exist at decision time, like a field filled in after the investigation. Different causes, same inflated estimate.
Example. You weaken shortcuts at collection time: several cameras, sites and seasons, so that no nuisance factor lines up with the label. You augment only with transformations that preserve the label and look realistic: flipping a photo of a cat is fine, flipping the digit 6 is not, because it becomes a 9.

Inductive bias

What it is. A finite sample is compatible with infinitely many functions: functions that agree on every training row and disagree everywhere else. So learning cannot be just "read the data"; it needs assumptions that prefer some solutions over others off the sample. Those assumptions are the inductive bias, and they enter through four doors. The data and the representation decide which cues exist at all. The hypothesis space decides which relationships can be expressed. Regularisation decides which of the fits that explain the data are preferred. The optimiser decides which of many minimisers is actually reached. The lesson is that generalisation is never a property of the data alone; it succeeds when the assumptions you built in are still true where the model is used.
Goes wrong. The tempting attitude is to hope the data will speak for themselves. They cannot: without a bias, every unseen input is undetermined. And a bias that happens to be false at deployment, like "grass means cow", fails silently, with a perfect training score. You catch it only by probing, as in the previous card.
Versus. Students confuse this "bias" with the statistical bias of the bias–variance decomposition. Inductive bias is a *preference among functions*, something the designer chooses. Statistical bias, $\mathrm{Bias}(x) = \bar{f}(x) - f^*(x)$, is a *number at each input*, the systematic error of a procedure, which we trade against variance. Same word, two different objects.

Capacity, nested families, underfitting and overfitting, the validation U

What it is. Capacity is how varied the functions of a family $\mathcal{H}$ can be. It is set by the polynomial degree, the tree depth, the network width, the neighbourhood size k (and be careful: capacity grows as k *shrinks*), by regularisation, and by the data; it is not the parameter count alone. Families are nested when $\mathcal{H}_1 \subseteq \mathcal{H}_2$, every function of the smaller family being available in the larger one. For nested families the training error can only fall: $\min_{\mathcal{H}_{m+1}} \widehat{R}_{\text{train}} \leq \min_{\mathcal{H}_m} \widehat{R}_{\text{train}}$ [on your sheet], because the larger family can always reuse the smaller one's minimiser. Nothing of the sort holds for the population risk.
Underfitting means the family is too restricted to capture important structure, and you recognise it because training and validation errors are both high. Overfitting means the family is so flexible that it followed the accidents of this particular sample, and you recognise it because the training error is low while the validation error is much higher. The question is never whether the model is complicated in the abstract; it is whether its flexibility matches the evidence you have.
Manipulate / measure. Here is the validation curve. We hold the rows, the preprocessing and the metric fixed. We vary one knob, the degree, the depth, or $1/k$, over a grid. At each value we fit and we measure two numbers, training risk and validation risk. Look at the picture: the training curve falls monotonically, because of the nested-family argument. The validation curve falls at first, because the extra flexibility captures real structure, and then rises, because the extra flexibility starts capturing noise. The minimum of that U is the capacity your data can support, and if you collect more data the minimum moves to the right.
Goes wrong. Students compare families by training error, which always picks the largest family, and treat zero training error as an achievement, when with enough capacity it is guaranteed. Both mistakes inflate your opinion of the flexible model. You catch them by reading the validation curve and nothing else.
Example. Four points, $(0,1), (1,3), (2,2), (3,4)$, fitted by polynomials of degree 0, 1 and 3, give training risks 1.25, 0.45 and 0. Now a fifth point arrives, $(4,5)$. The constant 2.5 is off by 6.25. The line $0.8x + 1.3$

predicts 4.5 and is off by 0.25. The interpolating cubic $1+5.5x-4.5x^2+x^3$ passes through all four training points and predicts 15: off by 100. The model with the best training score is the worst one to deploy, because it bent itself around the noise of four points.

Bias and variance of a learning procedure

What it is. Fix one input x, say one particular house. Imagine training a thousand times on a thousand different training sets drawn from the same world, and recording the thousand predicted prices. Their average is $\bar{f}(x) = \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(x)]$. The bias asks whether that average is off from the true conditional mean $f^*(x) = \mathbb{E}[Y \mid X = x]$ in a consistent direction: $\text{Bias}(x) = \bar{f}(x) - f^*(x)$. The variance asks how widely the thousand predictions scatter around their own average: $\text{Var}(x) = \mathbb{E}_{\mathcal{D}}[(\hat{f}_{\mathcal{D}}(x) - \bar{f}(x))^2]$. And the squared error at x splits exactly: $\mathbb{E}[(Y - \hat{f}_{\mathcal{D}}(x))^2] = \sigma^2(x) + \text{Bias}^2 + \text{Var}$ [on your sheet]. Please notice that these are properties of the *procedure*, the family together with the fitting rule and the sample size, averaged over training sets. One fitted model is one draw; it has no variance. We define things this way because we want to separate three kinds of error that respond to different cures: systematic error, which more data cannot fix; sample sensitivity, which more data does reduce; and noise, which nothing fixes.

Manipulate / measure. We simulate a world where we know f^* and the noise σ, and we hold them and N fixed. We draw 500 training sets and refit the procedure each time. At each test input we now have a cloud of 500 predictions: the centre of the cloud minus $f^*(x)$ is the bias, the spread of the cloud is the variance. Now sweep the capacity: the squared bias falls, the variance rises, and their sum plus σ^2 traces exactly the U you saw on the validation curve. Then sweep N instead: the variance falls and the bias stays flat. That flat line is the most important thing in the picture.

Goes wrong. Two sentences I hear every year. "Doubling the data halves the bias": no, it halves the variance; the bias comes from the family and the features. "Moderate capacity gives low bias and low variance": at the bottom of the U both are positive, just small; zero bias and zero variance would mean the procedure returns the truth regardless of the sample, which means it already knew the answer.

Versus. Keep three reference points in mind. A 1-nearest-neighbour rule has small bias (it follows the data closely) and large variance (one noisy neighbour decides). A 50-nearest-neighbour rule has higher bias (it averages far from x) and variance of about $\sigma^2/50$. A constant model has zero variance and large bias.

Example. The simplest case makes the numbers exact. Estimate a mean from $N = 4$ draws with $\sigma^2 = 1$. The sample mean has bias 0 and variance $1/4 = 0.25$. The rule "predict 0" has bias $-\mu$ and variance 0. Half the sample mean has bias $-\mu/2$ and variance 0.0625. Add squared bias and variance: for $\mu = 1$ you get 0.25, 1 and 0.3125, so the unbiased rule wins; for $\mu = 0.2$ you get 0.25, 0.04 and 0.0725, so both biased rules win. When the truth is near the shrinkage target, a little bias buys a lot of variance. Regularisation is exactly this trade with a dial on it.

Three sources of excess error; double descent

What it is. Measure a learned model against the Bayes risk, the best anyone could do. The extra risk has three sources. Approximation error: the family cannot express a good enough function. Estimation error: the data are finite, so you cannot identify the best member of the family. Optimisation error: the algorithm does not reach a good solution of its own objective. Raising capacity lowers the first and raises the second; that tension is the U again. One more thing you should know: in modern, heavily overparameterised models the test error can peak at the interpolation threshold, where the number of parameters is about N, and then fall again beyond it [on your sheet]. This double descent depends on the regularisation, the optimiser, the geometry of the data and which of the many interpolating solutions is selected; a similar phenomenon can appear over training time.

Manipulate / measure. Keep increasing capacity well past the point where the training sample is fitted exactly, and keep measuring validation error instead of stopping. The classical U is the left half of the picture, not a law of nature.

Goes wrong. Assuming the U always holds, or ending a capacity sweep at the interpolation threshold because "it can only get worse". You catch it by looking: validation behaviour in large models must be examined, not assumed.

Versus. On a learning curve, approximation error is the plateau, the height that more data cannot lower; estimation error is the gap between the curves, the part that more data closes.

Regularised ERM as a soft constraint

What it is. Regularised empirical risk minimisation is $\hat{\theta}_\lambda \in \arg\min_\theta[\widehat{R}_{\text{train}}(\theta) + \lambda\Omega(\theta)]$ with $\Omega_2 = \|\theta\|_2^2$ (ridge) or $\Omega_1 = \|\theta\|_1$ (lasso) [on your sheet]. Read it as an instruction: fit the data *and* keep the parameters small. The strength λ is the price of one unit of penalty in units of training loss; at $\lambda = 0$ you have plain ERM, and for large λ any sizeable coefficient becomes too expensive. The key fact is that the penalised problem is the same as plain ERM over the smaller family $\{\theta : \Omega(\theta) \leq c\}$ with $c = \Omega(\hat{\theta}_\lambda)$ [on your sheet]. So regularisation reduces capacity without removing a single parameter: it removes functions from the contest, not numbers from the model. Capacity is about which functions can win, not how many numbers are stored.

Manipulate / measure. Standardise the features on the training rows and hold the rows fixed. Vary λ on a logarithmic grid, $10^{-3}, 10^{-2}, \ldots, 10^{2}$, and at each value measure training and validation risk. The training risk is smallest at $\lambda = 0$, by the nested-family argument, and climbs as λ grows; the validation risk is a U. To feel the two penalties, take coefficients $(3, -4)$: the ridge penalty is $9 + 16 = 25$, the lasso penalty is $3 + 4 = 7$.

Goes wrong. Four things people do. They choose λ on the training data, which always answers zero. They penalise unstandardised features, so that a feature measured in millimetres is penalised a thousand times more than the same feature in metres. They penalise the intercept, which is not a flexibility knob at all. They search λ on a linear grid, when the useful range spans orders of magnitude.

Versus. There are three dials for the single quantity "effective capacity", and students treat them as unrelated. A penalty is a soft constraint: it trades fit against size at the rate λ. Restricting the family (degree, depth) is a hard constraint. Early stopping limits how far the optimiser is allowed to travel. All three shrink the set of functions that can win.

Ridge in one variable: smooth shrinkage

What it is. For the model $f_w(x) = wx$ the ridge solution is $\hat{w}_\lambda = \dfrac{\sum_i x^{(i)} y^{(i)}}{\sum_i (x^{(i)})^2 + N\lambda}$ [on your sheet], which you should read as the ordinary least-squares slope multiplied by the factor $\sum x^2 / (\sum x^2 + N\lambda)$. That factor lies in $(0, 1]$. So ridge scales every coefficient toward zero, smoothly, and never exactly to zero for finite λ. As $\lambda \to \infty$ all penalised coefficients vanish and the model is reduced to its intercept.

Manipulate / measure. Take $\sum x^2 = 14$, $\sum xy = 19$, $N = 4$. At $\lambda = 0$ the slope is $19/14 = 1.357$; at $\lambda = 1$ it is $19/18 = 1.056$; at $\lambda = 10$ it is $19/54 = 0.352$. Plot the coefficient against $\log \lambda$ and you see a path that bends toward zero without ever touching it.

Goes wrong. Expecting ridge to select features. It keeps all of them, with smaller weights; and when two features are identical it splits the weight equally between them, because $a^2 + b^2$ is smallest at $a = b$ for a fixed sum.

Versus. Use L_2 when you believe many features each contribute a little; use L_1 when you believe few features matter and you want to know which ones.

Lasso in one variable: soft threshold, exact zeros, the diamond

What it is. Minimising $\frac{1}{2}(w - z)^2 + \lambda|w|$, where z is where the unpenalised fit would put the coefficient, gives $\hat{w} = \text{sign}(z)\max(|z| - \lambda, 0)$ [on your sheet]. In words: move the coefficient toward zero by exactly λ, and if it would cross zero, stop at zero. That is why the lasso produces exact zeros and ridge does not. Geometrically, the constraint region $\|\theta\|_1 \leq c$ is a diamond whose corners sit on the coordinate axes, and an optimum very often lands on a corner, where some coordinates are exactly zero; the slides' example moves the unconstrained optimum $(0.6, 2.2)$ to the lasso solution $(0, 1.5)$, dropping the first feature entirely. The L_2 region is a disc with no corners, so nothing special happens on the axes. Because of this, the lasso performs feature selection as part of fitting; we call it embedded selection.

Manipulate / measure. With $\lambda = 0.5$: $z = 2.2$ becomes 1.7, $z = 0.4$ becomes 0, $z = -3$ becomes -2.5. With $\lambda = 1.5$, $z = 1.2$ becomes 0. If you trace the whole regularisation path, the coefficients hit zero one after another as λ grows, and the order in which they die is a ranking of the features.

Goes wrong. Trusting a lasso's selected subset when features are correlated. Among near-duplicates the lasso keeps one and drops the others, and which one survives can change from one fold to the next. The fix is to report groups of correlated features, not individual survivors. One more caution: in several dimensions the lasso solution is not obtained by thresholding each coordinate separately, which is why the clean result is stated in one variable.

Versus. Ridge multiplies by a factor and never reaches zero, keeping everything. Lasso subtracts λ and stops at zero, producing exact zeros, but it is unstable under correlation.

Example. A bank with 300 candidate variables: ridge keeps all 300 with small weights and gives no story; the lasso at the validation-chosen λ keeps 18 variables that the risk team can audit and that are cheap to collect at application time.

Early stopping

What it is. An iterative optimiser produces a sequence $\hat{\theta}_0, \hat{\theta}_1, \ldots$. The early iterations learn the strong, recurring patterns; the later ones start fitting exceptions and noise. Early stopping keeps the checkpoint with the lowest validation risk, $t^* = \arg\min_t \widehat{R}_{\text{val}}(\hat{\theta}_t)$ [on your sheet]. A patience p means you keep training for p more epochs without improvement before you stop, and you keep the best checkpoint seen, so that a temporary wobble of a noisy validation curve does not stop you too soon. The deep point is that training time controls effective capacity while the parameter count stays fixed. It is regularisation for free in any iterative method.

Manipulate / measure. Hold everything fixed except the number of epochs. Train on, saving a checkpoint after every epoch, and record training and validation loss each time. The training loss falls throughout; the validation loss falls, reaches its bottom at t^*, and then rises. When you finally stop, you roll back to the checkpoint of t^*.

Goes wrong. Keeping the last epoch instead of the best one; stopping on the first wobble without patience; and stopping on the test loss, which quietly turns the test set into a validation set. Be aware too that patience can still miss a later decrease if the problem shows epoch-wise double descent.

Versus. This is the same U as the capacity sweep, with epochs on the horizontal axis instead of degree. Students sometimes think it is a different phenomenon; it is not.

Choosing a hyperparameter: the validation curve, and regularisation in practice

What it is. A hyperparameter is any knob that training error would turn all the way to maximal capacity: λ, k, the degree, the depth, the number of epochs, the decision threshold, the feature subset. Because training error cannot be trusted with them, they are set on validation rows. There are two nested problems: the inner one fits $\hat{\theta}_h$ on the training rows for a given configuration h, and the outer one chooses $\hat{h} \in \arg\min_h \widehat{R}_{\text{val}}(\hat{f}_h)$ [on your sheet], or the same thing by cross-validation. In practice you standardise on the training rows before penalising, you do not penalise the intercept, you search λ on a logarithmic grid, you prefer L_1 when few features matter and you want to know which and L_2 when many contribute a little, and you use early stopping in any iterative method with the best checkpoint kept.

Manipulate / measure. We hold all the other knobs, the preprocessing and the rows fixed, and we vary one knob over its grid, measuring training and validation risk. You read $\hat{h}$ at the bottom of the validation U. If two configurations are within one standard error of each other, treat them as tied and prefer the simpler. If the minimum sits at the edge of the grid, extend the grid. Once $\hat{h}$ is chosen, refit on training plus validation rows, and report on the test set once.

Goes wrong. A fine grid on a tiny validation set reads noise: the standard error of an accuracy near 0.9 on 100 rows is about 0.03, so differences of a point or two mean nothing. And tuning on the test set makes it a validation set, after which you have no honest number left.

Versus. Remember the direction of each knob. Capacity grows with degree, depth and epochs, but it grows with *smaller* λ and *smaller* k. On the λ and k axes the U is mirrored, and that is where students read curves backwards.

Feature selection: 2^d subsets, filters, greedy searches, embedded

What it is. Choosing which inputs to use changes the representation. Done well, it removes irrelevant variables (which only add variance), redundant variables, and variables that will not be available at decision time, and it makes the model easier to explain. The difficulty is combinatorial: with d features there are 2^d subsets [on your sheet], and 2^{30} is already 1,073,741,824, so exhaustive search is out. A filter scores each variable on its own, for instance $s_j = |\widehat{\text{Corr}}(X_j, Y)|$, and keeps the top scorers; it needs no fits but is blind to nonlinear relationships, to interactions and to redundancy between the variables it keeps. Forward selection starts from nothing and repeatedly adds the feature with the largest validation gain; backward elimination starts from everything and removes the least useful; both cost about $d(d+1)/2$ fits [on your sheet] and both are greedy, so an early choice can block a better combination later, and backward elimination is impossible when $d > N$. The embedded route is the lasso: one regularisation path does the selection, with the instability under correlation you already know.

Manipulate / measure. Hold the rows fixed, vary the subset according to the chosen search, measure the validation risk at each step, and stop where the validation gain turns negative.

Goes wrong. The classic error is to run the selection on all the rows and then cross-validate the classifier. The validation folds have then helped choose the representation they are scored on, which is leakage, and the estimate is inflated, sometimes to near perfection. Selection belongs inside every fold [on your sheet].
Versus. Here is why filters cannot be trusted alone. Let X_1, X_2 be independent ± 1 coins and $Y = X_1 X_2$. Each feature has correlation 0 with Y, so a filter drops both; together they determine Y exactly. Weak marginal correlation never proves that a feature is useless jointly with others.
Example. Take 300 rows and 5,000 features that have nothing to do with the labels. Choose the top 20 by correlation on all rows, then run 10-fold cross-validation: you get above 90% accuracy from pure noise. Do the selection inside the folds and you get about 50%, which is the truth.

The three subsets; the winner's curse; K-fold cross-validation

What it is. The training rows fit θ. The validation rows choose the configuration $\hat{h} \in \arg\min_{h \in \mathcal{C}} \widehat{R}_{\text{val}}(\hat{f}_h)$ [on your sheet], where a configuration is everything you might vary: features, family, λ, k, threshold, epoch. The test rows report once. Now the subtle point. Each validation score is honest for its own candidate, because that candidate was fitted without the validation rows. But the *minimum* over many candidates is optimistic: $\mathbb{E}[\min_h \widehat{R}_{\text{val}}(\hat{f}_h)] \leq \min_h R(\hat{f}_h)$ [on your sheet], with strict inequality as soon as the estimates are noisy. Picking the best of many noisy numbers partly picks the luckiest noise. K-fold cross-validation is the way to use scarce data: $\widehat{R}_{\text{CV}}(h) = \frac{1}{K} \sum_j \widehat{R}^{(j)}_{\text{val}}(\hat{f}_h^{(-j)})$ [on your sheet], where each row is validated exactly once and used for fitting $K - 1$ times, at a cost of K fits per configuration. The whole pipeline, scaler and selector included, is refitted inside each fold, and afterwards you refit $\hat{h}$ on all the development rows.
Manipulate / measure. Try n random rules on labels that are pure coin flips and keep the best validation score. Watch it rise with n, 50%, 53%, 56% and so on, while the winner's test score stays at 50%. Nothing was learned, yet the validation number improved. The optimism is about $\sigma\sqrt{2 \ln n}$: linear in the noise σ of the estimate and only logarithmic in n, so a small validation set hurts more than a long search.
Goes wrong. Reporting the winner's validation or cross-validation score as the final number; continuing to develop after looking at the test score, at which point the test set has become one more validation set; using random folds on grouped or temporal rows; deploying one of the fold models instead of refitting on all development rows.
Versus. A single validation set is one slice, noisy if small. Cross-validation holds every row out once, so it is less noisy, but it is still optimistic for the winner. The test set chose nothing, which is why it is honest, and only once. Nested cross-validation, with an outer loop for estimation and an inner loop for selection, is honest for the whole procedure "select, then fit" when you cannot afford a test set.

Splits: random, stratified, grouped, temporal

What it is. The rule is to hold out the kind of novelty that deployment will bring [on your sheet]. A random split assigns rows independently; it is credible only when the new cases really are independent draws from the same population. A stratified split is random but preserves the class proportions in every subset, which stabilises the counts of a rare class; it must never scatter a group across subsets just to match proportions. A grouped split keeps every user, device, patient or hospital entirely in one subset, so the model is evaluated on groups it has never seen. A temporal split trains on the past and validates and tests on later rows, which preserves the direction of prediction and reveals drift.
Manipulate / measure. Hold the model, the features and the metric fixed, and vary only the split rule on flows from 50 devices over 6 months. With a random split the model scores 99.5%, because it learned that device 17 is the compromised laptop. With a grouped split, holding out 10 devices, it must recognise attack behaviour and scores 96%. With a temporal split, training on months 1 to 4 and testing on months 5 and 6, during which a new attack tool appeared, it scores 90%. Only the last number tells you what will happen next month. The others are not wrong arithmetic; they answer a question nobody asked.
Goes wrong. A random split for "will this customer churn next quarter", which is a temporal question (features from before, outcome from after), or for "does the model work on a hospital it never saw", which is a grouped question. In both cases the number is inflated by identity and period shortcuts, and you catch it by asking what kind of new case deployment will face.
Versus. Grouped answers "new entities"; temporal answers "the future". When deployment is both, combine them.

Metrics from the confusion matrix; accuracy hides minority failure

What it is. Call the rare class positive and count true positives, false negatives, false positives and true negatives. Recall, $\mathrm{TP}/(\mathrm{TP}+\mathrm{FN})$, asks: of the real attacks, what fraction did we catch? Precision, $\mathrm{TP}/(\mathrm{TP}+\mathrm{FP})$, asks: of the alarms we raised, what fraction were real? Specificity, $\mathrm{TN}/(\mathrm{TN}+\mathrm{FP})$, asks: of the benign flows, what fraction did we leave alone? Balanced accuracy is the average of recall and specificity and is immune to imbalance. $F_1 = 2PR/(P+R)$ is the harmonic mean of precision and recall; it is high only when both are, and it ignores true negatives [on your sheet]. Now the trap: if a fraction π of the data is positive, the classifier that always says "negative" has accuracy $1-\pi$ and recall 0 [on your sheet]. Its specificity is 1, so its balanced accuracy is 0.5, and that is the number that exposes it.

Manipulate / measure. Hold the fitted model fixed and vary the decision threshold on validation rows, measuring recall, precision, false alarms per 100,000 benign flows and the expected cost. Recall falls and precision rises as the threshold increases, and the expected cost is lowest near $t = C_{\mathrm{FP}}/(C_{\mathrm{FP}}+C_{\mathrm{FN}})$ when the probabilities are calibrated. The threshold is a hyperparameter: validation chooses it, the test set reports it.

Goes wrong. Reporting accuracy when $\pi = 0.01$: you get 99% with no attack ever detected. And choosing the metric after seeing the results, which is selection in disguise.

Example. A detector on 10,000 flows with 50 attacks gives TP 40, FN 10, FP 90, TN 9860. Accuracy is 0.990; recall 0.80; precision $40/130 = 0.31$; specificity 0.991; balanced accuracy 0.895; $F_1 = 0.44$. Note that 0.44 is well below the average of 0.31 and 0.80, which is 0.56: the harmonic mean is dragged toward the worse rate, and that is its purpose. Another case: TP 30, FN 70, FP 0 gives precision 1.0, recall 0.3, $F_1 \approx 0.46$.

Leakage; preprocessing inside the pipeline

What it is. Leakage is any use, during development, of information that will not be available when the new decision is made [on your sheet]. Its usual forms are duplicates across subsets; the same device, session or patient in training and test; features created after the outcome, such as "ticket closed as intrusion" or "number of analyst comments"; preprocessing fitted with validation or test rows; and hyperparameters chosen on test results. Every form gives an unrealistically favourable estimate. The fix for preprocessing is mechanical: the training rows estimate $\hat{\mu}_j$ and $\hat{\sigma}_j$, and likewise the imputer, the selector, any projection and any resampling, and the *same* fitted values are reused unchanged on validation, test and deployment rows [on your sheet]. Inside cross-validation each fold fits them on its own training portion.

Manipulate / measure. Compare two pipelines. The leaking one: all data, then estimate μ and σ, then split. The valid one: split, then estimate on the training rows, then reuse. And for every feature ask one question: at what moment does its value become known, relative to the moment of the decision?

Goes wrong. Standardising all 10,000 rows before splitting is a small leak, because two sample means differ little. Selecting features or fitting an imputer on all rows is a large leak. Shuffling a time series before splitting is a total leak, because the model sees the future of the series it is tested on.

Versus. Leakage is information that will not exist at decision time; a shortcut is a cue that exists but will not hold. Different causes, same inflated number.

Learning curves and the diagnostic table

What it is. A learning curve varies the amount of training data and plots training and validation performance against N [on your sheet]. Read it in four patterns. Both curves high and flat: underfitting, which is bias, so improve the representation or the capacity. Training low and validation high: variance, so regularise, reduce capacity, or collect more representative data. The curves close together and both still falling: data-limited, more examples will help. Test performance degrading over calendar time: distribution shift, which calls for time-aware evaluation and retraining. Treat curves as evidence, not proof, and read them together with domain knowledge.

Manipulate / measure. Hold the configuration, the validation rows and the metric fixed. Vary N by nested subsamples, 10%, 20% and so on up to 100%, with several random draws at each size, and measure training and validation risk. The training curve rises toward a plateau and the validation curve falls toward the same plateau. That plateau is approximation error plus noise; the gap between the curves is estimation error.

Goes wrong. Confusing the three horizontal axes, capacity for the validation curve, epochs for the training curve and data size for the learning curve, and expecting more data to lower the plateau. More data closes the gap; it does not move the plateau.

Versus. A gap that refuses to narrow across a fiftyfold increase in N is not variance, because variance shrinks with N. It is a split or shortcut problem: a systematic difference between training and validation

rows.

Lab 1 mapped to the chapter

What it is. The lab audits the labelled-flow dataset for imbalance, missing or non-finite values, duplicates, conflicting labels and redundancy. It pre-declares random-row, grouped and chronological protocols, which is the splits card. It fits baselines, majority class, a depth-3 tree, logistic regression and kNN, under each protocol. It probes metadata shortcuts and shuffled labels, which is the shortcut probe. It selects the decision threshold on validation, which is the selection card. And it reports test recall, false-positive rate and false alarms per 100,000 benign flows, which is the metrics card.
Goes wrong. Any model that does not beat the majority-class baseline under the deployment-like protocol has learned nothing usable, whatever its random-split accuracy says.
Example. Should the team label 20,000 more flows? Validation recall at 1,000 up to 8,000 rows reads $0.71, 0.78, 0.83, 0.86, 0.87, 0.875, 0.878, 0.88$, with training recall near 1. That is a wide gap closing slowly and a curve that has reached a plateau: doubling the data would buy perhaps 0.01. Regularise instead, with larger leaves and fewer features per split, and work on the features.

What each lever does to bias, variance and the curves

Lever	Bias	Variance	What you see
more capacity (degree, depth, $1/k$)	down	up	training falls, validation U
more data N	unchanged	down	learning-curve gap closes, plateau fixed
L_2 penalty	up (shrinkage)	down	all coefficients scaled, none zero
L_1 penalty	up	down	coefficients hit zero in order; selection
early stopping	up slightly	down	validation loss bottoms at t^*
feature selection	down if relevant features kept	down	must run inside the folds
better features, better split	attack the plateau	attack shortcuts	the only levers against noise and shortcuts

Training risk of a fitted model is optimistic (the sandwich)

Let $\hat{f}_{\mathcal{D}}$ minimise $\widehat{R}_{\mathcal{D}}$ over $\mathcal{H}$ and let $f^*_{\mathcal{H}}$ minimise R over $\mathcal{H}$. (1) Because $\hat{f}_{\mathcal{D}}$ is the minimiser of the empirical risk, $\widehat{R}_{\mathcal{D}}(\hat{f}_{\mathcal{D}}) \le \widehat{R}_{\mathcal{D}}(f^*_{\mathcal{H}})$ on every sample; this is the only place where minimisation is used. (2) Because $f^*_{\mathcal{H}}$ is a fixed function that did not look at the sample, $\mathbb{E}_{\mathcal{D}}[\widehat{R}_{\mathcal{D}}(f^*_{\mathcal{H}})] = R(f^*_{\mathcal{H}})$: an average of i.i.d. losses is an unbiased estimate of the expected loss. (3) Because $f^*_{\mathcal{H}}$ is the best member of the family, $R(\hat{f}_{\mathcal{D}}) \ge R(f^*_{\mathcal{H}})$ for every sample. Chain the three: $\mathbb{E}_{\mathcal{D}}[\widehat{R}_{\mathcal{D}}(\hat{f}_{\mathcal{D}})] \le \min_{\mathcal{H}} R \le \mathbb{E}_{\mathcal{D}}[R(\hat{f}_{\mathcal{D}})]$, hence $\mathbb{E}[G] \ge 0$ [on your sheet]. The training score sits below the best achievable risk, and the true risk of the fitted model sits above it; that is why the rows that chose the model cannot grade it.

Bias–variance decomposition

Assume $Y = f^*(X) + \varepsilon$ with $\mathbb{E}[\varepsilon \mid x] = 0$ and $\mathrm{Var}(\varepsilon \mid x) = \sigma^2(x)$, and assume the training set is independent of the test pair. Step 1, split off the noise: write $Y - \hat{f}_{\mathcal{D}}(x) = \varepsilon + (f^* - \hat{f}_{\mathcal{D}})$ and square. The term $\mathbb{E}[\varepsilon^2]$ is σ^2. The cross term is $\mathbb{E}[\varepsilon] \cdot \mathbb{E}_{\mathcal{D}}[f^* - \hat{f}_{\mathcal{D}}]$, which is allowed because ε is independent of the training set, and it is zero because ε has mean zero. Step 2, split around the average prediction: write $f^* - \hat{f}_{\mathcal{D}} = (f^* - \bar{f}) + (\bar{f} - \hat{f}_{\mathcal{D}})$. The first bracket is the constant $-$Bias; the second has $\mathbb{E}_{\mathcal{D}}$-mean zero by the definition of $\bar{f}$. Square and take $\mathbb{E}_{\mathcal{D}}$: the constant squared gives Bias^2, the cross term vanishes, and the second bracket squared averages to Var. Result: $\sigma^2 + \mathrm{Bias}^2 + \mathrm{Var}$ [on your sheet]. It is the same move twice: add and subtract a mean, square, and watch the cross term die because a mean-zero quantity multiplies a constant.

Training error favours capacity (nested families)

Let $\mathcal{H}_m \subseteq \mathcal{H}_{m+1}$ and let f_m minimise $\widehat{R}_{\text{train}}$ over $\mathcal{H}_m$. Since f_m also belongs to $\mathcal{H}_{m+1}$, it is one of the candidates there, so the minimum over the larger family cannot exceed its value: $\min_{\mathcal{H}_{m+1}} \widehat{R}_{\text{train}} \leq \widehat{R}_{\text{train}}(f_m) = \min_{\mathcal{H}_m} \widehat{R}_{\text{train}}$ [on your sheet]. So training error is nonincreasing in capacity, whether the knob is degree, depth, $1/k$, $1/\lambda$ or epochs. The population risk has no such monotonicity, and that is why the validation curve, never the training curve, chooses the capacity.

Why the two learning curves converge to the same limit

At every training size N the sandwich holds: $\mathbb{E}[\widehat{R}_{\text{train}}(\hat{f}_N)] \leq R(f^*_{\mathcal{H}}) \leq \mathbb{E}[R(\hat{f}_N)]$. For small N the minimiser can fit the sample's accidents, so the left term sits far below $R(f^*_{\mathcal{H}})$ and the right term far above it. For large N the empirical risk of every member of the family approaches its population risk, so the fitted model approaches $f^*_{\mathcal{H}}$ and both outer terms approach $R(f^*_{\mathcal{H}})$. Therefore the training curve rises to the limit from below and the validation curve falls to it from above. The limit is approximation error plus noise, set by the family and the features and untouched by N; the gap at finite N is estimation error, which closes with N and grows with capacity.

A penalty is a soft constraint; training risk is minimal at $\lambda = 0$

Put $c = \Omega(\hat{\theta}_\lambda)$ and take any θ with $\Omega(\theta) \leq c$. By optimality of $\hat{\theta}_\lambda$ for the penalised problem, $\widehat{R}(\hat{\theta}_\lambda) + \lambda c \leq \widehat{R}(\theta) + \lambda\Omega(\theta)$, and since $\Omega(\theta) \leq c$ the right side is at most $\widehat{R}(\theta) + \lambda c$. Cancel λc: $\widehat{R}(\hat{\theta}_\lambda) \leq \widehat{R}(\theta)$. So no feasible point beats $\hat{\theta}_\lambda$, which is itself feasible, and $\hat{\theta}_\lambda$ solves $\min \widehat{R}$ subject to $\Omega \leq c$ [on your sheet]. The constrained families are nested, $c_1 < c_2 \Rightarrow \mathcal{H}_{c_1} \subseteq \mathcal{H}_{c_2}$, and a larger λ gives a smaller c. A minimum over a smaller set is at least as large, so $\widehat{R}_{\text{train}}(\hat{\theta}_\lambda)$ is nondecreasing in λ and smallest at $\lambda = 0$. The training data would always answer "no regularisation", which is why λ is chosen on validation.

Ridge in one variable

The objective is $J(w) = \frac{1}{N}\sum_i (y^{(i)} - wx^{(i)})^2 + \lambda w^2$, a convex quadratic in w, so setting the derivative to zero gives the minimum. $J'(w) = -\frac{2}{N}\sum_i x^{(i)}(y^{(i)} - wx^{(i)}) + 2\lambda w = 0$. Rearranging collects w: $w\big(\sum_i (x^{(i)})^2 + N\lambda\big) = \sum_i x^{(i)} y^{(i)}$. Hence $\hat{w}_\lambda = \dfrac{\sum xy}{\sum x^2 + N\lambda} = \hat{w}_{\text{LS}} \cdot \dfrac{\sum x^2}{\sum x^2 + N\lambda}$ [on your sheet], with a factor in $(0, 1]$. The coefficient is shrunk but never zero, because the derivative of λw^2 vanishes at $w = 0$, so nothing holds the coefficient there.

Lasso in one variable: soft threshold

Minimise $\frac{1}{2}(w - z)^2 + \lambda|w|$ and take $z > 0$; the other sign is symmetric. On $w \geq 0$ the objective is $\frac{1}{2}(w - z)^2 + \lambda w$, a parabola whose unconstrained minimiser is $w = z - \lambda$. If $z > \lambda$ this is positive and is the minimum on that side; if $z \leq \lambda$ the parabola is increasing on $[0, \infty)$, so the minimum on that side is at $w = 0$. On $w < 0$ the objective is $\frac{1}{2}(w - z)^2 - \lambda w$, whose unconstrained minimiser $z + \lambda$ is positive, so the objective decreases toward 0 and no negative w beats $w = 0$. Hence $\hat{w} = \max(z - \lambda, 0)$ for $z > 0$, and in general $\hat{w} = \text{sign}(z)\max(|z| - \lambda, 0)$ [on your sheet]: exactly 0 whenever $|z| \leq \lambda$, because the kink of $|w|$ at 0 absorbs any pull of size up to λ.

The best of many validation scores is optimistic

Each $\widehat{R}_{\text{val}}(\hat{f}_h)$ is unbiased for $R(\hat{f}_h)$, because the model was fitted without the validation rows. For any fixed h_0, the minimum over all h is at most the value at h_0: $\min_h \widehat{R}_{\text{val}}(\hat{f}_h) \leq \widehat{R}_{\text{val}}(\hat{f}_{h_0})$ on every sample. Take expectations: $\mathbb{E}[\min_h \widehat{R}_{\text{val}}(\hat{f}_h)] \leq R(\hat{f}_{h_0})$. This holds for every h_0, so it holds for the best one: $\leq \min_h R(\hat{f}_h)$ [on your sheet]. The inequality is strict as soon as the estimates are noisy and no single h wins on every sample; the size is about $\sigma/\sqrt{\pi}$ for two equal candidates and $\sigma\sqrt{2\ln n}$ for n [on your sheet]. This is the reason you keep one untouched test set.

Accuracy hides minority failure

Let a fraction π of the data be positive and consider the classifier that always predicts "negative". It is correct on every negative and wrong on every positive, so its accuracy is $1 - \pi$ [on your sheet]. It detects no positive, so TP = 0 and recall = $0/(0+\text{FN}) = 0$; its specificity is 1; its balanced accuracy is $(0+1)/2 = 0.5$;

its precision is undefined (0/0) and its F_1 is 0. With $\pi = 0.01$ this is 99% accuracy and not a single attack detected; the balanced accuracy of 0.5 is what gives it away.

F_1 lies between the worse rate and the average

Take $F_1 = 2PR/(P+R)$ with $P, R \in (0, 1]$. Lower bound: if $P \le R$ then $F_1 = P \cdot \frac{2R}{P+R} \ge P$, because $2R \ge P + R$. Upper bound: $\frac{P+R}{2} - \frac{2PR}{P+R} = \frac{(P+R)^2-4PR}{2(P+R)} = \frac{(P-R)^2}{2(P+R)} \ge 0$, with equality only when $P = R$. So $\min(P, R) \le F_1 \le \frac{P+R}{2}$. Precision 0.31 and recall 0.80 give 0.44, not 0.56: a great recall cannot hide a poor precision, which is exactly what a harmonic mean is for.

How he asks it

Q. *Prove that the expected generalisation gap of an empirical risk minimiser is nonnegative and say where minimisation is used.*
A. The claim is $\mathbb{E}[\widehat{R}_{\mathcal{D}}(\hat{f}_{\mathcal{D}})] \le \mathbb{E}[R(\hat{f}_{\mathcal{D}})]$. Let $f^*_{\mathcal{H}}$ be the population minimiser in the family. Minimisation is used exactly once: because $\hat{f}_{\mathcal{D}}$ minimises the empirical risk, $\widehat{R}_{\mathcal{D}}(\hat{f}_{\mathcal{D}}) \le \widehat{R}_{\mathcal{D}}(f^*_{\mathcal{H}})$ on every sample. Taking expectations turns the right side into $R(f^*_{\mathcal{H}})$, because $f^*_{\mathcal{H}}$ is fixed and an average of i.i.d. losses is unbiased for a fixed function. Finally $R(f^*_{\mathcal{H}}) \le R(\hat{f}_{\mathcal{D}})$ pointwise, because $f^*_{\mathcal{H}}$ is the best in the family. Chaining the three gives the sandwich. What it means: a model's own training score is not evidence about new cases, and that is the whole reason validation sets exist.

Q. *A model scores 100% on a collection where all cows stand on grass and all camels on sand. Why is this uninformative, and which experiment reveals the problem?*
A. It is uninformative because two cues, shape and background, predict every training label, and training performance cannot separate them: both give zero loss. Only inputs where the two cues disagree can. So the experiment is to hold the animal fixed and manipulate the background: collect or construct cows on sand and camels on grass and measure accuracy on them; a collapse shows the model used the grass. I would add a label-shuffle control, which must fall to chance, and a grouped or site-wise split. The fix is at collection time, by varying the background so that it no longer lines up with the label.

Q. *A procedure has* Bias $= 0.3$, Var $= 0.05$, $\sigma^2 = 0.1$. *Expected squared error, and which part each lever reduces?*
A. The expected squared error is $0.1 + 0.3^2 + 0.05 = 0.1 + 0.09 + 0.05 = 0.24$. More data acts on the 0.05, because variance shrinks roughly like $1/N$ for a stable procedure. A richer family acts on the 0.09, because it can move the average prediction closer to the truth, but it will raise the variance while doing so. Nothing acts on the 0.1 except new information in the features, because that part is the randomness of Y that x does not explain.

Q. *Why can ridge never set a coefficient exactly to zero while the lasso can?*
A. Ridge gives $\hat{w}_\lambda = \hat{w}_{\text{LS}} \cdot \sum x^2 / (\sum x^2 + N\lambda)$, and the factor is strictly positive, so the coefficient keeps its sign and is only scaled. The deeper reason is that the derivative of λw^2 at $w = 0$ is zero: there is no force holding the coefficient at zero. The lasso penalty $|w|$ has a kink at zero whose subgradient is the whole interval $[-\lambda, \lambda]$, so whenever the pull of the data, $|z|$, is at most λ, the kink absorbs it and the solution is exactly zero, which is the soft threshold $\text{sign}(z)\max(|z| - \lambda, 0)$.

Q. *With 300 rows and 5,000 features a student ranks features by correlation on all rows, keeps 20, then 10-fold cross-validates logistic regression: 97%. The labels are unrelated to the features. Explain.*
A. With 5,000 features and only 300 rows, some features will correlate strongly with the labels by pure chance, and ranking on all rows picks out exactly those lucky ones. Every cross-validation fold then scores rows that already took part in choosing the features, so the validation rows are not unseen by the selection step: the fold's labels leaked into the representation it is scored on. If the selection were repeated inside each fold on the 270 training rows, different lucky features would be chosen each time and the validation folds would score about 50%, which is the truth.

Q. *Two identical features enter a ridge model and a lasso model. What does each do, and why does it matter for reading a lasso's selected subset?*
A. Ridge splits the weight equally between them, because for a fixed sum $a + b$ the penalty $a^2 + b^2$ is smallest at $a = b$. The lasso is indifferent among all same-sign splits, because $|a| + |b|$ is then constant, so it may keep either feature and drop the other, and which one survives can change from one fold or one sample to the next. A lasso's selected subset is therefore an unstable ranking under correlation; the honest report is a group of correlated features, or the path across folds, not a single survivor.

Q. *A detector reports 99% accuracy on data with 1% attacks. What can it mean, what is the balanced accuracy of the trivial classifier, and which metrics do you ask for?*

A. The classifier that always says "benign" has accuracy $1 - \pi = 0.99$ and recall 0, so 99% accuracy is consistent with a detector that never fires. Its specificity is 1 and its recall is 0, so its balanced accuracy is 0.5, which exposes it. Ask for recall, the fraction of attacks caught, and for precision or false alarms per 100,000 benign flows, which is what each alarm costs an analyst; F_1 summarises the two but ignores true negatives, so say which one matters for the deployment.

Five mistakes

- Comparing families or choosing λ, k, depth or epochs by training error: with nested families it always picks maximal capacity.
- Speaking of the bias or variance of one fitted model, or expecting more data to remove bias when it only removes variance.
- Penalising unstandardised features or the intercept; expecting ridge to select features or the lasso to be stable under correlation; keeping the last epoch instead of the best.
- Selecting features or fitting the scaler on all rows before the split or outside the folds; reporting the winner's validation score as the final number.
- A random split for the future or for new entities; accuracy on imbalanced data; features whose value is known only after the outcome.

3 Chapter 4: Discriminative and Generative Classification

Let me start by telling you what this chapter is really about, because the titles hide it. In Chapter 2 you learned that once you know the posterior $p(y = c \mid x)$, the decision is a one-line rule: declare the class with the smallest expected cost. So the hard part of classification is not deciding, it is estimating that posterior from labelled examples. *There are exactly two ways to do it. The discriminative way fits the posterior directly: you build a score from the features, squash it into a probability, and train it to predict labels (logistic regression, neural networks). The generative way tells a story of how each class produces its inputs, fits that story class by class, and then reverses it with Bayes' rule (Gaussian class models, LDA and QDA, Naive Bayes). Every method you will meet is a choice of what you assume about the data, how many numbers you must estimate from it, and what you can and cannot ask the fitted model afterwards: a boundary, a probability, a plausibility, a correction when the prior changes. Keep that one question in mind throughout: "what did this method assume, and what did that assumption buy or cost?"*

The two routes to the posterior

What it is. A discriminative model fits $p_\theta(y = c \mid x)$ directly. Think of it as learning *where the label changes* as you move through feature space; it never asks what the inputs look like, only which side of the boundary they fall on. A generative classifier instead fits the prior $\pi_c = p(y = c)$ (how common each class is) and the class-conditional $p_\theta(x \mid y = c)$ (what inputs class c produces), and then obtains the posterior through Bayes' rule, $p(c \mid x) = \pi_c p(x \mid c) / \sum_k \pi_k p(x \mid k)$ [on your sheet]. It learns *what each class looks like* and asks which class explains this input best. Both routes end at the same object, a posterior; they differ in what they commit to on the way.

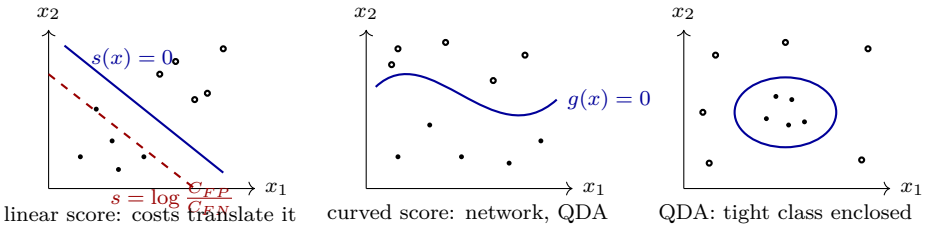
Manipulate / measure. Here is the experiment I would run to compare them honestly. We hold the features, the split and the metric fixed, so that nothing but the modelling choice differs. We change only the route: Naive Bayes or logistic regression on the same rows. Later we also change the training size N. We measure validation error and, just as important, calibration. The table that comes out tells you whether the assumptions of the generative route were worth the bias they introduced on this particular data; it is never decided by argument alone.

Goes wrong. Students pick a route by slogan: "generative for small data, discriminative for big data" and skip the validation. It feels right because the slogan is often true, but the crossing point depends on the data and the families, and sometimes there is no crossing at all. What you should do is ask the three model-choice questions at the end of this chapter (is a class model credible, do you have many labels, is

the raw input high-dimensional) and then compare on the same split.
Versus. Students mix the two routes up because both produce a posterior. The way to keep them apart is to count how many times each route explains an example. The generative route explains it *twice*: its class, through π, and its features, through $p(x \mid c)$. That is more structure, which is why it needs less data, and also more assumptions, which is why a wrong family leaves a bias that no amount of data removes. The discriminative route explains the label given the features and nothing else: fewer assumptions, more data needed, and no opinion about the input itself.
Example. With 200 emails and a 10,000-word vocabulary, Naive Bayes works from day one because it only counts words per class. With 2 million emails logistic regression wins, because it can learn interactions such as "*free* is spammy unless *invoice* is nearby", which the independence assumption forbids Naive Bayes from ever representing.

Scores, sigmoid, log-odds, and the boundary

What it is. A discriminative model first produces a real number, the score or logit $s_\theta(x)$, and only then turns it into a probability with the sigmoid $\sigma(s) = 1/(1+e^{-s})$ [on your sheet]. Why a score first? Because a score can be any real number, which is what a linear function or a network naturally produces; the sigmoid is the bridge from "any number" to "a number in $(0, 1)$". Logistic regression is the case $s = w^\top x + b$, and then $\log \frac{p}{1-p} = w^\top x + b$ [on your sheet]: the log-odds are linear in x. Four facts you should be able to use without looking them up: $\sigma(-s) = 1 - \sigma(s)$, σ is increasing, $\sigma' = \sigma(1-\sigma)$, and $\sigma(s) > t \iff s > \log \frac{t}{1-t}$ [on your sheet]. For orientation, $\sigma(0) = 0.5$, $\sigma(2) = 0.88$, $\sigma(5) = 0.993$.
Manipulate / measure. Let me run the threshold experiment in front of you. We fix the fitted w, b and the validation rows; nothing about the model changes. We slide the decision cut $\tau = \log(C_{\text{FP}}/C_{\text{FN}})$ from -4 to 4 and at each value we measure recall and the false-positive rate. Plotting one against the other draws this model's ROC curve. Notice what did not move: the *ranking* of the flows by score is the same at every τ; only the place where we cut the ranking moved. So if the curve is not good enough, no threshold will save you; you need a better model.
Goes wrong. The tempting mistake is to believe that changing the probability threshold bends the boundary. It does not: with a linear score the boundary is a hyperplane, and a new threshold is a *parallel* hyperplane; the geometry comes from the score family, the costs only translate it. The second temptation is to choose τ by looking at test results, which turns the test set into a validation set. Take τ from the costs, check it on validation rows.
Versus. Keep three objects apart. The score ranks the inputs and decides. The probability tells you how uncertain the model is between the classes. The loss, through the costs, picks the action. With $C_{\text{FN}} = 10$ and $C_{\text{FP}} = 1$ you raise the alarm as soon as $s > \log 0.1 = -2.30$, which is already at probability $\sigma(-2.30) = 0.09$: an expensive miss lowers the bar for alarming.



linear score: costs translate it — curved score: network, QDA — QDA: tight class enclosed

Softmax and its shift invariance

What it is. With C classes you have C scores, and softmax turns them into probabilities: $p_\theta(y = c \mid x) = e^{s_c} / \sum_k e^{s_k}$ [on your sheet]. Read it as two moves: exponentiate, so that every entry is positive, then divide by the total, so that the entries sum to one. The largest score always gets the largest probability, so the decision only needs the scores. One property matters more than it looks: adding the same constant to every score changes nothing, because it multiplies numerator and denominator by the same factor. So only $C - 1$ score *contrasts* are identifiable, and with $C = 2$ you get $\text{softmax}(s)_1 = \sigma(s_1 - s_2)$ [on your sheet], the sigmoid of a difference.
Manipulate / measure. Take the scores $(2, 0, -1)$. The exponentials are $(7.39, 1.00, 0.37)$, their total is 8.76, and the probabilities are $(0.84, 0.11, 0.04)$. When you do this by hand, subtract the largest score from all of them first: by shift invariance the answer is identical, and you avoid enormous exponentials.
Goes wrong. Students read a softmax output as an absolute statement, "this input is a cat with probability 0.84". It is a comparison among the modelled classes and nothing more; it says how much better

"cat" explains the input than the other two classes do. If you need to know whether the input is plausible at all, you need the compatibility check of the reject card.
Versus. Sigmoid and softmax are not two different ideas: the sigmoid is softmax applied to a score difference. Implementations still output C scores rather than $C-1$ so that every class plays the same role and relabelling only permutes the outputs.

Cross-entropy is conditional maximum likelihood

What it is. The training loss of every discriminative classifier is $\ell_{\text{CE}} = -\log p_\theta(y^{(i)} \mid x^{(i)})$ [on your sheet]: minus the log of the probability the model gave to the *true* class. If the true class got 0.9, 0.5 or 0.1, the loss is 0.105, 0.693 or 2.303 (natural logs). Look at the shape: a confident mistake is punished far more than a hesitant one, which is exactly what you want from a probability model. The derivation below shows that minimising the average cross-entropy is the same instruction as maximising the product $\prod_i p_\theta(y^{(i)} \mid x^{(i)})$ [on your sheet]. The word *conditional* is important: the model is asked to explain the labels given the inputs, never the inputs themselves.
Manipulate / measure. Three examples with true-class probabilities 0.9, 0.5, 0.1: the likelihood is 0.045, its negative log is 3.10, and the mean cross-entropy is 1.03. Same numbers, two names; when you see "maximum likelihood" in a generative chapter and "cross-entropy" in a network chapter, it is the same instruction.
Goes wrong. Two temptations. The first is to train directly on accuracy: the 0–1 loss is flat almost everywhere, so there is no gradient to follow and nothing to optimise. The second is the reverse: to compare and report models by cross-entropy when the deployment cost is a 0–1 count. The two can rank models differently. The rule is to optimise the smooth loss, then select and report with the metric the deployment cares about.
Versus. Discriminative maximum likelihood maximises $\prod p(y \mid x)$; generative maximum likelihood maximises $\prod p(x, y)$, and so explains each example twice.

Deep discriminative classifiers

What it is. A feed-forward network computes a sequence of representations, $h^{(\ell)} = \varphi(W_\ell h^{(\ell-1)} + b_\ell)$, each one "multiply by a matrix, add a vector, bend with a nonlinearity", and ends with scores $s = W_L h^{(L-1)} + b_L$, a softmax and the cross-entropy, trained end to end [not on your sheet: know it]. Logistic regression is the case with no hidden layer at all. The reason to add hidden layers is simple: they make the score a curved function of x, so the boundary $g(x) = 0$ can curve.
Manipulate / measure. You tune depth and width exactly as you tune any capacity knob, with a validation curve. Remember what the learned representation is optimised for: predicting the label, not describing or reconstructing the input. That is the seed of the next card.
Versus. The network changes the *geometry* of the boundary. The costs still only translate it along the score axis; a curved boundary under new costs is the same curve shifted by $\log(C_{\text{FP}}/C_{\text{FN}})$ in score.

What the discriminative route cannot say: confidence is not familiarity

What it is. Because the objective only asks for $p(y \mid x)$, the model is under no obligation to know whether x is a plausible input. It cannot sample inputs, it cannot marginalise a missing feature, and a large $\max_c p(y = c \mid x)$ means only that one known class outranks the others. This is not a defect of a particular network; it is built into the question the loss asks.
Manipulate / measure. The toaster experiment. We fix a network trained on cats, dogs and horses. We feed it inputs from outside its training population, say a photograph of a toaster, and we measure the top softmax probability. The scores come out $(4.0, 0.5, -1.0)$, softmax gives $(0.96, 0.03, 0.01)$, and the network says "cat, 96%". Nothing in training ever asked "is this even an animal?", so nothing in the output can answer it.
Goes wrong. The temptation is to use $\max_c p(c \mid x)$ as an anomaly score, "low confidence means unfamiliar". It feels right because unfamiliar inputs sometimes do get low confidence, but the toaster shows they need not. You catch it by holding out a whole family of inputs and looking at the top probability on them. The fix is a generative compatibility check with a threshold validated on the intended novelty.
Versus. Do not confuse this with class imbalance. If training and deployment share the same distribution, a prior of 5% is simply correct: the fitted posterior should reflect it, and the 0.5 contour *should* sit toward the positives. Imbalance is a question about the prior; familiarity is a question about the input.

The generative story and Bayes' rule

What it is. The generative classifier imagines how an example came to be: first a class is drawn, $Y \sim \text{Categorical}(\pi)$, then features are drawn from that class, $X \mid Y = c \sim p_\theta(\cdot \mid c)$. So the joint is $p(x, y = c) = \pi_c p_\theta(x \mid c)$. Bayes' rule reverses the story: $p(c \mid x) = \pi_c p(x \mid c) / \sum_k \pi_k p(x \mid k)$ [on your sheet]. Since the denominator is the same for every class, classification only compares the prior-weighted likelihoods $\pi_c p(x \mid c)$. Written as scores, $s_c = \log \pi_c + \log p(x \mid c)$, and the posterior is the softmax of these scores [on your sheet], which is the bridge to the discriminative vocabulary.

Manipulate / measure. Hold the class models fixed and change only the prior; watch the posterior. Benign flows have prior 0.99 and give this flow likelihood 0.001, product 0.00099; attacks have prior 0.01 and likelihood 0.20, product 0.0020. The posterior probability of attack is $0.002/0.00299 = 0.67$. The lesson is that a rare class can still win, provided it explains the input much better; the prior is a weight, not a veto.

Goes wrong. Two errors I see every year: forgetting the prior altogether (comparing raw likelihoods), or in Naive Bayes counting the prior once per feature. There is exactly one $\log \pi_c$ in each class score.

Versus. The odds form, $\frac{p(1|x)}{p(0|x)} = \frac{\pi_1}{\pi_0} \cdot \frac{p(x|1)}{p(x|0)}$ [not on your sheet: know it], prior odds times likelihood ratio, is worth carrying in your head: it is exactly what the prior-shift correction reweights.

The generative likelihood splits by class; the MLE prior

What it is. Take the negative log of the joint likelihood: $-\log L(\phi) = -\sum_c N_c \log \pi_c - \sum_c \sum_{i:y^{(i)}=c} \log p_\theta(x^{(i)} \mid c)$. Look at the structure: the first part depends only on π, and each term of the second part depends only on the parameters of one class and the examples of that class. So everything is fitted *separately*: the prior from the class counts, class c's model from class c's rows alone. The maximum-likelihood prior is $\hat{\pi}_c = N_c/N$ [on your sheet], and the derivation below shows that this is Gibbs' inequality again.

Manipulate / measure. Fit class c's model from its own rows, then add a thousand rows of another class: nothing in class c's model changes. That independence between classes is what makes generative fitting cheap and stable.

Goes wrong. Students estimate one class's parameters from all rows, or "fit" the prior from a resampled training set, in which case $\hat{\pi}$ equals the resampled prevalence and not the real one. Keep in mind where the prior came from.

Versus. Discriminative fitting couples all parameters through every example (each example pulls on the whole boundary); generative fitting decouples by class.

Two kinds of error of a fitted generative posterior

What it is. When the fitted posterior $p_{\hat{\phi}}(c \mid x)$ differs from the true $p(c \mid x)$, there are two separate reasons and you should name them separately. Shape approximation: the family $p_\theta(x \mid c)$ does not contain the truth, so even the best member is wrong; this error is persistent and does not vanish with more data. Finite-sample estimation: $\hat{\phi}$ differs from the best member because it was estimated from finite data; this error shrinks with N.

Manipulate / measure. On a learning curve you can see both: the first is the plateau of the generative curve, the second is its spread at small N.

Versus. This is Chapter 3's approximation error versus estimation error, applied to density estimation rather than to a predictor.

Gaussian class models and their closed-form MLE

What it is. For continuous features the simplest "world" for a class is a bell-shaped cloud, $p(x \mid c) = \mathcal{N}(x; \mu_c, \Sigma_c)$ [on your sheet]. The mean μ_c is the centre of the cloud; the covariance Σ_c is its shape (a multiple of the identity gives circles, a diagonal gives axis-aligned ellipses, off-diagonal entries rotate them); the quadratic form $(x-\mu_c)^\top \Sigma_c^{-1} (x-\mu_c)$ is the squared Mahalanobis distance, "how many standard deviations from the centre, in this direction". The maximum-likelihood fit needs no iteration: $\hat{\pi}_c = N_c/N$, $\hat{\mu}_c$ is the class mean, and $\widehat{\Sigma}_c = \frac{1}{N_c} \sum (x^{(i)} - \hat{\mu}_c)(x^{(i)} - \hat{\mu}_c)^\top$ [on your sheet], with denominator N_c. The unbiased sample covariance uses $N_c - 1$; for classification the difference is immaterial when N_c is large, but say which one you use.

Manipulate / measure. Let us do one dimension by hand. Class 0 is $\{0, 1, 2, 0, 1, 2\}$, class 1 is $\{3, 4, 5\}$. Then $\hat{\pi} = (2/3, 1/3)$, $\hat{\mu} = (1, 4)$ and $\hat{\sigma}^2 = (2/3, 2/3)$. At $x = 2$ the two densities share the same prefactor, so we compare exponentials: $\exp(-0.75) = 0.472$ against $\exp(-3) = 0.050$; weighting by the priors gives 0.315 against 0.017, and normalising gives $p(y = 0 \mid x = 2) = 0.950$. With equal priors the boundary

would sit at the midpoint 2.5; the larger prior of class 0 pushes it to 2.65, into class 1's territory.
Goes wrong. The classic slip is to compare raw densities and forget the prior, or to keep constants that differ between classes when the variances differ: with unequal variances the $-\frac{1}{2}\log\sigma_c^2$ term does not cancel and must stay.
Versus. With equal variances you compare $\log\hat{\pi}_c - (x-\hat{\mu}_c)^2/(2\sigma^2)$ only and the boundary is a single threshold. With unequal variances the score is quadratic in x, and there can be two cut points: the narrow class is enclosed by the wide one.

LDA versus QDA: an assumption about covariance becomes a shape

What it is. This is the most important idea of the Gaussian section, so let me say it slowly. If every class shares the same covariance $\widehat{\Sigma}$, the score reduces to $s_c(x) = x^\top\widehat{\Sigma}^{-1}\hat{\mu}_c - \frac{1}{2}\hat{\mu}_c^\top\widehat{\Sigma}^{-1}\hat{\mu}_c + \log\hat{\pi}_c$ [on your sheet], which is affine in x, so every boundary is a hyperplane: Linear Discriminant Analysis. If each class has its own $\widehat{\Sigma}_c$, the score is $s_c = -\frac{1}{2}(x-\hat{\mu}_c)^\top\widehat{\Sigma}_c^{-1}(x-\hat{\mu}_c) - \frac{1}{2}\log|\widehat{\Sigma}_c| + \log\hat{\pi}_c$ [on your sheet], quadratic in x, and the boundaries are conic sections: Quadratic Discriminant Analysis. The price of the flexibility is counted in parameters: a covariance costs $d(d+1)/2$ numbers [on your sheet], which is 5050 for $d = 100$; QDA with $d = 10$ and $C = 3$ classes estimates 165 covariance numbers, LDA only 55.
Manipulate / measure. Same rows, same priors; we change only the covariance assumption and draw the boundary. LDA draws a line, tilted by $\widehat{\Sigma}^{-1}$ rather than perpendicular to the segment between the means; QDA draws an ellipse or a hyperbola, and the tighter class ends up enclosed by the diffuse one. Now shrink the class size from 500 to 5 and refit on new samples each time: QDA's boundary swings wildly from sample to sample, and once $N_c < d+1$ the covariance has rank at most $N_c - 1$, it is singular, and the score cannot even be computed.
Goes wrong. The temptation is to choose QDA "because it is more flexible". With 120 defaults in a credit dataset and $d = 40$ features that is 820 covariance numbers for one class, estimated from 120 rows: training error goes down, validation error goes up, the classic overfitting signature. The remedy is LDA, or shrinkage $\widehat{\Sigma}_\lambda = (1-\lambda)\widehat{\Sigma} + \lambda\alpha I$ [not on your sheet: know it], which is positive definite for any $\lambda > 0$ and whose λ is chosen on validation; its validation curve is just Chapter 3's capacity U in another costume.
Versus. Students think "linear" describes the Gaussian; it describes the *boundary*, which is linear precisely because the quadratic term cancels when the covariance is shared. And a practical rule: if LDA and QDA tie on validation, the extra parameters bought nothing, so take LDA.

Naive Bayes: independence given the class

What it is. When estimating a full joint density of the features is hopeless, we pretend the features are independent *within each class*: $p(x \mid c) = \prod_j p(x_j \mid c)$ [on your sheet]. Take logs and the class score becomes a ledger, $s_c = \log\pi_c + \sum_j \log p(x_j \mid c)$ [on your sheet]: start with the prior evidence, then add one line per feature, each line coming from a one-dimensional model of that feature in class c. One-dimensional models are stable with little data, and they need not share a family: a Gaussian here, a Bernoulli there, a Poisson for a count, a histogram for an irregular feature. One more fact makes Naive Bayes far more useful than its assumption suggests: a misspecified density can still rank the classes correctly, because getting the argmax right is a weaker requirement than getting every density right.
Manipulate / measure. Here is the experiment that shows what the assumption costs. Duplicate the "bytes" column k times and keep everything else fixed. Accuracy barely moves, because no information was added or removed. But the reliability curve bends into an S: a flow that was given 0.7 is now given 0.9 and then 0.99, while the observed fraction of attacks among such flows has not changed. The reason is that the log-odds are multiplied by k: the same evidence was counted k times.
Goes wrong. So correlated features double-count evidence. The ranking survives and the calibration dies, and because accuracy looks fine the problem is missed. Either remove redundant features, or use a model that sees correlations (LDA, logistic regression). Two practical rules: work in logs, and count the prior once.
Versus. Gaussian Naive Bayes with class-specific variances per feature is a diagonal QDA, so its boundary is quadratic even though each feature is Gaussian; with shared variances it is a diagonal LDA.

Gaussian NB and variance smoothing

What it is. Gaussian Naive Bayes models $x_j \mid c \sim \mathcal{N}(\hat{\mu}_{cj}, \hat{\sigma}_{cj}^2)$, feature by feature. Two things go wrong with the variance. If it is exactly zero (every class-c example had the same value), the density is undefined. If it is merely tiny, that one narrow feature dominates the whole class score. The fix is variance smoothing, $\tilde{\sigma}_{cj}^2 = \hat{\sigma}_{cj}^2 + \lambda s_j^2$ [not on your sheet: know it], where s_j^2 is the overall variance of feature j.

Manipulate / measure. Suppose feature 2 has $\hat{\sigma}^2 = 10^{-6}$ because every class-c example used the same port, and a test flow's value differs by 0.01. Its contribution to the class score is $-\frac{1}{2}(0.01)^2/10^{-6} = -50$, which drowns anything the other features say. Add $\lambda s^2 = 0.01$ to the variance and the contribution becomes -0.005: the feature still counts, but it no longer vetoes.
Goes wrong. Letting an almost-constant feature decide every classification. If the value of λ changes predictions materially, choose it on validation like any other knob.
Versus. Variance smoothing is the continuous twin of additive smoothing for counts: both add a little imaginary spread so that no single observation can produce a zero or an infinity.

Naive Bayes variants, by feature type

What it is. The independence assumption is what lets you mix families freely: Gaussian for measurements such as flow statistics, Bernoulli for presence or absence indicators, multinomial for counts such as words or events, and a hybrid where each feature gets the distribution of its type. All of them are multiplied under *one* class prior.
Manipulate / measure. A Bernoulli example by hand. Features: has a link, from a known sender. Spam rates $(0.9, 0.1)$, ham rates $(0.2, 0.8)$, priors 0.5. An email with a link from a known sender scores spam $0.5 \cdot 0.9 \cdot 0.1 = 0.045$ and ham $0.5 \cdot 0.2 \cdot 0.8 = 0.08$: ham, with posterior $0.08/0.125 = 0.64$.
Goes wrong. Forcing a Gaussian on a count or a binary feature; or, in a hybrid, including the prior once per feature.
Versus. Multinomial treats a repeated word as repeated evidence; Bernoulli only records presence, and for it absence is evidence too, through the factor $1 - \theta$.

Multinomial Naive Bayes, zero counts, additive smoothing

What it is. Represent a document by its count vector over a vocabulary of V words; class c is a bag of words with proportions θ_{cj}, and $p(x \mid c) \propto \prod_j \theta_{cj}^{x_j}$ with $\sum_j \theta_{cj} = 1$ [on your sheet]. Repeated words are repeated evidence, word order is ignored, and the multinomial coefficient cancels in Bayes' rule because it does not depend on the class. The estimate with additive smoothing is $\hat{\theta}_{cj} = (n_{cj} + \alpha)/(\sum_\ell n_{c\ell} + \alpha V)$ [on your sheet], which you should read as "pretend you saw every word α extra times in every class". It is a proper distribution for any $\alpha > 0$; as $\alpha \to 0$ you recover the raw counts, as $\alpha \to \infty$ everything tends to the uniform $1/V$.
Manipulate / measure. Counts: spam $(3, 2, 2, 0, 0, 0)$, ham $(0, 0, 0, 3, 2, 2)$ over (free, win, prize, meeting, project, report). Without smoothing, "free meeting" gets likelihood 0 under both classes, because "meeting" never appeared in spam and "free" never in ham: the posterior is 0/0. With $\alpha = 1$ the estimates become spam $(4, 3, 3, 1, 1, 1)/13$ and ham $(1, 1, 1, 4, 3, 3)/13$; "free meeting" now has posterior 1/2 (the evidence balances), "free prize" gives $12/13 = 0.92$, and "free free prize" gives $48/49 = 0.98$, the repeated word counting twice. Sweep α on validation: the log-loss is infinite at $\alpha = 0$, has its minimum near 0.1 to 1, and rises toward the uniform model (with $\alpha = 100$ on counts $(5, 0, 3, 2)$ and $V = 4$ you get $(0.256, 0.244, 0.251, 0.249)$, almost no class information left).
Goes wrong. One unseen word vetoing a class; smoothing after normalising instead of before; forgetting the αV in the denominator; and a category never seen in training crashing at test time (give the feature an "other" level among its K_j levels and smooth it).
Versus. α is a capacity knob like any other: small trusts the counts (variance, vetoes), large trusts the uniform prior (bias). Choose it on validation.

Comparing the routes: the data-size experiment

What it is. Generative structure reduces estimation error: a restricted description of each class has few parameters, so it can be fitted reliably from little data, provided the assumptions are roughly right. A wrong family adds an approximation bias that no amount of data removes. A sufficiently flexible discriminative model spends no capacity on aspects of X that are irrelevant to Y, and with enough labels it reaches lower error.
Manipulate / measure. The experiment that settles it: fix the features, an untouched test set, the smoothing α and the regularisation λ; vary $N \in \{50, 200, 10^3, 10^4, 10^5, 2 \cdot 10^6\}$ with ten random subsets at each size; measure test error. Naive Bayes starts low and flattens at its bias floor. Logistic regression starts high with a wide spread, falls, and crosses below somewhere between 10^3 and 10^5. The crossing point is the result you report; its existence and location depend on the data and on both families.
Goes wrong. Reading a flat generative curve as "it needs more data": the floor is approximation error, so the fix is a better class model, not more rows. Reading the wide spread of the discriminative curve at

small N as bias: it is variance.
Versus. The question is never "which route is better". It is "which assumptions are credible here, and which deployment objective matters", and then you validate under deployment-like conditions.

Calibration versus threshold

What it is. A probabilistic classifier is calibrated if, among the cases it assigns probability u, about a fraction u turn out positive. Calibration is a statement about how to *interpret* the probabilities; the threshold is about the *action* you take from them. Two separate questions, and both must be right for a cost-sensitive decision to be right.
Manipulate / measure. Draw the reliability diagram on validation rows: bin the predicted probabilities, plot the observed positive fraction in each bin against the bin's probability. Points below the diagonal mean the model is overconfident.
Goes wrong. The temptation is to repair a calibration problem by moving the threshold. The ranking is unchanged by any monotone distortion of the probabilities, so accuracy may look fine, but the expected counts of alarms and the precision at the cost threshold are still wrong. Calibrate first, then set the threshold from the costs.
Versus. Resampling and duplicated features both break calibration while leaving accuracy alone, which is exactly why the problem goes unnoticed until the counts come in wrong.

Class imbalance versus class-prior shift

What it is. Two situations are constantly confused, so let me separate them. *Imbalance*: a class is rare, but the distribution is the same at training and deployment. Then the posterior *should* reflect the prevalence, and nothing needs correcting; the only real difficulty is that few minority examples make the boundary hard to estimate, where weighting or resampling can help estimation at the price of calibration. *Prior shift*: the class-conditionals are unchanged, $p_d(x \mid c) = p_t(x \mid c)$, but the priors differ, $\pi_d \neq \pi_t$ [not on your sheet: know it], as when attacks are 20% of the training data and 5% of deployment traffic. The generative correction is exact and free: keep the fitted class models and swap in π_d [not on your sheet: know it]; no refit. The discriminative correction reweights the odds: $r_d = \frac{(\pi_d/\pi_t)r_t}{(\pi_d/\pi_t)r_t + \frac{1-\pi_d}{1-\pi_t}(1-r_t)}$ [not on your sheet: know it], and it requires r_t to be a calibrated probability.
Manipulate / measure. Build a training set at 20% attacks and a test set at 5% from one pool, so the class-conditionals are identical by construction. Vary only the correction: none, swap the priors, reweight the odds. Measure log-loss, the reliability diagram, and the number of alarms at the cost threshold. Uncorrected, flows predicted at 0.5 are observed at 0.17 and you raise four times too many alarms. Corrected, the reliability diagram is back on the diagonal and the alarms are what the costs intended; recall drops slightly, and accuracy is almost unchanged, because the correction is monotone and does not reorder the flows.
Goes wrong. The temptation is to oversample to a balanced set and then trust the probabilities. The effective prior is now 0.5, so the rare-class probabilities are inflated by the factor $\frac{0.5/0.5}{\pi/(1-\pi)}$. Correct back with π_t equal to the resampled prior, and apply the correction *before* the cost threshold, never after.
Example. $r_t = 0.5$, $\pi_t = 0.2 \to \pi_d = 0.05$: the factors are 0.25 for attack and $0.95/0.80 = 1.1875$ for benign; $A = 0.125$, $B = 0.594$, $r_d = 0.125/0.719 = 0.17$. What looked like a coin flip in training is a 17% attack in deployment, because attacks are four times rarer there.

Reject option, combined systems, language models

What it is. To say "I do not recognise this", ask two questions instead of one. Which known class fits best? That is the posterior, $c^* = \arg\max_c p(c \mid x)$. Does even the best class fit well enough? That is the likelihood against a threshold: accept iff $\log p(x \mid c^*) \geq \tau_{c^*}$, else reject [not on your sheet: know it]. Only a generative model can answer the second question, because only it has a notion of what inputs of class c look like. Modern systems combine the routes: a discriminative ranker for the known classes, a generative or anomaly model as the plausibility check it lacks, feeding an accept / review / reject policy. Language models are the same marriage at scale: generative pretraining, $p(w_{1:T}) = \prod_t p(w_t \mid w_{<t})$, followed by a discriminative head $p_\psi(y \mid w_{1:T})$ that gives one label per sequence, or by prompting the model to score label tokens.
Manipulate / measure. Train on five attack families and hold out the sixth. Sweep τ and measure false rejects of known flows against true rejects of the novel family. The trade-off curve is the result, and a softmax-only system gives you no handle at all, because its top probability on the novel family is high.
Goes wrong. Tuning τ on data that contains no novelty of the intended kind; and treating a low likelihood

as proof of attack, when high-dimensional likelihoods behave strangely and a misspecified class model gives low likelihood to perfectly valid cases.
Versus. Softmax confidence compares the known classes with one another; the likelihood check asks whether the input resembles the training data of the winning class. Different questions, different tools.

The three model-choice questions (and Lab 2)

What it is. When you must choose, ask in this order. Can a credible class-conditional model be stated? If yes, its structure reduces the labels you need. Is the goal accurate prediction with many labels? Then a flexible discriminative model avoids needless assumptions about the input. Is the raw input complex and high-dimensional? Then learn a representation first and compare simple heads on it; that is more credible than modelling the raw density. In the end deployment-like validation decides; the labels "generative" and "discriminative" do not.
Manipulate / measure. Lab 2 is this chapter as an experiment: Gaussian, multinomial and hybrid Naive Bayes against logistic regression and a decision tree, on one fixed stratified split and the same rows; then limited data (20% of each class); then a deployment prior $\pi_d = 0.05$ corrected both ways; then multiclass with and without the destination port.
Goes wrong. Different rows or splits for different models, so that the differences you measure reflect the data and not the models.

The five classifiers of the chapter

Model	Modelled quantity	Parameters	Boundary	Data needed	Main failure
Logistic regression	$p(y \mid x)$, log-odds linear	$(C-1)(d+1)$	hyperplane	most	underdetermined when $d \gg N$; no input plausibility
LDA	Gaussian $p(x \mid c)$, shared Σ	$(C-1) + Cd + \frac{d(d+1)}{2}$	hyperplane	little	wrong shape if clouds differ
QDA	Gaussian, own Σ_c	$(C-1) + Cd + C\frac{d(d+1)}{2}$	conic	much per class	singular or unstable Σ_c when $N_c \lesssim d$
Gaussian NB	independent Gaussians per feature	$(C-1) + 2Cd$	quadratic (diagonal QDA)	very little	double counting, tiny variances
Multinomial NB	word-count model per class	$C(V-1)$, smoothed	linear in counts	very little	zero counts without α; no interactions

The chapter's experiments at a glance

Experiment	Manipulate	Measure	What it teaches
threshold sweep	cut τ on the score	recall, FPR, alarms	one model has one ROC; costs move the cut, not the curve
toaster	inputs outside the training population	top softmax probability	confidence is relative, not familiarity
LDA vs QDA, shrinking N_c	covariance assumption; class size	boundary shape, validation error, invertibility	assumption → shape; flexibility must be estimable
shrinkage / α sweep	λ in $\widehat{\Sigma}_\lambda$; α in counts	validation log-loss	both are capacity knobs with a U; choose on validation
duplicated feature	copies of one feature	accuracy, reliability diagram	NB ranking survives, calibration dies
NB vs logistic vs N	training size on a log axis	test error, spread	crossing point; floor = bias, spread = variance
prior shift 20% → 5%	the correction applied	log-loss, reliability, alarms	accuracy hides it; probabilities and counts do not

Sigmoid properties and the cost threshold

(1) $1-\sigma(s) = \frac{e^{-s}}{1+e^{-s}} = \frac{1}{e^{s}+1} = \sigma(-s)$: multiply numerator and denominator by e^s; this is why one score suffices for two classes.
(2) Odds: $\frac{\sigma(s)}{1-\sigma(s)} = \frac{1/(1+e^{-s})}{e^{-s}/(1+e^{-s})} = e^s$, so $\log\frac{p}{1-p} = s = w^\top x + b$: the score *is* the log-odds, which is what makes a linear score a linear boundary.
(3) Inversion: $\sigma(s) > t \iff 1 + e^{-s} < 1/t \iff e^{-s} < \frac{1-t}{t} \iff s > \log\frac{t}{1-t}$; each step is allowed because every quantity is positive and log is increasing.
(4) The cost threshold from Chapter 2 is $t = \frac{C_{\text{FP}}}{C_{\text{FP}}+C_{\text{FN}}}$, hence $\frac{t}{1-t} = \frac{C_{\text{FP}}}{C_{\text{FN}}}$: the costs enter only through their ratio.
(5) Decision: $\hat{y} = 1 \iff s_\theta(x) > \log\frac{C_{\text{FP}}}{C_{\text{FN}}}$. For a linear score this is the parallel hyperplane $w^\top x + b = \log(C_{\text{FP}}/C_{\text{FN}})$: equal costs give $s > 0$, and $C_{\text{FN}} = 10C_{\text{FP}}$ gives $s > -2.30$.

Cross-entropy equals negative conditional log-likelihood

(1) Assume the labels are conditionally independent given the inputs, so the probability of all the observed labels is a product: $L_{\text{cond}}(\theta) = \prod_{i=1}^{N} p_\theta(y^{(i)} \mid x^{(i)})$.
(2) Take the negative log; the product becomes a sum, which is what we can differentiate: $-\log L_{\text{cond}} = -\sum_i \log p_\theta(y^{(i)} \mid x^{(i)})$.
(3) The per-example cross-entropy with a one-hot label is $-\sum_c \mathbf{1}(y^{(i)} = c)\log p_\theta(c \mid x^{(i)}) = -\log p_\theta(y^{(i)} \mid x^{(i)})$: only the true class survives the indicator.
(4) Sum over i and divide by N: $-\log L_{\text{cond}} = N\widehat{R}_{\text{CE}}(\theta)$.
(5) Since $-\log$ is decreasing, maximising L_{cond} and minimising $\widehat{R}_{\text{CE}}$ have the same minimisers. Numbers: $0.9 \cdot 0.5 \cdot 0.1 = 0.045$; $-\log = 0.105 + 0.693 + 2.303 = 3.10$; $\widehat{R}_{\text{CE}} = 1.03$.

Bayes' rule for classification and its odds form

(1) Start from the definition of conditional probability: $p(y = c \mid x) = p(x, y = c)/p(x)$.
(2) Use the generative factorisation for the numerator: $p(x, y = c) = \pi_c p_\theta(x \mid c)$, the story "choose a class, then generate features".
(3) The denominator is the marginal over the exhaustive, mutually exclusive classes: $p(x) = \sum_k \pi_k p_\theta(x \mid k)$.
(4) Hence $p(c \mid x) = \pi_c p(x \mid c)/\sum_k \pi_k p(x \mid k)$. The denominator is common to all c, so the argmax is decided by the numerator $\pi_c p(x \mid c)$; equivalently the posterior is $\text{softmax}_c(\log \pi_c + \log p(x \mid c))$, which is what lets us speak of generative "scores".
(5) For two classes divide the two numerators and the denominator cancels: $\frac{p(1|x)}{p(0|x)} = \frac{\pi_1}{\pi_0} \cdot \frac{p(x|1)}{p(x|0)}$, prior odds

times likelihood ratio. Rare attack: $0.01 \cdot 0.20/(0.99 \cdot 0.001) = 2.02$, posterior $2.02/3.02 = 0.67$.

The MLE prior is the class frequency (Gibbs)

(1) The prior part of the negative log-likelihood is $-\sum_c N_c \log \pi_c$, because label c occurs N_c times and each occurrence contributes $\log \pi_c$.
(2) Write $q_c = N_c/N$, which is a probability vector: $\sum_c N_c \log \pi_c = N \sum_c q_c \log \pi_c = -N\, H(q, \pi)$, the cross-entropy between the empirical class distribution and the model prior.
(3) Gibbs' inequality says $H(q, \pi) = H(q) + \mathrm{KL}(q \| \pi) \geq H(q)$ with equality iff $\pi = q$ [on your sheet].
(4) So the prior part is minimised, and the likelihood maximised, exactly at $\hat{\pi}_c = N_c/N$.
(5) Read it as a lesson rather than a formula: generative training explains each example twice, its class through π and its features through $p_\theta(x \mid y)$, and fitting the class part is Gibbs' inequality once more.

Gaussian MLE in one dimension, and the 1-D boundary

(1) Because the likelihood splits by class, fit class c from its own $n = N_c$ points; the negative log-likelihood is $J(\mu, \sigma^2) = \frac{n}{2} \log(2\pi\sigma^2) + \frac{1}{2\sigma^2} \sum_i (x^{(i)} - \mu)^2$.
(2) Mean: $\partial J/\partial \mu = -\frac{1}{\sigma^2} \sum_i (x^{(i)} - \mu) = 0$ gives $\hat{\mu} = \frac{1}{n} \sum_i x^{(i)}$, and J is a convex quadratic in μ, so this is the minimum.
(3) Variance: put $v = \sigma^2$ and $S = \sum_i (x^{(i)} - \hat{\mu})^2$; $\partial J/\partial v = \frac{n}{2v} - \frac{S}{2v^2} = 0$ gives $\hat{v} = S/n$, and the derivative is negative before and positive after, so it is a minimum. The denominator is $n = N_c$, not $N_c - 1$.
(4) In d dimensions the same two steps give the class mean and the average outer product of the centred examples [on your sheet].
(5) The boundary of the worked example (equal variances $2/3$, priors $2/3, 1/3$, means $1, 4$): set the two scores equal, $\log \frac{2}{3} - \frac{3}{4}(x-1)^2 = \log \frac{1}{3} - \frac{3}{4}(x-4)^2$, i.e. $\frac{3}{4}[(x-4)^2 - (x-1)^2] = -\log 2$, so $\frac{3}{4}(15 - 6x) = -0.693$ and $x = 2.65$: the midpoint 2.5 pushed toward the rarer class by 0.15. LDA in one dimension is a single threshold.

LDA: shared covariance makes the score affine

(1) Take the log of the Gaussian: $\log p(x \mid c) = -\frac{d}{2} \log 2\pi - \frac{1}{2} \log |\Sigma| - \frac{1}{2}(x - \mu_c)^\top \Sigma^{-1} (x - \mu_c)$.
(2) Expand the quadratic form, using the symmetry of Σ^{-1} to merge the two cross terms: $x^\top \Sigma^{-1} x - 2x^\top \Sigma^{-1} \mu_c + \mu_c^\top \Sigma^{-1} \mu_c$.
(3) This is the key step: with a *shared* Σ, the terms $-\frac{d}{2} \log 2\pi$, $-\frac{1}{2} \log |\Sigma|$ and $-\frac{1}{2} x^\top \Sigma^{-1} x$ are identical for every class, so they drop out of any comparison between classes.
(4) What remains is $s_c(x) = x^\top \Sigma^{-1} \mu_c - \frac{1}{2} \mu_c^\top \Sigma^{-1} \mu_c + \log \pi_c$, affine in x.
(5) Two affine scores are equal on a hyperplane; in the binary case the normal is $w = \Sigma^{-1}(\mu_1 - \mu_0)$ [on your sheet]. With separate Σ_c step (3) fails: $x^\top \Sigma_c^{-1} x$ and $\log |\Sigma_c|$ stay, and the score is quadratic (QDA). One dimension, equal variances, priors $0.9/0.1$, means 0 and 4: $4x - 8 = \log 9 = 2.197$, boundary $x = 2.55$.

Discriminative prior-shift correction via odds

(1) At training time the odds are $\frac{r_t}{1 - r_t} = \frac{\pi_t}{1 - \pi_t} \cdot \frac{p(x|1)}{p(x|0)}$, by the odds form of Bayes' rule; the same holds at deployment with π_d.
(2) Under prior shift the likelihood ratio $p(x \mid 1)/p(x \mid 0)$ is the same at both times, because the class-conditionals did not change; this is the assumption that makes the correction possible.
(3) Solve the training odds for the ratio and substitute into the deployment odds: $\frac{r_d}{1 - r_d} = \frac{\pi_d/(1-\pi_d)}{\pi_t/(1-\pi_t)} \cdot \frac{r_t}{1 - r_t}$.
(4) Odds A/B correspond to probability $A/(A + B)$; with $A = \frac{\pi_d}{\pi_t} r_t$ and $B = \frac{1-\pi_d}{1-\pi_t}(1 - r_t)$ we get $r_d = A/(A + B)$.
(5) The formula silently requires r_t to be a calibrated posterior, since step (1) read it as one; the generative route instead swaps π_t for π_d in Bayes' rule, exactly. Numbers: $0.2 \to 0.05$, $r_t = 0.5$: $A = 0.125$, $B = 0.594$, $r_d = 0.17$.

How he asks it

Q. *A network trained on cats, dogs and horses outputs "cat, 96%" for a photograph of a toaster. Explain why this is not a bug of the training, what the number measures, and what you would add.*
A. It is not a bug, because the discriminative objective only ever asked for $p(y \mid x)$ among three known classes. The number 0.96 measures how much better "cat" explains the toaster than "dog" and "horse" do; it is a relative comparison produced by the softmax, and nothing in the cross-entropy loss ever rewarded

the network for knowing whether x is plausible. What you add is a generative compatibility check: accept the winning class c^* only if $\log p(x \mid c^*) \geq \tau_{c^*}$, with τ validated on held-out novelty, for instance a class excluded from training; otherwise reject the input or send it to review. The softmax ranks, the likelihood checks familiarity.

Q. *Why is LDA's boundary linear although the Gaussian density is not? What breaks when each class gets its own covariance?*
A. The boundary is linear because of a cancellation, not because of the density. The class score is $\log \pi_c + \log \mathcal{N}(x; \mu_c, \Sigma)$, and the only term quadratic in x, $-\frac{1}{2} x^\top \Sigma^{-1} x$, together with the normaliser $-\frac{1}{2} \log |\Sigma|$, is the same for every class when Σ is shared; both drop out of every comparison, leaving the affine score $x^\top \Sigma^{-1} \mu_c - \frac{1}{2} \mu_c^\top \Sigma^{-1} \mu_c + \log \pi_c$, and two affine functions are equal on a hyperplane. With a Σ_c per class both terms depend on c, nothing cancels, the comparison becomes a quadratic equation and the boundary a conic. It also costs $d(d+1)/2$ extra numbers per class, which is why QDA needs far more data and becomes unstable or singular when N_c is small.

Q. *Attacks are 20% of the training set and 5% in deployment; the class-conditional distributions are unchanged. A flow gets training posterior 0.5. What is its deployment posterior, how do you get it for a generative and for a discriminative model, and which assumption does the discriminative correction add?*
A. About 0.17. For a generative model keep the fitted $p(x \mid c)$ and replace $\pi_t = (0.8, 0.2)$ by $\pi_d = (0.95, 0.05)$ in Bayes' rule; that is exact and needs no refit, because the prior is an explicit component of the model. For a discriminative model use the fact that the likelihood ratio is shared between the two times and reweight the odds: multiply the attack side by $\pi_d/\pi_t = 0.25$ and the benign side by $(1-\pi_d)/(1-\pi_t) = 1.1875$, giving $A = 0.125$, $B = 0.594$ and $r_d = 0.125/0.719 = 0.17$. The extra assumption is that r_t is a calibrated probability; a model trained on a resampled set, or an overconfident one, must be calibrated first. And apply the cost threshold after the correction, not before.

Q. *Multinomial Naive Bayes classifies "free meeting" as neither spam nor ham. Explain the failure, fix it with $\alpha = 1$, and say what large α does.*
A. The word "meeting" never occurred in spam and "free" never in ham, so each class likelihood contains a factor of zero and the posterior is 0/0: a single unseen word vetoes a class no matter how strong the other evidence is. Additive smoothing adds α imaginary occurrences of every word to every class, $\hat{\theta}_{cj} = (n_{cj} + 1)/(7 + 6)$, so spam becomes $(4, 3, 3, 1, 1, 1)/13$ and ham $(1, 1, 1, 4, 3, 3)/13$; "free meeting" now scores $\frac{1}{2} \cdot \frac{4}{13} \cdot \frac{1}{13}$ under both classes and the posterior is $\frac{1}{2}$. As $\alpha \to \infty$ every $\hat{\theta}_{cj} \to 1/V$ and the classes become indistinguishable, which is bias; as $\alpha \to 0$ the vetoes come back, which is variance. So α is a capacity knob and you choose it on validation.

Q. *Naive Bayes on flow data keeps its accuracy when a duplicate of the bytes feature is added, yet its probabilities become extreme. Explain, and name a consequence.*
A. Conditional independence makes the class score additive, so a duplicated feature adds the same log-likelihood term twice and the log-odds are doubled. The sign of the log-odds, hence the ranking and the accuracy at threshold 0.5, is unchanged, but a flow that deserved 0.7 is now given 0.9 and the reliability curve bends below the diagonal: the model has become overconfident without becoming more wrong. The consequence is that everything which reads the probability as a probability is now broken: cost-sensitive thresholds, the prior-shift correction and soft voting. The fix is to drop redundant features or to use a model that sees correlations, such as LDA or logistic regression.

Q. *With 200 labelled emails and a 10,000-word vocabulary, which route would you start with and why? What changes with 2 million emails? Describe the experiment that settles it.*
A. Start with the generative route. Multinomial Naive Bayes estimates $2 \times 10{,}000$ smoothed word probabilities from simple counts and is stable from day one, whereas logistic regression must locate a 10,000-dimensional hyperplane from 200 points, which is badly underdetermined and therefore high-variance. With millions of labels the picture flips: the independence assumption is now a persistent approximation bias, since it cannot represent interactions such as "free unless invoice", while logistic regression's variance has vanished. The experiment: fix the features, α, λ and an untouched test set; vary N on a log scale with repeated random subsets; plot both test-error curves; report where they cross and the height of the generative floor.

Q. *A class has 5 examples in $d = 10$ dimensions. What goes wrong with QDA, why, and what are the two remedies?*
A. The class covariance is an average of 5 centred outer products, so its rank is at most $4 < 10$: it is singular, cannot be inverted, and the Mahalanobis score cannot be computed at all. With a few more examples it becomes invertible but remains unstable, meaning that a different sample would give a very different ellipse, which is variance. The two remedies are to shrink, $\widehat{\Sigma}_\lambda = (1-\lambda)\widehat{\Sigma} + \lambda\alpha I$, which is positive

definite for any $\lambda > 0$ with λ chosen on validation, or to impose structure: a shared covariance (LDA, 55 numbers estimated from all classes together) or a diagonal one (Gaussian Naive Bayes, 10 variances).

Q. *Attacks are 5% of both the training data and the deployment traffic. A colleague says the classifier is "biased" because its* 0.5 *contour sits toward the attacks and wants to rebalance. Is a correction needed?*
A. No. When the joint distribution is the same at training and deployment, the fitted posterior should reflect the 5% prevalence, and the boundary sitting toward the positives is Bayes' rule doing its job, not an error. Imbalance only becomes a problem of estimation, when the minority is too small to locate the boundary reliably; then weighting or resampling can help, but it changes the effective prior, and the probabilities must be corrected back before any cost threshold is applied. A correction is needed when the prior *changes* between training and deployment, and that is a different situation with a different name.

Five mistakes

- Reading a softmax probability as the plausibility of the input, or moving a threshold to fix a calibration problem.
- Forgetting the prior in a generative score, or counting it once per feature in Naive Bayes.
- Choosing QDA with few examples per class ($N_c \lesssim d$): a singular or wildly unstable covariance; use LDA or shrinkage.
- Unsmoothed counts or variances: one unseen word or one constant feature vetoes a class.
- Confusing imbalance with prior shift, resampling and then trusting the probabilities, or thresholding before correcting.

4 Chapter 5: Learning by Combining Models

Let me start with the problem this chapter solves. In Chapter 3 you saw that a flexible model trained on one sample has high variance: if I handed you a second sample from the same world and you trained again, you would get a noticeably different fit. The idea of this chapter is almost embarrassingly simple: if one fit is unstable, train several and combine them, because the parts of their errors that differ will cancel in the average while the signal they share survives. That one sentence contains a condition, and the condition is everything: the errors must differ. So the chapter is really a catalogue of ways to manufacture models whose errors differ (different algorithms, different data, different targets) together with the rules for combining them (vote, average, sum of corrections, learned combination). The base learner throughout is the decision tree, because it is flexible, fast and, conveniently for us, unstable. Every formula below is on your sheet unless I tag it otherwise; what I will test is whether you know what each knob manipulates, what the experiment looks like, and what goes wrong.

Why combine models

What it is. An ensemble is nothing more than several fitted models together with a rule, a vote, an average or a sum, that turns their several predictions into one. Why would you want this? Because a single flexible fit reflects the particular sample it was trained on, and that sample-dependence is the variance of Chapter 3. Averaging is the classical remedy for variance. But notice the two conditions hidden in the word "averaging". First, the members must be competent, because averaging pure noise gives you noise. Second, their errors must differ, because a mistake that every member makes survives the average untouched. Three copies of the same model are not an ensemble; they have the same strengths and exactly the same mistakes.
Manipulate / measure. Here is the experiment I would run to show you what variance looks like. I hold the learning algorithm fixed, say a deep tree. I draw two training samples from the same population, fit the tree on each, and overlay the two decision boundaries on one plot. They do not coincide; the gap between them is $\mathrm{Var}(x)$ made visible. Now look at where the two fits make their mistakes: where they err in different places, their average errs less than either one, and where they err in the same place, the average inherits the error. That picture is the whole chapter.

Goes wrong. The tempting shortcut is to ensemble three variants of the same model, trained on the same rows with the same missing feature, and to expect an improvement because "it is an ensemble". It feels right because you did train three models. But their errors share the same direction, so ρ is close to one and the average changes nothing. You catch this by checking whether the members disagree on the validation rows; if they almost never do, the fix is diversity of data or of targets, not more copies.
Versus. Students mix up the three families because they all end with "combine". Keep them apart by asking where the diversity comes from. Voting uses different models on the same data. Bagging and random forests use the same learner on randomised data. Boosting uses a sequence of models, each one aimed at the current errors of the ensemble so far.
Example. Galton's livestock fair in 1906: 787 visitors guessed the weight of an ox, individual guesses were all over the place, and their mean, 1197 lb, was within a pound of the true 1198 lb. Majority labels from three annotators on a network event work the same way: two say malicious, one says benign, you keep the majority and send the strong disagreements to an expert.

Averaging, hard voting, soft voting

What it is. For regression the rule is the plain average, $\hat{f}_{\text{ens}} = \frac{1}{M}\sum_m \hat{f}_m$. For classification you have two choices. Hard voting lets each classifier name a class and keeps the most frequent one. Soft voting averages the posterior probabilities and takes the largest, $\hat{p}_{\text{ens}}(c \mid x) = \frac{1}{M}\sum_m \hat{p}_m(c \mid x)$ [on your sheet]. The difference is what each rule keeps: soft voting uses the margin by which each model preferred its class, hard voting throws that margin away and caps every model at one vote.
Manipulate / measure. Fix three classifiers and one input and compare the two rules side by side. Suppose A gives class 1 a probability of 0.49, B gives 0.48, and C gives 0.95. The hard vote reads the three decisions: A and B say class 0, C says class 1, so the tally is 2 to 1 for class 0. The soft vote averages the probabilities: $(0.49 + 0.48 + 0.95)/3 = 0.64$, which is above one half, so it says class 1. Why the disagreement? Because A and B were coin flips and C was nearly certain; the soft vote used that information and the hard vote discarded it. Whether the soft vote is right depends entirely on whether C's 0.95 deserves to be believed.
Goes wrong. The temptation is to soft-vote whatever models you have. If one of them is overconfident, or calibrated under a different class prior, it dominates every average, because it always contributes a number near 0 or 1. You catch this by looking at the calibration of each member on validation rows. The fixes are to calibrate, to weight the models, or to fall back to hard voting, which is robust to overconfidence at the price of discarding the margins.
Versus. Hard voting counts heads and needs no calibration at all. Soft voting pools confidence and therefore needs probabilities that are comparable, reasonably calibrated, and computed under the same prior. Choose by asking whether you trust the probabilities.

Variance of an average of correlated errors

What it is. Fix an input x and let e_m be the error of model m there, viewed as a random variable over training samples. If every error has variance σ^2 and every pair has correlation ρ, then the error of the average has variance $\text{Var}(\bar{e}) = \sigma^2\big(\rho + \frac{1-\rho}{M}\big)$ [on your sheet]. Read the three cases: with $\rho = 0$ you get σ^2/M, the textbook gain of averaging independent things; with $\rho = 1$ you get σ^2, nothing gained; and as $M \to \infty$ the variance does not go to zero but to the floor $\rho\sigma^2$. That floor is the part of the error all the models share, and averaging is powerless against it. Note also what the formula is silent about: bias. Averaging does nothing for bias.
Manipulate / measure. I fix σ^2 and the base learner and vary only the number of models, $M = 1, 5, 10, 25, 100, 1000$, for several correlations $\rho = 0, 0.2, 0.5, 0.9$, and I plot $\text{Var}(\bar{e})/\sigma^2$ against M. Every curve falls steeply at first and then flattens, and each one flattens at its own height, which is exactly ρ. Only the $\rho = 0$ curve goes all the way to zero. Read the height of the plateau as "how much error the models share" and the knee of the curve as "how many models are enough". With $\rho = 0.2$ and $M = 25$ you get $0.232\sigma^2$, not the $0.04\sigma^2$ you would naively expect from $1/M$; and with $M = 1000$ you are still at $0.2008\sigma^2$.
Example. Twenty-five intrusion-detection trees, each with error variance $\sigma^2 = 4$ and pairwise correlation $\rho = 0.2$: the ensemble variance is $4(0.2 + 0.8/25) = 0.928$. Had the errors been independent it would have been 0.16; had ρ been 0.9, it would have been 3.62, barely better than one tree.
Goes wrong. Students keep adding trees expecting the error to keep falling. Beyond the plateau, extra models cost computation and change nothing, because they are attacking the $(1 - \rho)/M$ term, which is already small. The lever that moves the plateau is ρ: lower the correlation, by bootstrap rows and random features, without raising σ^2 too much. Everything that follows in this chapter is a way of doing that.

Versus. Independent errors shrink like $1/M$; correlated errors leave a floor. And here is the link to bias that students miss: if every model is missing the same important feature, every error points in the same direction, which is the case $\rho \approx 1$.

Decision trees as base learners

What it is. A tree with J leaves cuts the feature space into axis-aligned boxes $S_1, \dots, S_J$ and predicts one constant per box, $T(x) = \sum_j \theta_j \mathbf{1}[x \in S_j]$ [on your sheet]; exactly one indicator is on for any x, so the formula just says "the constant of the box you fall in". For fixed boxes the best constant has a closed form: the mean of the leaf under squared error, the median under absolute error, the majority class under zero–one loss. The boxes themselves are chosen greedily, one split at a time, by the largest reduction in an impurity. For classification the impurity is the Gini index $I(S) = 1 - \sum_c \hat{p}_c(S)^2$ [on your sheet], which you should read as the probability that two rows drawn from the node have different classes: it is 0 for a pure node and 0.5 for a balanced binary node. The size controls are the maximum depth and the minimum leaf size.

Manipulate / measure. Let me do one split by hand so you see what "greedy" means. Eight rows sorted by x carry labels $y = (0, 0, 0, 1, 0, 1, 1, 1)$; the parent has four of each class, so $I = 0.50$. Try the threshold $x \le 3.5$: the left child is $\{0, 0, 0\}$ with impurity 0, the right child is $\{1, 0, 1, 1, 1\}$ with impurity $1 - 0.8^2 - 0.2^2 = 0.32$; weighting by size gives $\frac{3}{8} \cdot 0 + \frac{5}{8} \cdot 0.32 = 0.20$, a reduction of 0.30. Now try $x \le 4.5$: both children are $\{0, 0, 0, 1\}$ and $\{0, 1, 1, 1\}$ with impurity 0.375, so the reduction is only 0.125. The greedy rule takes 3.5, because it produces a pure leaf now, even though 4.5 splits the data in half. It does not look ahead, and it need not find the globally best tree.

Goes wrong. The temptation is to trust one deep tree because it fits the training data so well. But a small change in the data can alter an early split, and everything below that split is rebuilt; this is hierarchical lock-in, and it makes deep trees high-variance learners. You catch it by refitting on a bootstrap sample and watching the root test change. The cure is the subject of the chapter: average many of them.

Versus. Compared with logistic regression, a tree needs no feature scaling (only the order of values matters), captures interactions automatically (a test on x_1 followed by a test on x_2 is an interaction) and nonlinearity, but it is greedy and unstable. Logistic regression is stable and linear and cares about scaling. That is precisely why trees, not linear models, are the base learner of ensembles: they have the variance that averaging removes.

Hierarchical lock-in versus global adaptability

What it is. I want you to hold two pictures. One deep tree commits to its root split first, and every later split only refines the sub-region it sits in; a poor early split is never revised and is easily amplified by noise. Boosted shallow trees build the decision surface incrementally: the first tree isolates the loudest pattern, later trees target what is left, and each new tree is free to adjust the boundary anywhere in feature space, not only below some earlier test; the shrinkage η makes this convergence gradual. You should recognise the bias–variance trade-off underneath: each shallow tree has high bias and low variance, the sum of many of them drives the bias down, and η together with early stopping keeps the variance in check.

Manipulate / measure. Fix a dataset and perturb it slightly, for instance by dropping or relabelling a few rows near the root split. Refit a deep tree and a boosted model and compare the new boundaries with the old. The deep tree's root test changes and the whole subtree beneath it is rebuilt, so the boundary jumps. The boosted surface moves only locally, around the rows you changed.

Goes wrong. Students try to fix an error by growing the single tree deeper. But if the error was made at a split above, no amount of depth below can undo that early wrong test. The remedy is many shallow corrections, or averaging many trees, not a deeper single tree.

Versus. Do not confuse the two meanings of depth. In one tree, depth is how many successive tests a row passes. In boosting, the depth of each tree is the interaction order it can express (depth 1 is additive effects only, depth 2 is pairwise interactions), while the capacity of the whole model is carried by the number of rounds.

Bagging and the bootstrap

What it is. We want to give the same learner different training sets without throwing data away. A bootstrap sample does this: it is N draws with replacement from $\mathcal{D}$, so some rows appear several times and others not at all; count(i) records how often row i was drawn, and rows with count zero are called out of bag. Bagging, short for bootstrap aggregating, fits the same learner to M bootstrap samples and averages the predictions, or the probabilities [on your sheet]. How much of the data does each model see?

The probability that a given row is absent is $(1-1/N)^N$, which tends to $e^{-1} \approx 0.368$ [on your sheet]; so about 63.2% of the distinct rows are in each sample and about 36.8% are out, and it is a different third each time.

Manipulate / measure. Take $\mathcal{D} = \{1, \ldots, 8\}$ and draw eight times with replacement; one outcome is $\{2, 5, 2, 8, 1, 5, 7, 2\}$. Count each row: $(1, 3, 0, 0, 2, 0, 1, 1)$. Row 2 was drawn three times and row 5 twice, while rows 3, 4 and 6 never appeared: they are out of bag for this sample. A different count pattern gives a different empirical distribution and hence a different fitted tree. For the numbers, $(7/8)^8 = 0.344$ already at $N = 8$ and $(0.99)^{100} = 0.366$ at $N = 100$, close to the limit.

Goes wrong. The alternative that feels natural is to split $\mathcal{D}$ into M disjoint parts and fit one model per part. The trouble is that each model then sees only N/M rows, far too few unless N is enormous. The bootstrap gives every model a perturbed copy of the whole set.

Versus. Bagging randomises the rows only. A random forest, which comes next, additionally randomises the candidate features at every node.

Out-of-bag (OOB) prediction and error

What it is. Here is the free lunch that the bootstrap provides. For training row i, let $\mathcal{M}_i$ be the set of models whose bootstrap sample omitted i; there are about $0.368M$ of them. The out-of-bag prediction for that row uses only those models, $\hat{f}_{\text{OOB}}(x^{(i)}) = \frac{1}{|\mathcal{M}_i|} \sum_{m \in \mathcal{M}_i} \hat{f}_m(x^{(i)})$ [on your sheet], and the OOB error is the loss of these predictions over all training rows. Why is this honest? Because the models that vote on row i never saw it, so their prediction is a genuine unseen-data prediction even though i is a training row. You get a validation estimate without holding anything out. It is slightly pessimistic, since only a third of the trees vote for each row.

Manipulate / measure. Fix the data and the tree settings and vary the number of trees, $M = 10, 50, 100, 200, 500$, plotting the OOB error against M. The curve falls and then plateaus; that plateau is where you stop adding trees, because more trees stabilise the ensemble but do not deepen the model. Look at the left end of the curve too: with small M some rows have very few OOB trees, so the estimate is noisy there; with $M = 10$ a row has no OOB tree at all with probability $0.632^{10} \approx 0.010$.

Goes wrong. The temptation is to report the OOB error as the test performance. But if you used OOB to choose m_{try} or the leaf size, repeated OOB-based choices overfit it exactly as repeated validation overfits a validation set, and OOB is tied to the training distribution, so it says nothing about deployment shift. OOB replaces the validation set, not the untouched test set.

Versus. Four held-out notions that students conflate. OOB is the bootstrap by-product inside a bagged ensemble. Out-of-fold predictions come from an explicit fold split and work for any learner; stacking uses them. A validation set is one fixed slice. The test set is used once.

Tune the ensemble, not the trees

What it is. The tree-size setting ξ, meaning the maximum depth and the minimum leaf size, is a hyperparameter of the ensemble, not of each tree: every tree gets the same ξ, and ξ is chosen by the OOB or validation error of the whole bagged ensemble. In practice bagging uses deep trees, because the averaging removes their variance; with noisy or small data the OOB error may prefer larger leaves.

Manipulate / measure. Fix the data, vary ξ over a grid, fit a complete ensemble for each value, and plot the ensemble's OOB error against ξ. The minimum of that curve is your tree size.

Goes wrong. It feels sensible to tune each tree to its own best single-tree accuracy. But that optimises the wrong target: one tree's error rather than the average's. Worse, it pushes every tree toward the same strong fit, which raises ρ and defeats the purpose of bagging.

Random forests and m_{try}

What it is. A random forest is bagging plus one extra randomisation: at each node, draw m_{try} of the d features at random and choose the best split among those candidates only [on your sheet]; a fresh subset is drawn at every node. Setting $m_{\text{try}} = d$ recovers bagged trees; common starting values are $\sqrt{d}$ for classification and $d/3$ for regression. Why bother? Because if one feature is very strong, almost every bootstrap tree will put it at the root, the trees will have similar shapes and make similar mistakes, and ρ stays large. Withholding that feature at most nodes forces the trees to start differently. The controls are M (stability), m_{try} (strength versus correlation), the minimum leaf size and the maximum depth.

Manipulate / measure. Fix the data, vary m_{try}, and record two things at each value: the error of a single tree and the error of the forest. As m_{try} shrinks, each tree gets a little weaker (its σ^2 rises) but the trees become much less alike (ρ falls), and the forest error goes down although every tree in it is worse.

Push m_{try} too small and the nodes are starved of good candidates, so the splits are poor and the forest error rises again. For the arithmetic, with $d = 25$ and $m_{\text{try}} = 5$ a given feature is offered at a node with probability 0.2, and it is absent from all ten roots of a ten-tree forest with probability $0.8^{10} = 0.107$.

Goes wrong. The mistake is to judge the forest by the quality of its trees. The target is the average's error, $\sigma^2(\rho + (1-\rho)/M)$, and a forest of individually weaker but less correlated trees can beat a forest of strong, similar ones. That is the content of the strength-versus-correlation trade-off.

Versus. Large m_{try} gives strong, similar trees (small σ^2, large ρ); small m_{try} gives weaker, diverse trees. The strong feature is not lost: it is still offered at a fraction m_{try}/d of the nodes and gets used deeper down.

Example. On flow data with $d = 16$ features, $m_{\text{try}} = 4$ candidates per node. The destination-port feature, which would sit at the root of every bagged tree, is offered at only a quarter of the nodes, so the other features get their turn and the trees stop looking alike.

Feature importance: MDI versus permutation

What it is. A forest of hundreds of trees is not readable, so we summarise which features it relies on. The first summary is the mean decrease in impurity. Each split earns a credit $\Delta_t = \frac{N_t}{N}\left[I(S_t) - \frac{N_L}{N_t}I(S_L) - \frac{N_R}{N_t}I(S_R)\right]$, its impurity reduction weighted by the fraction of data it acts on, and that credit goes to the feature used at the split; sum a feature's credits, average over trees, normalise to one [on your sheet]. It is free, computed during training, but biased toward features with many possible split points, and it measures the training fit. The second summary is permutation importance: shuffle the values of x_j among the OOB rows, keeping every other column and the labels fixed, predict again without retraining, and record the increase in error, $I_j = \widehat{\text{Err}}_{\text{shuffled}(j)} - \widehat{\text{Err}}_{\text{orig}}$ [on your sheet]. Shuffling destroys the information in the feature while preserving its marginal distribution, so the trees still receive plausible values; this measures reliance in prediction.

Manipulate / measure. Let me compute both. A root node holds 5 attacks and 5 benign flows, so its Gini is 0.5. The test "port = 22" sends 4 benign flows left (Gini 0) and 5 attacks plus 1 benign right (Gini $1 - (5/6)^2 - (1/6)^2 = 10/36 = 0.278$). The weighted child impurity is $\frac{4}{10} \cdot 0 + \frac{6}{10} \cdot 0.278 = 0.167$, and since the root sees all the data the credit is $0.5 - 0.167 = 0.333$, booked to destination port. Now two trees: tree 1 credits port 0.18 and duration 0.06, tree 2 credits port 0.12 and packets 0.04. Averaging over the two trees gives 0.15, 0.03, 0.02, which sum to 0.20; normalising gives 0.75/0.15/0.10. For permutation, on 100 OOB flows the forest makes 10 errors; shuffling port raises that to 18 (an increase of 0.08), shuffling duration to 13 (+0.03), shuffling packets to 11 (+0.01). Breaking the port adds eight errors: that is what "important for this fitted forest" means.

Goes wrong. Two misreadings. The first is importance as causality: a high score means the fitted model relies on the feature, which may be a proxy (destination port stands in for the attacked service), a consequence recorded after the fact (leakage), or an artefact of collection; intervening on it would change nothing. The second is masking: if two features carry the same information, shuffling one alone hurts little because the trees that use the other still have the information, so both look unimportant. Permute them jointly, or drop one and refit.

Versus. MDI is training-time, free, and biased toward features with many split points. Permutation importance is OOB- or validation-time, honest about prediction, and masked by correlated features. When the two rankings disagree, that disagreement itself tells you which bias is at work.

Boosting for regression

What it is. Bagging fits many models independently and averages. Boosting asks a different question: what should the next model learn from the current mistakes? The ensemble after m rounds is $F_m(x) = F_{m-1}(x) + \eta T_m(x)$, so unrolled $F_M = F_0 + \eta \sum_m T_m$ [on your sheet]: a constant plus a scaled sum of trees, where each tree predicts a correction rather than the whole target. For squared error something beautiful happens: the loss at round m is $\frac{1}{2}\sum_i \left(r_m^{(i)} - \eta T_m(x^{(i)})\right)^2$ with $r_m^{(i)} = y^{(i)} - F_{m-1}(x^{(i)})$ the current residual, so the next tree is simply a regression on the residuals, which are also the negative gradient of the loss. The algorithm: start at $F_0 = \bar{y}$; compute the pseudo-residuals $r_m = -\partial\ell/\partial F$; fit a tree to them; set each leaf's output to the mean residual in that leaf; update with the learning rate η. Each shallow tree has high bias and low variance, the sum lowers the bias, and η with early stopping keeps the variance in check.

Manipulate / measure. One round by hand. Three rows with $y = (10, 20, 30)$ and $\eta = 0.1$. Initialise at the mean, $F_0 = 20$, so the residuals are $(-10, 0, 10)$. A split at $x < 2.85$ puts the first row alone in one leaf and the other two together; the leaf outputs are the mean residuals, -10 and $(0 + 10)/2 = 5$. Update: $F_1 = (20-1,\ 20+0.5,\ 20+0.5) = (19, 20.5, 20.5)$, and the new residuals are $(-9, -0.5, 9.5)$; the next tree is

fitted to those. Now the curve experiment: plot training and validation loss against the number of rounds for $\eta = 0.3$ and for $\eta = 0.03$. Both training curves fall forever. Each validation curve bottoms out and turns up, the small-rate curve later and usually slightly lower, and the number of rounds you keep is the round at the minimum for that rate.

Goes wrong. Students tune M, η and the depth one at a time. They are coupled: halving η roughly doubles the best M, and deeper trees reach the validation minimum in fewer rounds. Search over (η, depth) on a grid, let early stopping choose M for each pair, and compare configurations at their own selected round.

Versus. Bagging averages M full predictions, made in parallel, and attacks variance. Boosting sums M corrections, made sequentially, and attacks mostly bias. One deep tree locks in its early splits; boosted shallow trees can adjust the boundary anywhere at every round. Depth here limits the interaction order of each tree, and two to six is the usual range.

Example. Think of credit scoring. The first tree learns the coarse rule "high debt-to-income means risky". Its residuals are now large for a particular group, say young applicants with short credit histories who were scored too harshly; the second tree is fitted to exactly those leftovers and corrects them, and so on. Each tree is a specialist on what the previous ones got wrong, which is why a sequence of weak, shallow trees ends up with a lower bias than any of them alone.

Boosting for binary classification

What it is. Probabilities live in $(0, 1)$, so you cannot add tree outputs to them: a probability of 0.9 plus a correction of 0.4 is 1.3. The fix is to boost the logit F and let the sigmoid bring you back, $\hat{p} = \sigma(F)$, with the loss $\ell = -[y \ln \hat{p} + (1 - y) \ln(1 - \hat{p})]$ [on your sheet]; this is the discriminative route of Chapter 4 with the score now a sum of trees. Two derivatives in the logit carry everything: $g = \partial \ell / \partial F = \hat{p} - y$ and $h = \partial^2 \ell / \partial F^2 = \hat{p}(1 - \hat{p})$ [on your sheet]. The pseudo-residual is $-g = y - \hat{p}$, how far the probability is from the label, and h is the Bernoulli variance, 0.25 when the model is unsure at $\hat{p} = 0.5$ and near zero when it is confident. Now the difficulty: the leaf output that minimises a sum of log-sigmoids has no closed form, so each leaf needs a numerical line search, which is the bottleneck. A second-order expansion of the loss restores a closed form, the Newton leaf $w = \sum(-g) / \sum h$, and with an L_2 penalty on the output, $w = \sum(-g) / (\sum h + \lambda)$ [not on your sheet: know it].

Manipulate / measure. Take a leaf holding three rows with $(y, \hat{p}) = (1, 0.2), (1, 0.4), (0, 0.8)$. The gradients $g = \hat{p} - y$ are $(-0.8, -0.6, 0.8)$, summing to $G = -0.6$; the Hessians $h = \hat{p}(1 - \hat{p})$ are $(0.16, 0.24, 0.16)$, summing to $H = 0.56$. The unregularised leaf value is $0.6 / 0.56 = 1.071$; with $\lambda = 1$ it is $0.6 / 1.56 = 0.385$; and with $\eta = 0.1$ the logit actually moves by 0.0385. The positive sign raises all three probabilities, helping the two positives more than it hurts the negative. If you now vary λ, you will see the leaf outputs shrink, and shrink most where H is small.

Goes wrong. Picture a leaf full of confidently-wrong rows: $y = 1$ with a very negative F. Then $-g \to 1$, the largest possible residual, while $h \to 0$, almost no curvature. The Newton step G/H divides a large number by a tiny one and explodes, which is extreme overfitting on exactly the rows you can trust least. The λ in the denominator is a floor that caps the step, and a minimum child weight refuses such leaves altogether.

Versus. In regression boosting the residual is $y - F$ and the leaf output is the mean residual, a closed form. In classification the residual is $y - \hat{p}$ and the leaf output is a Newton step, residual divided by curvature. Same skeleton, different leaf arithmetic.

XGBoost: one objective for structure and leaf values

What it is. The gradient-boosting variants above still grow the tree structure by plain squared-error splits on the raw residual and only afterwards compute the Newton outputs; the split search is blind to curvature. XGBoost scores a whole proposed tree with one number, $\mathcal{L} = \sum_q \sum_{i \in S_q} \left[g^{(i)} w_q + \frac{1}{2} h^{(i)} w_q^2 \right] + \gamma Q + \frac{\lambda}{2} \sum_q w_q^2$ [on your sheet]: the second-order change in loss, a price γ per leaf, and a penalty on large outputs. Completing the square turns each row's term into $\frac{1}{2} h^{(i)} \left(r_{\text{XGB}}^{(i)} - w_q \right)^2$ with target $r_{\text{XGB}} = -g/h = (y - \hat{p}) / (\hat{p}(1 - \hat{p}))$ and weight h. So the objective is a weighted least-squares problem, and the tree is grown by weighted splits on these Newton targets: gradients, Hessians and regularisation now act while the tree is being built. The optimal leaf is $w_q^* = -G_q / (H_q + \lambda)$ with value $-\frac{1}{2} G_q^2 / (H_q + \lambda)$, and a split is worth gain $= \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{G^2}{H + \lambda} \right] - \gamma$, accepted only if positive [on your sheet]. Read the two penalties differently: λ sits inside every denominator and shrinks every output continuously, most where H is small; γ is a flat threshold that discourages marginal splits. Further controls are row and feature subsampling, a minimum child weight that rejects a split whose child has tiny H (this directly prevents the exploding

step), and depth or leaf limits; the speed comes from histogram-based split search, sparsity handling and parallel evaluation across features, while the rounds themselves stay sequential.
Manipulate / measure. Does a split pay for its leaf? Parent $(G, H) = (-4, 3)$, children $(-3, 1)$ and $(-1, 2)$, with $\lambda = 1$ and $\gamma = 0.2$. The parent's best value is $-\frac{1}{2} \cdot 16/4 = -2$. The left child gives $-\frac{1}{2} \cdot 9/2 = -2.25$ and the right $-\frac{1}{2} \cdot 1/3 = -0.167$, together -2.417. The fit improves by 0.417, and after paying γ the gain is $0.217 > 0$: accept. The same split would be rejected for any $\gamma \geq 5/12 \approx 0.417$. If you vary λ at fixed depth and rate you see fewer and smaller leaf values; if you vary γ you see fewer leaves.
Goes wrong. Students see the target for a confidently-wrong row, $(y - \hat{p})/(\hat{p}(1 - \hat{p})) \approx 1/\hat{p}$ when $y = 1$ and $\hat{p} \to 0$, and conclude the step will be huge. Read the target together with its weight: the weight h is tiny, and the product $h\, r_{\text{XGB}} = y - \hat{p}$ is bounded by one. That is why the leaf values stay finite.
Example. To see why target and weight must be read together, take a row with $y = 1$ and $\hat{p} = 0.01$: its target is $0.99/0.0099 = 100$, a request to move the logit by a hundred, but its weight is $h = 0.0099$, so it pulls on the leaf with a force of $h \cdot r = 0.99$, the same bounded force as its raw residual. A row with $y = 1$ and $\hat{p} = 0.5$ asks for $0.5/0.25 = 2$ with weight 0.25, force 0.5. The confident row shouts a huge number in a whisper; the unsure row asks for a moderate correction in a clear voice.
Versus. Four leaf-value mechanisms, in order. Basic boosting grows the structure on the raw error $y - \hat{p}$ and finds outputs by an exact line search. Unregularised GBM: same structure, Newton outputs $\sum(-g)/\sum h$. Regularised GBM: same structure, $\sum(-g)/(\sum h + \lambda)$. XGBoost grows the structure on $-g/h$ weighted by h, uses the same regularised leaf, and includes γ in the gain, all in one pass.

The five knobs of boosted trees, and what each one manipulates

What it is. Let me name the knobs and what each one does. The number of rounds M is capacity over time: too few underfits, too many fits noise. The learning rate η is the size of each correction; roughly $\eta \propto 1/M$, and a smaller rate usually generalises slightly better at more cost. The depth, or number of leaves Q, is the interaction order a tree can express, with two to six usual. The penalty λ shrinks every leaf output continuously, most where H is small. The penalty γ is a flat price per leaf that rejects marginal splits. On top of these come row and feature subsampling, which import the random-forest idea into boosting, and the minimum child weight.
Manipulate / measure. Run a grid over (η, depth), choosing M by early stopping on validation for each pair, and compare the configurations by their validation loss at their own selected round. Then run two separate sweeps at fixed depth and rate: as λ grows, the leaf values shrink and extreme outputs disappear; as γ grows, each tree has fewer leaves. Each penalty leaves its own fingerprint.
Goes wrong. A colleague switches η from 0.1 to 0.01 and keeps $M = 400$, then reports a worse validation loss. The model is underfitted: only a tenth of the total correction was applied. Re-select M by early stopping, which will land around ten times more rounds.
Versus. Keep λ and γ apart as continuous shrinkage inside every denominator versus a discrete threshold on the gain. Keep η and M apart as step size versus number of steps, and remember that they are coupled.

Multi-class boosting, AdaBoost, libraries

What it is. With K classes the ensemble keeps K logits and maps them through the softmax, $\hat{p}_k = e^{F_k} / \sum_j e^{F_j}$, with loss $-\sum_k y_k \ln \hat{p}_k$; the gradient is $g_k = \hat{p}_k - y_k$ and the Hessian has $\hat{p}_k(1 - \hat{p}_k)$ on the diagonal and $-\hat{p}_k \hat{p}_r$ off it [on your sheet]. The softmax couples the classes, because raising F_1 lowers every other probability, and that is what the off-diagonal terms record. Keeping only the diagonal separates the objective into K binary-style problems, so the algorithm builds K trees per round, one per class, which can be fitted in parallel; the rounds stay sequential because the softmax is recomputed jointly from all updated logits before the next round. It is K times the work per round. Finally, a matter of vocabulary: AdaBoost is a different boosting algorithm, which reweights misclassified rows and combines weak classifiers by a weighted vote, whereas XGBoost, LightGBM (histogram splits, leaf-wise growth) and CatBoost (ordered handling of categorical features to limit target leakage) are libraries implementing gradient-boosted trees.
Goes wrong. Two slips I see every year: listing AdaBoost, XGBoost, LightGBM and CatBoost as four algorithms of the same kind, and calling the K trees "independent" across rounds, when they are independent only within a round and only under the diagonal approximation.

Bagging versus boosting; when no ensemble helps

What it is. Put the two families side by side. Bagging and forests fit parallel, randomised, deep trees and average their unstable predictions; they attack variance, they are robust to noise, they parallelise naturally, and their controls are m_{try} and the leaf size. Boosting fits sequential, shallow corrections and

reduces the current loss; it attacks mostly bias, it can chase noisy or mislabelled rows because it treats every residual as signal, and its controls are η, depth and rounds. So start from the source of the error rather than from a favourite method. Is one strong learner unstable across samples? Bag it or grow a forest. Is there a structured pattern left in the residuals? Boost. Do credible but different model families make complementary errors? Try validated voting. Is the dominant error due to missing information or deployment shift? No ensemble will help.

Manipulate / measure. Flip 10% of the training labels, fit a forest and a boosted model, and plot validation loss against the number of trees or rounds. The forest degrades gently, because averaging dilutes the flipped rows. The boosted model's validation loss turns up early, because later trees are fitting the flipped rows as if they were signal; early stopping and a smaller η limit the damage.

Goes wrong. Expecting an ensemble to fix a missing feature (that is $\rho \approx 1$), identical models, noisy labels in boosting, overused validation data, or a train-to-deployment shift. And remember that extra models cost fit time, prediction time and memory, which Lab 3 measures.

Versus. The average of predictions, with diversity from rows and feature subsets, versus the sum of corrections, with diversity from the changing residuals.

Stacking

What it is. Voting fixes the combination rule in advance. Stacking learns it: the base predictions z_m, usually probabilities, are fed to a meta-model h_γ, for instance $\hat{p}_{\text{stack}} = \sigma(\beta_0 + \sum_m \beta_m z_m)$ fitted by cross-entropy on the true labels [on your sheet]; soft voting is the special case $\beta_m = 1/M$. Keep the meta-model simple, a regularised logistic regression on only M inputs, because it must not overfit. The one procedural rule that keeps this honest: train the meta-model on out-of-fold predictions. Split the training set into K folds, fit every base model on the other folds, predict the held-out fold, join the held-out predictions, and fit the meta-model on those. To predict a new row, refit each base model on all the training data and pass their outputs through the fitted meta-model.

Manipulate / measure. Fit the meta-model once on in-sample base predictions and once on out-of-fold predictions, and compare the validation loss. In-sample, a deep forest predicts its own training rows almost perfectly, so the meta-model learns to trust the most overfitted base model and its own fit looks unrealistically good. Out-of-fold, the weights reflect genuine complementarity between the base models.

Goes wrong. Training the meta-model on in-sample predictions leaks training information into the combination rule. And do not confuse out-of-fold with OOB: OOB is bagging's by-product and exists only inside a bagged ensemble, whereas out-of-fold is an explicit fold split that works for any base learner.

Versus. Voting fixes the weights in advance; stacking learns one global set of weights; a mixture of experts, next, learns weights that depend on the input.

Example. Stack logistic regression, a forest and a boosted model with a regularised logistic meta-model. A small or negative β_m marks a base model that is redundant given the others; a large one marks the model the others cannot replace.

Mixture of experts

What it is. Stacking learns one combination for all inputs. A mixture of experts lets the combination depend on x: $p(y \mid x) = \sum_m g_m(x) p_m(y \mid x)$, where the gate $g_m(x) = \text{softmax}(a_m(x))$ with $a_m(x) = \beta_m^\top x + b_m$, or a small network for nonlinear routing [on your sheet]. Soft routing uses all experts with their weights; top-k routing evaluates only the k experts with the largest gate scores, which is conditional computation and is how very large language and vision models use the idea. Gate and experts are trained jointly by minimising $-\sum_i \log \sum_m g_m(x^{(i)}) p_m(y^{(i)} \mid x^{(i)})$: experts improve on the rows where they receive weight, the gate learns which expert is good where, and the regions are not specified in advance.

Manipulate / measure. Track each expert's share of the gate weight over training. With joint updates the shares separate by region, which is specialisation. Without a load-balancing penalty, an expert that received more weight early learns faster, becomes better, attracts still more inputs, and ends up handling almost everything while the others sit idle: expert collapse.

Goes wrong. Letting the gate collapse onto one expert. The fix is a load-balancing term that encourages the gate to spread its weight.

Versus. Stacking uses the same weights for every input and is fitted after the base models. A mixture of experts uses input-dependent weights and is trained together with the experts.

Where each method is used, and Lab 3

What it is. Voting and averaging appear in medical prediction, sensor fusion and deep-network ensembles, where different models are combined for stabler predictions. Random forests are the strong baseline for intrusion detection, remote sensing, bioinformatics and tabular problems in general, because they handle nonlinearity and interactions with little preprocessing. Gradient-boosted trees dominate credit scoring, fraud detection, churn and ranking on tabular data, where successive corrections buy the highest accuracy. Stacking shows up in competitions and in systems that combine model families. Mixtures of experts power large language and vision models and multi-domain prediction, through specialisation and conditional computation. The right method still depends on the data, the cost and the validation results.
Manipulate / measure. Lab 3 runs on the cybersecurity flows with one fixed stratified split and identical rows for every model, and each experiment asks one question: bagged trees against a random forest (how much does aggregation stabilise?); gradient boosting against XGBoost (how do rate, depth and rounds interact?); the best Lab 2 models against both ensembles (which tree errors get corrected?); the multiclass problem (which attack classes stay hard?); with and without destination port (does service information help?); and fit time, prediction time and model size (what does the accuracy cost?).
Goes wrong. Comparing models fitted on different rows or splits: the differences then reflect the data, not the methods.

Reading the curves of this chapter

What it is. Four plots carry the chapter, and I want you to recognise each on sight. First, error against M for bagging or a forest: it falls and then plateaus at the floor, and the forest's plateau sits lower than bagging's. Second, single-tree error and forest error against m_{try}: the tree curve rises as m_{try} shrinks, while the forest curve is U-shaped with its minimum near $\sqrt{d}$. Third, training and validation loss against boosting rounds: training falls monotonically, validation is a U, and the minimum moves right as η or the depth shrinks; the number of rounds is read at that minimum. Fourth, importance bars: when the MDI and permutation rankings disagree, suspect a feature with many split points (MDI inflated) or correlated features (permutation masked).
Manipulate / measure. On any of these plots ask the three questions in order. Are both curves high? Then it is bias: the base learner or the features are too weak, and no ensemble fixes that. Is the gap between them large? Then it is variance: use more models, more diverse models, or early stopping. Are the curves still moving? Then you have not reached the plateau or the minimum yet.
Goes wrong. Reading a falling training loss over rounds as progress, when only the validation curve can tell you that. And reading an OOB plateau as "the model cannot improve": it means that M cannot improve it; m_{try}, the depth or the features still can.

The numbers you should be able to reproduce

What it is. Before the exam, check that you can redo each of these on paper without looking. The soft vote is $(0.49+0.48+0.95)/3 = 0.64$. The variance with $\rho = 0.2$ and $M = 25$ is $0.2+0.8/25 = 0.232$ times σ^2. The balanced Gini is 0.5; the right child $\{1,0,1,1,1\}$ has $1-0.64-0.04 = 0.32$ and the child $\{0,0,0,1\}$ has $1-0.5625-0.0625 = 0.375$, giving reductions of 0.30 and 0.125. The out-of-bag numbers are $(7/8)^8 = 0.344$, $(0.99)^{100} = 0.366$ and $e^{-1} = 0.368$. A feature is offered with probability $5/25 = 0.2$ and absent from ten roots with probability $0.8^{10} = 0.107$. The MDI credit is $0.5 - 0.167 = 0.333$, and the forest-level MDI goes from 0.15/0.03/0.02 to 0.75/0.15/0.10; the permutation increases are $+0.08/+0.03/+0.01$. The boosting round starts at $F_0 = 20$, uses leaves -10 and 5, reaches $F_1 = (19, 20.5, 20.5)$ and leaves residuals $(-9, -0.5, 9.5)$. The Newton leaf with $G = -0.6$ and $H = 0.56$ is 1.071, becomes 0.385 with $\lambda = 1$, and moves the logit by 0.0385 after $\eta = 0.1$. In XGBoost the parent is worth -2, the children -2.25 and -0.167, and the gain is $0.417 - 0.2 = 0.217$, with rejection exactly when $\gamma \geq 5/12$.

The ensemble methods side by side

Method	Where diversity comes from	How combined	What you manipulate	Main risk
Voting / averaging	different algorithms, settings, features, seeds on the same data	fixed vote or average	which models; hard vs soft	correlated errors; one overconfident model
Bagging	bootstrap rows	average / vote	M, tree size (ensemble-level)	trees still alike if one feature dominates
Random forest	bootstrap rows + random candidate features per node	average / vote	m_{try}, min leaf, M to the OOB plateau	m_{try} too small gives poor splits
Boosting	each tree targets current residuals / gradients	sum of corrections	η, depth, rounds by early stopping; λ, γ	fits noise and wrong labels; coupled knobs
Stacking	different model families	learned global weights	meta-model, out-of-fold folds	in-sample leakage into the meta-model
Mixture of experts	joint training specialises experts	input-dependent gate	gate, top-k, load balancing	expert collapse

Variance of the average of M correlated errors

Setup: $\text{Var}(e_m) = \sigma^2$ for every model and $\text{Corr}(e_m, e_{m'}) = \rho$ for every pair; we want the variance of $\bar{e} = \frac{1}{M}\sum_m e_m$.
Expand the variance of a sum into variances plus covariances; this is allowed for any random variables: $\text{Var}(\sum_m e_m) = \sum_m \text{Var}(e_m) + \sum_{m \neq m'} \text{Cov}(e_m, e_{m'})$.
Count the terms, which is where the structure enters: there are M variance terms worth σ^2 each and $M(M-1)$ ordered pairs worth $\rho\sigma^2$ each, giving $M\sigma^2 + M(M-1)\rho\sigma^2$.
Divide by M^2 because $\bar{e}$ is the sum over M: $\frac{\sigma^2}{M} + \frac{(M-1)\rho\sigma^2}{M} = \sigma^2\big(\rho + \frac{1}{M} - \frac{\rho}{M}\big) = \sigma^2\big(\rho + \frac{1-\rho}{M}\big)$.
Read the result: the M diagonal terms shrink like $1/M$ after the division, but there are M^2-many cross terms, so they do not shrink, and that leaves the floor $\rho\sigma^2$ as $M \to \infty$.

Out-of-bag fraction $(1 - 1/N)^N \to e^{-1}$

One draw picks each row with probability $1/N$, so it misses row i with probability $1 - \frac{1}{N}$.
The N draws are independent because we draw with replacement, so all of them miss row i with probability $(1 - \frac{1}{N})^N$.
Take the limit using $(1 + \frac{x}{N})^N \to e^x$ with $x = -1$: the probability tends to $e^{-1} \approx 0.368$, and it is already $(7/8)^8 = 0.344$ at $N = 8$ and $(0.99)^{100} = 0.366$ at $N = 100$.
Conclude: about 63.2% of the distinct rows are in each bootstrap sample and about 36.8% are out of bag, so each row has about $0.368M$ OOB models voting for it.

Probability that a feature is offered at a node, and at no root

Choosing m_{try} of d features without replacement, a given feature is included with probability m_{try}/d; if you want the count, it is $\binom{d-1}{m_{\text{try}}-1}/\binom{d}{m_{\text{try}}} = m_{\text{try}}/d$.
With $d = 25$ and $m_{\text{try}} = 5$ the feature is offered with probability 0.2 and is absent from one root with probability 0.8.
Different trees draw their root candidates independently, so the feature is absent from all $M = 10$ roots with probability $0.8^{10} \approx 0.107$; about 2 of the 10 roots use it, against all 10 in bagging.
Read: the strong feature is not lost, it is used at a fraction m_{try}/d of the nodes, deeper down; the trees start differently and ρ falls.

Softmax derivatives for multi-class boosting

Write the loss with a one-hot label: $\ell = -\sum_k y_k \ln \hat{p}_k = -\sum_k y_k F_k + \ln \sum_j e^{F_j}$, which uses $\ln \hat{p}_k = F_k - \ln \sum_j e^{F_j}$ and $\sum_k y_k = 1$.
Differentiate once: $\partial \ell / \partial F_k = -y_k + e^{F_k} / \sum_j e^{F_j} = \hat{p}_k - y_k$, the same shape as the binary gradient.
Differentiate $\hat{p}_k$ by the quotient rule: $\partial \hat{p}_k / \partial F_r = \hat{p}_k(\mathbf{1}[k = r] - \hat{p}_r)$, which is $\hat{p}_k(1 - \hat{p}_k)$ on the diagonal and $-\hat{p}_k \hat{p}_r$ off it; the off-diagonal terms say that raising F_1 lowers every other probability.
Keep only the diagonal, $H \approx \operatorname{diag}(\hat{p}_k(1 - \hat{p}_k))$: the objective becomes a sum of K binary-style problems, one tree per class per round with target $(y_k - \hat{p}_k)/(\hat{p}_k(1 - \hat{p}_k))$, and the coupling is restored when the softmax is recomputed.

Optimal leaf constant (squared error: the mean)

For fixed regions the objective $\sum_i \ell(y^{(i)}, \sum_j \theta_j \mathbf{1}[x^{(i)} \in S_j])$ splits into one sum per leaf, because each row lies in exactly one leaf; so minimise $\sum_{i \in S_j} \ell(y^{(i)}, \theta)$ separately for each leaf.
For squared error set the derivative to zero: $\frac{d}{d\theta} \sum_i (y^{(i)} - \theta)^2 = -2 \sum_i (y^{(i)} - \theta) = 0$, which gives the leaf mean.
For absolute error the derivative is $\sum_i \operatorname{sign}(\theta - y^{(i)})$, which changes sign at the median; for zero–one loss the leaf should name the class that is wrong on the fewest rows, the majority.

Squared-error boosting fits the residuals

Start from the loss after round m: $L = \frac{1}{2} \sum_i (y^{(i)} - F_m(x^{(i)}))^2$ with the update $F_m = F_{m-1} + \eta T_m$.
Substitute the update and group what is already frozen: $y^{(i)} - F_{m-1}(x^{(i)}) =: r_m^{(i)}$ is a fixed number at round m, because F_{m-1} was fitted in earlier rounds.
The loss becomes $L = \frac{1}{2} \sum_i (r_m^{(i)} - \eta T_m(x^{(i)}))^2$, which is exactly the least-squares objective of a regression whose targets are the residuals and whose predictor is ηT_m.
The gradient view explains the general recipe: $-\partial \ell / \partial F = -\partial [\frac{1}{2}(y - F)^2] / \partial F = y - F = r$, so "fit the residual" and "fit the negative gradient" coincide for squared error; for other losses you fit the negative gradient and then set the leaf outputs by minimising the true loss.

$g = \hat{p} - y$ and $h = \hat{p}(1 - \hat{p})$ for the cross-entropy in the logit

First the sigmoid's derivative: with $\sigma = (1 + e^{-F})^{-1}$, $\sigma' = e^{-F}(1 + e^{-F})^{-2} = \sigma \cdot \frac{e^{-F}}{1 + e^{-F}} = \sigma(1 - \sigma)$.
Apply the chain rule to $\ell = -[y \ln \hat{p} + (1 - y) \ln(1 - \hat{p})]$: $\frac{\partial \ell}{\partial F} = \big(-\frac{y}{\hat{p}} + \frac{1 - y}{1 - \hat{p}} \big) \hat{p}(1 - \hat{p})$.
Multiply out: $-y(1 - \hat{p}) + (1 - y)\hat{p} = -y + y\hat{p} + \hat{p} - y\hat{p} = \hat{p} - y = g$.
Differentiate once more: $\partial(\hat{p} - y)/\partial F = d\hat{p}/dF = \hat{p}(1 - \hat{p}) = h$, the Bernoulli variance, 0.25 at $\hat{p} = 0.5$ and near zero when the model is confident.

Newton leaf value with λ

Expand row i's loss to second order around the current logit, which is legitimate for a small step: $\ell(F + w) \approx \ell(F) + g^{(i)} w + \frac{1}{2} h^{(i)} w^2$.
Sum over the leaf and add the penalty: $\sum_i g^{(i)} w + \frac{1}{2} \big(\sum_i h^{(i)} + \lambda \big) w^2 + \text{const} = G_q w + \frac{1}{2}(H_q + \lambda) w^2$.
This is a quadratic in one number with positive curvature, since $h > 0$, so its derivative $G_q + (H_q + \lambda) w = 0$ gives the minimiser $w_q^* = -G_q / (H_q + \lambda) = \sum (y - \hat{p}) / (\sum \hat{p}(1 - \hat{p}) + \lambda)$.
Read: residual divided by curvature; λ is a floor on the denominator that matters most where H_q is smallest, the confident leaves, which is exactly where the unregularised step explodes.

XGBoost: completing the square, the leaf value and the gain

Complete the square: $\frac{1}{2} h (w + \frac{g}{h})^2 = \frac{1}{2} h w^2 + g w + \frac{g^2}{2h}$, so $g w + \frac{1}{2} h w^2 = \frac{1}{2} h \big(w - (-\frac{g}{h}) \big)^2 - \frac{g^2}{2h}$, and the last term does not involve w.
Hence $\mathcal{L} = \sum_q \sum_{i \in S_q} \frac{1}{2} h^{(i)} (r_{\text{XGB}}^{(i)} - w_q)^2 + \gamma Q + \frac{\lambda}{2} \sum_q w_q^2 + \text{const}$ with $r_{\text{XGB}} = -g/h$: a weighted least-squares problem with weight h, which is why the split search itself uses the Hessian.
For one leaf, $\phi_q(w) = G_q w + \frac{1}{2}(H_q + \lambda) w^2$ is minimised at $w_q^* = -G_q / (H_q + \lambda)$ with value $\phi_q(w_q^*) = -\frac{1}{2} G_q^2 / (H_q + \lambda)$.
Replacing one leaf by two changes the fit by $\phi_L + \phi_R - \phi_{\text{parent}}$ and the penalty by $+\gamma$, so gain $= \frac{1}{2} \big[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{G^2}{H + \lambda} \big] - \gamma$, and the split is accepted only if the gain is positive.

Check with the numbers: $(-4, 3) \to (-3, 1), (-1, 2)$ and $\lambda = 1$ gives $-2 \to -2.25 - 0.167$, an improvement of 0.417 and a gain of $0.417 - \gamma$.

How he asks it

Q. *Explain why adding more trees to a bagged ensemble stops helping after some point, and what a random forest changes about that.*
A. The claim is that the variance of the average has a floor. By the formula $\sigma^2(\rho + (1-\rho)/M)$, the $(1-\rho)/M$ term vanishes as you add trees, but the $\rho\sigma^2$ term, the error that all the trees share, does not move at all. Bagged trees share a lot of their error because a strong feature sits at the root of nearly every one of them, so their shapes and their mistakes are alike. A random forest offers only m_{try} random features at each node, so the trees are forced to differ: ρ falls, each tree gets slightly weaker, and the floor drops. You therefore add trees only until the OOB error plateaus, and you lower the plateau with m_{try}, not with more trees.

Q. *A student reports the OOB error of a tuned random forest as its final test performance. What is wrong?*
A. The OOB error is a validation score, not a test score. It was used to choose m_{try} and the leaf size, so the reported value is the minimum over several candidates and is optimistic by the winner's curse. It is also computed on rows from the training distribution, so it is blind to any shift at deployment. OOB saves you a validation set for free; it does not replace the untouched test set, on which the chosen forest should be scored once.

Q. *Why does gradient boosting for classification need a second-order (Newton) step, and why must it be regularised?*
A. Because you cannot add tree outputs to probabilities, boosting works on the logit, and there the leaf output that minimises a sum of log-sigmoids has no closed form, which would force a numerical line search in every leaf. A second-order expansion of the loss turns each leaf into a quadratic and gives the closed form $w = \sum(-g)/\sum h$ with $g = \hat{p} - y$ and $h = \hat{p}(1-\hat{p})$. The danger appears in a leaf of confidently-wrong rows: there $-g \to 1$ while $h \to 0$, so the step divides a large residual by almost no curvature and explodes. Putting λ in the denominator caps the step, and a minimum child weight refuses such leaves outright.

Q. *Derive what the next tree fits in squared-error boosting and say what replaces it for cross-entropy.*
A. With $F_m = F_{m-1} + \eta T_m$ the loss is $\frac{1}{2}\sum(r_m - \eta T_m)^2$, where $r_m = y - F_{m-1}$ is frozen at round m, so the new tree is a least-squares fit of the current residuals. Those residuals are also the negative gradient $-\partial\ell/\partial F$ of the squared loss, which is why the two descriptions coincide. For cross-entropy the negative gradient is $y - \hat{p}$: the tree structure is fitted to that, and each leaf output is then set by a Newton step, residual over curvature. XGBoost goes one step further and grows the tree directly on the Newton target $-g/h$ with weights h.

Q. *Compare MDI and permutation importance on a dataset with two near-duplicate features.*
A. MDI splits the credit between the twins according to which one the trees happened to pick at each split, and it favours whichever has more split points, so the ranking between them is partly an accident. Permutation importance shuffles one twin while the other still carries the information, so the error barely moves and both twins look unimportant; this is masking. Neither method says the pair is useless, and neither is causal: a high score only means the fitted model relies on the feature. To see the shared contribution, permute the two jointly, or drop one and refit.

Q. *Why must a stacking meta-model be trained on out-of-fold predictions, and how is that different from OOB?*
A. If you fit the meta-model on in-sample predictions, a deep forest predicts its own training rows almost perfectly, so the meta-model learns to trust the most overfitted base model and its own fit becomes unrealistically good: training information has leaked into the combination rule. Out-of-fold predictions avoid this because they come from base models that never saw the row, through an explicit K-fold split that works for any learner. OOB is a different thing: it is the bootstrap by-product inside a bagged ensemble, and it exists only there.

Five mistakes

- Expecting M to drive the variance to zero: the floor is $\rho\sigma^2$, and averaging never removes bias or shared missing information.
- Soft-voting uncalibrated models; tuning each tree instead of the ensemble; reading the OOB error as a test score.

- Confusing the gradient with the negative gradient ($g = \hat{p} - y$, residual $y - \hat{p}$); forgetting that classification boosting acts on the logit.
- Tuning η, depth and rounds separately; not early-stopping on validation; letting boosting chase noisy labels.
- Reading importance as causality; ignoring masking by correlated features; fitting the meta-model on in-sample predictions; listing AdaBoost as a library.

5 Chapter 1: What Does It Mean to Learn?

Let me start with the question the whole course rests on. A machine has an ability when it maps situations to good actions. There are two ways to give it one: you can sit down and write the rules yourself, or you can let the machine find the rules inside a collection of experience. Learning is the second route, and we choose it precisely when nobody can write the rules down. This chapter names the kinds of experience a machine can learn from, the four ingredients that every learning system is built from, and the one difficulty that never goes away: a finite sample is consistent with infinitely many rules, so whatever the machine does on situations it has not seen comes from the designer's assumptions, not from the data. At the end we meet entropy, cross-entropy and the KL divergence, which give us a yardstick for "how surprised is someone who believes q when the truth is p", and that yardstick becomes the training loss of every classifier in the course.

Rules versus learning

What it is. Route 1 is to encode human knowledge as explicit rules: a payroll program, a chess-opening book. Route 2 is to let the machine extract regularities from examples. Why would you ever prefer the second? Because some knowledge is tacit: you recognise a handwritten 3 instantly, but you cannot write the rule you use. Because some rule sets are too large or change too fast to maintain. And because sometimes nobody knows the rule at all. In those cases learning is not a luxury, it is the only route available.

Manipulate / measure. Here is the experiment I would run in front of you. Take a hand-written rule set for recognising digits and a learner trained on examples, and feed both the same digit while you perturb it: change the slant, the stroke width, add noise. Watch what happens. The rule set works until the first variation its author did not foresee, and then it breaks completely. The learner degrades gradually, but only for the kinds of variation that were present in its experience; for a variation it never saw, it is as lost as the rules. That last clause is the lesson of the whole chapter.

Goes wrong. Students like the slogan "machine learning replaces programming". It feels right because the decision rule is no longer typed by a person. But the learner still needs a program: a data pipeline, a model, a training procedure, an evaluation. What learning replaces is the hand-written decision rule, nothing else, and the quality of the surrounding program decides whether the learned rule is any good.

Versus. Students mix the two routes up because both end with a machine that maps inputs to outputs. The way to keep them apart is to ask where the rule lives. With rules, it is transparent and brittle and needs no data. With learning, it is implicit in the parameters, it adapts, it needs data, and it is opaque: you cannot read it off the way you read a program.

The three forms of experience

What it is. Supervised learning uses labelled examples (x, y) and learns the map $x \to y$. Unsupervised learning sees observations x alone and looks for structure: clusters, a density, a representation. Reinforcement learning learns from interactions: the machine acts, receives a reward, and learns a policy. Notice that the same records can serve all three; the form of experience is not a property of the data but of the question you ask and the feedback you can obtain.

Manipulate / measure. Imagine one table of hospital records. I ask you to predict the diagnosis from the other columns: that is supervised, because the diagnosis column is the label. I ask you instead to group patients who resemble each other: that is unsupervised, there is no label, only a criterion on the grouping. I ask you to choose the treatment that maximises recovery: that is reinforcement, because the feedback is the outcome of an action, and it may arrive late. Same table, three paradigms. What changed

was the question and the feedback.
Goes wrong. The tempting mistake is to call a task supervised because labels *could* be collected. It is supervised only once they have been collected, and collecting them is usually the expensive, slow, noisy step. If you plan a supervised project without budgeting for labels, you have planned nothing.
Example. In cybersecurity the three forms look like this: label flows as attack or benign and learn to tell them apart (supervised); cluster hosts by their behaviour to discover a botnet nobody has named yet (unsupervised); tune a firewall policy by watching the consequences of its decisions (reinforcement).

Supervised learning: labels, outputs, decisions

What it is. A supervised dataset is $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{N}$ [on your sheet]. When y is a number we call the task regression; when y is a category we call it classification. Where do labels come from? From people, from sensors, from events that happen later, from other systems. Every one of these sources carries noise: on hard cases, human annotators disagree with each other about 10% of the time. So when you hear "ground truth", hear it as a strong phrase for something that is itself an estimate. Now the part students skip: the model produces a prediction, say a probability $p = 0.1$ of malignancy. A prediction is not yet a decision. A decision (biopsy or not) is obtained from the prediction by a criterion, and the decision becomes an action in the world.
Manipulate / measure. The training cycle, as I would narrate it at the board: the machine receives an input, makes a prediction, compares it with the label, moves its parameters a little toward the answer that would have been better, and repeats with the next example. The quantity we measure at every step is the loss, the distance between the prediction and the label. Chapter 2 turns that loss into mathematics.
Goes wrong. You will be tempted to optimise the prediction and forget the decision, because the prediction is where the mathematics is. But the same $p = 0.1$ means "no biopsy" if the threshold is 0.2 and "biopsy" if the threshold is 0.05. The threshold is part of the system; if you do not choose it from the costs, someone else's default chooses it for you.
Versus. Keep regression and classification apart by what they report. Regression averages and reports a number. Classification reports class probabilities, and then a separate step turns them into a class, either by taking the largest or by comparing with a threshold.

Representations, feature engineering, self-supervision

What it is. The representation is what you actually hand to the learner. It can be hand-designed features (word counts, flow statistics) or features learned from the data itself, as a deep network does. Self-supervision is a trick for learning a representation without labels: hide a part of the data and ask the model to predict it, the next word, the masked pixel. A model that can predict the next word can also be run forward to generate text, which is where generation comes from.
Manipulate / measure. Take the same emails and build two representations, a bag of words and a learned embedding, then train the same learner on each and plot the error against the number of training emails. Look at the two curves: with little data the hand-designed representation is ahead, because it already encodes what matters; with a lot of data the learned representation overtakes it, because it can discover regularities the designer did not think of. The point where the curves cross is the point where representation learning starts to pay for itself.
Goes wrong. Students treat the representation as neutral plumbing. It is not. It fixes which regularities exist for the learner at all: if the representation does not contain the information that decides the label, no model built on top of it can recover that information. In Chapter 2 this appears as the Bayes error, which is a property of the representation, not of the algorithm.
Example. A bigram model is self-supervision in miniature. Train it on "the cat sat on the mat . the cat ate the rat ." and you get $\mathbb{P}(\text{cat} \mid \text{the}) = \frac{2}{4}$, with mat and rat at $\frac{1}{4}$ each [not on your sheet: know it]. Now ask it about the sentence "the cat ate the mat .", which it never saw. It assigns probability $\frac{1}{16}$, exactly the same as a sentence it did see. That tells you what the model learned: transitions between words, not sentences. It generalises by recombining pieces.

The four ingredients of a learning system

What it is. Every learning system is made of the same four parts, and I want you to be able to name them for any system you meet. First, the experience: the data and, just as important, how they were collected. Second, the model: the hypothesis space, the set of functions the system can express at all. Third, the learning algorithm: how one function is picked out of that space, which means an objective and an optimiser. Fourth, the evaluation criterion: what "good" means, and measured on which examples.

Manipulate / measure. Here is how I would design one from scratch, say a controller for an irrigation pump. I write the criterion first: what does a dry field cost, what does a wasted litre cost. Then the experience: which sensors, over which seasons, including the failures. Then the output: a watering schedule. Only then the algorithm. Notice that every one of these choices is an assumption about the world, and the order matters: the criterion comes first because it decides what the other three are for.
Goes wrong. The tempting order is the reverse: start from the algorithm ("let us use XGBoost") because that is the exciting part. Then you discover that the labels you need do not exist, or that the criterion you optimised is not what the business pays for. Starting from the algorithm is how projects fail quietly.
Versus. Students confuse the model with the algorithm because both are "the method". Keep them apart like this: the model is what *can* be expressed; the algorithm is *which* expressible function gets chosen. They fail differently, too: a poor model gives approximation error, a poor algorithm gives optimisation error, and Chapter 3 names both.

Generalisation, shortcuts, and what data can and cannot say

What it is. Generalisation means performing well on situations that were not in the sample. Here is the uncomfortable fact, proved in the derivation below: the data say nothing at all about inputs outside the training set. Whatever the machine does there comes from the model and the algorithm, that is, from your assumptions. Now two words you must keep separate. A shortcut is a cue that is available at decision time and predicts the label only because of how the collection was made: snow in the wolf photographs. Target leakage is a cue that will not exist at decision time at all: "analyst comments" written after the investigation. Leakage disappears when you remove the field; a shortcut does not, because it is a real feature with an accidental correlation.
Manipulate / measure. The wolves-and-snow experiment. Collect 1000 wolves, 900 of them photographed in snow, and 1000 dogs, 100 of them in snow. Hold the learner fixed and vary only the test conditions. The rule "snow means wolf" scores 90% on this collection. Now deploy it where 30% of both wolves and dogs are in snow: it scores 50%, a coin toss. A husky in snow and a wolf on grass are the two images that expose it. And notice that you can test for this without opening the model: hold out data, break the suspected cue, shift the conditions, remove the feature and retrain.
Goes wrong. The seductive sentence is "our AI learned every possible situation from millions of examples". It confuses the situations that were observed with the situations that may occur. Quantity is not diversity, and diversity is not representativeness: a million near-repeats of the same situation contain one situation.
Versus. Three learners can all score 100% on the training set. The memoriser looks up the input and otherwise guesses: perfect in training, a coin toss on anything new. The shortcut learner is good on the whole collection and fails the moment the cue decouples from the label. The genuine learner survives the probes. The training score cannot distinguish them; only the probes can.

Entropy, cross-entropy, KL divergence

What it is. Entropy $H(p) = -\sum_c p_c \log p_c$ [on your sheet] measures the uncertainty of the outcome itself: it is large when the truth is spread out, zero when one outcome is certain. Cross-entropy $H(p, q) = -\sum_c p_c \log q_c$ [on your sheet] is the average surprise of someone who believes q when the truth is p. The KL divergence $\mathrm{KL}(p\|q) = \sum_c p_c \log(p_c/q_c) = H(p, q) - H(p) \geq 0$ [on your sheet] is the extra surprise caused by believing q instead of the truth. When the truth is one-hot, the cross-entropy collapses to $-\log q_y$ [on your sheet], and that is the classification loss you will use everywhere.
Manipulate / measure. The calibration experiment. Fix the truth at $p = (0.7, 0.3)$. Now vary only what the forecaster announces, q, and measure $H(p, q)$ in bits. Announce the truth and you pay 0.881, which is the floor $H(p)$. Hedge with $(0.5, 0.5)$ and you pay 1.000. Be overconfident in the right direction, $(0.9, 0.1)$, and you pay 1.103, more than hedging. Lean the wrong way, $(0.3, 0.7)$, and you pay 1.370. Say $(0.99, 0.01)$ and you pay 2.003. Plot the cost against q and you see a U with its minimum exactly at the truth: that is why minimising this loss pushes the model toward calibrated probabilities.
Goes wrong. The tempting reading is that a small cross-entropy means a good model. But the floor $H(p)$ may itself be small; compare the KL divergence, which is the part the model controls. And the second trap: overconfidence in the right direction looks like accuracy but can cost more than hedging, as the 1.103 against 1.000 shows.
Versus. Keep the three apart by what they depend on. $H(p)$ depends on the truth only. $H(p, q)$ and KL depend on both the truth and the prediction. The KL divergence is the cross-entropy with the problem's own noise subtracted, which is why it, not the raw cross-entropy, says how good the forecaster is.

Supervised versus unsupervised versus reinforcement

	Supervised	Unsupervised	Reinforcement
Experience	pairs (x, y)	inputs x only	interactions, delayed reward
Output	$x \to y$ predictor	structure, representation, density	policy: state $\to$ action
Feedback	the label, immediately	none; a criterion on the structure	reward, possibly delayed
Security example	attack / benign flows	cluster hosts, detect anomalies	adaptive firewall policy

Finite data are consistent with infinitely many rules

Let the training inputs be a finite set $S = \{x^{(1)}, \dots, x^{(N)}\}$ inside an infinite input space $\mathcal{X}$, with two labels $y_0 \neq y_1$.
For every subset $T \subseteq \mathcal{X} \setminus S$ define a function f_T: on S it returns the recorded label, on T it returns y_1, everywhere else y_0. This is a legitimate function because S, T and the rest are disjoint.
Every f_T fits the training data perfectly, because on S it copies the labels.
Distinct subsets T give distinct functions, because they differ on some unseen input; and an infinite set has infinitely many subsets.
Take $T = \mathcal{X} \setminus S$ and $T = \emptyset$: these two perfect fits disagree on *every* unseen input. [not on your sheet: know it]
Conclusion: the data constrain nothing off the sample. What the machine does there is decided by the model family and the algorithm, that is, by your assumptions.

Gibbs' inequality: $\mathrm{KL}(p\|q) \geq 0$, with equality iff $p = q$

The key fact is $\ln t \leq t - 1$ for $t > 0$, with equality only at $t = 1$ [on your sheet]; you check it by noting that $t - 1 - \ln t$ has derivative $1 - 1/t$, so its minimum is at $t = 1$ where it equals 0.
Apply it with $t = q_c/p_c$ on the classes where $p_c > 0$; this is allowed because the ratio is positive.
Then $-\mathrm{KL}(p\|q) = \sum_c p_c \ln(q_c/p_c) \leq \sum_c p_c(q_c/p_c - 1) = \sum_c q_c - 1 \leq 0$: the first inequality is the key fact weighted by p_c, the last holds because the q_c sum to at most one.
Equality needs $q_c = p_c$ wherever $p_c > 0$ and $\sum_c q_c = 1$, which together force $q = p$.
What it buys: $H(p, q) \geq H(p)$, so the truth is the best possible prediction; and $H(p) \leq \log C$, with equality for the uniform distribution, because the uniform q gives $H(p, q) = \log C$.

How he asks it

Q. *Explain why a system that is right on every training example may still be useless, and give the one-line argument.*
A. The claim is that training accuracy says nothing about new inputs. The reason is that infinitely many functions agree with a finite sample and differ elsewhere: the training inputs form a finite set S, and any assignment of labels on $\mathcal{X} \setminus S$ is consistent with the data. The perfect memoriser, which looks up x and otherwise says class 0, has zero training error and is a coin toss on anything new. So what decides behaviour off the sample is the model family and the algorithm, and only an evaluation on held-out inputs can tell the memoriser from a learner.

Q. *A wolf-versus-dog classifier scores 90% on the held-out split and 50% in the field. Explain, and design the experiment that exposes it.*
A. The held-out split was drawn from the same collection, so it shares the collection's accidental correlation: wolves in snow. A rule that reads the background scores $(900 + 900)/2000 = 90\%$ there. At deployment, with 30% snow in both classes, the same rule scores $0.5 \cdot 0.3 + 0.5 \cdot 0.7 = 50\%$. The experiment: keep the model fixed and decouple the cue from the label, with a husky in snow, a wolf on grass, or images from another season, and measure again; a collapse proves the shortcut. The repair is at collection time, by varying the background, or by removing the cue and re-evaluating under shifted conditions.

Q. *Why does minimising cross-entropy produce calibrated probabilities and not merely the correct argmax?*
A. Because at each x the expected loss is $H(p(x)) + \mathrm{KL}(p(x)\|q)$, and by Gibbs the second term is zero only when q equals $p(x)$. Any other q pays extra, including one that has the right argmax but the wrong

confidence. In the calibration table, $(0.9, 0.1)$ has the right argmax for the truth $(0.7, 0.3)$ and still costs 0.222 bits more than the truth, which is more than hedging at 0.119. The loss therefore rewards the right probabilities, not just the right ranking.

Q. *Distinguish target leakage from a shortcut and say how each is repaired.*
A. Leakage is a feature that is unavailable at decision time or is caused by the label, such as "host quarantined afterwards"; remove the field and the inflated score disappears, which is also how you diagnose it. A shortcut is a feature that is available at decision time but whose link with the label is an artefact of the collection, such as the hour of day or the camera; it survives field removal if another artefact remains, and it is exposed only by data gathered under different conditions. Leakage is repaired by removing the field; a shortcut by diversifying the collection, or by ablating the feature and re-evaluating under shifted conditions.

Five mistakes

- Reading training performance as evidence of learning, when the memoriser also scores 100%.
- Confusing quantity with diversity and representativeness: a million near-repeats contain one situation.
- Treating the representation as neutral, when it fixes the floor every model can reach.
- Forgetting that a prediction becomes a decision only through a criterion, a threshold set from the costs.
- Comparing cross-entropies without the floor $H(p)$ in mind, or reading overconfidence as accuracy.

6 Chapter 2: From Representations to Decisions

This chapter is where the mathematics of the course begins, and I want you to see it as one story. First, objects become feature vectors, and whatever the representation leaves out is lost for good. Second, suppose for a moment that we knew the conditional distribution $\mathbb{P}(Y \mid X = x)$: then the best action at x is simply the one with the smallest conditional expected loss, and each loss picks out its own summary of the distribution, the mean, a median, a quantile, the most probable class, or a cost threshold. Third, we never know the distribution, so we estimate it from a finite i.i.d. sample, either locally with nearest neighbours, which the curse of dimensionality defeats, or with a parameterised family fitted by empirical risk minimisation, which has the opposite problem of fitting its own sample too well. Fourth, cross-entropy is the training loss whose minimiser is the true probability, which is what makes the probabilities a classifier outputs mean something. Finally, training, validation and test rows have three different jobs, and the honesty of every number you report depends on not mixing them.

The representation controls the available information

What it is. Before any learning happens, an object has to become a vector $x \in \mathbb{R}^d$. A categorical value such as a neighbourhood becomes a one-hot vector, one coordinate per category, because if you coded the neighbourhoods as 1, 2, 3 you would be inventing an order and a distance between them that do not exist. The principle behind this card is simple and unforgiving: whatever the representation leaves out, no model downstream can recover.
Manipulate / measure. Take the same set of emails and describe each one in three ways: by its length only, by a bag of words, and by the words together with sender metadata. Train the same learner on each description and look at the error. It falls as the description carries more of what actually decides the label. The error that remains at the richest description is not the learner's fault; it is the noise of the problem itself on that representation, and it is the floor we will meet again as the Bayes error.
Goes wrong. The tempting shortcut is integer coding: neighbourhood A is 1, B is 2, C is 3. It feels harmless, but for a distance-based or linear model "neighbourhood 3" is now twice "neighbourhood 1.5", and the model will use that fictitious arithmetic. You catch it by asking, for every coded feature, whether the differences between codes mean anything.

Conditional risk and the Bayes decision rule

What it is. Suppose the application pays a decision loss $\ell(y, a)$ when the truth is y and we take action a. The conditional risk of action a at x is the expected loss given what we know, $R(a \mid x) = \sum_y \ell(y, a)\, p(y \mid x)$, and the Bayes rule chooses the action with the smallest conditional risk, $f^*(x) = \arg\min_a R(a \mid x)$ [on your sheet]. Why is minimising at each x enough? Because the total risk is an average over x of the conditional risks, and if you minimise every term of an average you minimise the average: pointwise optimality is global optimality. Notice what the decision needs: only $p(y \mid x)$ and the loss. Learning the distribution and choosing the action are two separate jobs, and keeping them separate is what lets one model serve several policies.

Goes wrong. The reflex is to declare the most probable class whatever the loss. That is the Bayes rule for the zero–one loss only, where every mistake costs the same. As soon as a missed attack costs more than a false alarm, the most probable class is the wrong answer.

Versus. Students confuse the decision loss with the training loss because both are called "the loss". The decision loss is what the application pays, and it decides the action. The training loss is what the algorithm optimises to find the parameters, and we will see in this chapter why it is usually a different function. One model, trained once, can then serve many decision policies.

Which number to report: squared, absolute, asymmetric loss

What it is. In regression the action is a number, and the loss decides which number. Squared loss selects the conditional mean $m(x)$, because $\mathbb{E}[(Y-a)^2 \mid x] = \mathrm{Var}(Y \mid x) + (m(x)-a)^2$ [on your sheet], and the only part you control is the second term. Absolute loss selects a conditional median. If being one unit under costs c_u and one unit over costs c_o, the best report is the quantile at level $\tau = c_u/(c_u + c_o)$ [on your sheet]: the more expensive it is to be under, the higher you aim.

Manipulate / measure. Here is a distribution I use in class: a delivery takes 100 minutes with probability 0.5, 200 with probability 0.3, 600 with probability 0.2. Change only the loss and watch the answer move. Under squared loss the best report is the mean, 230, with risk 36,100 against 37,000 if you said 200. Under absolute loss any value in $[100, 200]$ is a median, with risk 130, against 148 at the mean. If being late costs nine times as much as being early, then $\tau = 0.9$ and you promise 600. Same distribution, three honest answers, because three different losses.

Goes wrong. The tempting report is the mean, because it is the number you learned first. But look at the delivery example: the mean 230 is a value that never occurs, dragged there by a rare large outcome. The loss, not the distribution, decides what to report.

Versus. Keep the three summaries apart by what they listen to. The mean uses all the probability mass and is pulled by outliers. The median is robust and ignores how far away the outliers are. The quantile encodes asymmetric costs explicitly.

Which class to declare: zero–one loss and the Bayes error

What it is. Under zero–one loss, every mistake costs one, so the best action at x is the most probable class, $f^*(x) = \arg\max_c \mathbb{P}(Y = c \mid x)$ [on your sheet], and its conditional risk is $1 - \max_c \mathbb{P}(Y = c \mid x)$. Average that over x and you have the Bayes error [on your sheet], the lowest error any classifier can reach on this representation. It is a property of the problem, not of the method.

Goes wrong. Students hope that a better algorithm will beat the Bayes error. It cannot. If the class probabilities at x are $(0.5, 0.3, 0.2)$, the best possible rule errs half the time at that x, full stop. The only way to lower the floor is to enrich x with information that changes the conditional distribution.

Cost-sensitive threshold and the review band

What it is. Let class 1 be the class we want to detect, $p(x) = \mathbb{P}(Y = 1 \mid x)$, with C_{FP} the cost of a false alarm and C_{FN} the cost of a miss. Declaring positive costs $C_{\mathrm{FP}}(1-p)$ on average, declaring negative costs $C_{\mathrm{FN}}\, p$, so you declare positive exactly when $p > t = C_{\mathrm{FP}}/(C_{\mathrm{FP}} + C_{\mathrm{FN}})$ [on your sheet]. If a third action, sending the case to a human reviewer, costs a flat C_R, it is the best action on the band $C_R/C_{\mathrm{FN}} < p < 1 - C_R/C_{\mathrm{FP}}$, and that band is nonempty only if $C_R < C_{\mathrm{FP}}C_{\mathrm{FN}}/(C_{\mathrm{FP}} + C_{\mathrm{FN}})$ [on your sheet].

Manipulate / measure. The threshold-versus-cost experiment. Take calibrated probabilities on 10,000 validation flows and fix $C_{\mathrm{FP}} = 1$. Now sweep the threshold t and, in separate runs, set C_{FN} to $1, 3, 10, 30, 100$. Plot the total cost against t: each run is a U, and its minimum sits at $0.5, 0.25, 0.09, 0.03, 0.01$ respectively, exactly the formula. The experiment also works as a test: if the minimum is not where the formula says, the probabilities are not calibrated.

Goes wrong. The classic slip is to put C_{FN} on top of the fraction. Remember the direction: expensive misses give a *small* threshold and more alarms. With $C_{\text{FN}} = 10$ and $C_{\text{FP}} = 1$ the threshold is $1/11 \approx 0.09$; at $p = 0.2$ raising the alarm costs 0.8 on average and staying silent costs 2, so you alarm even though the attack is improbable.
Example. Tutorial 1, Question 2: $C_{\text{FP}} = 2$, $C_{\text{FN}} = 8$, $C_R = 0.9$. Declare benign below 0.1125, attack above 0.55, and review in between. The two cost lines cross at $p^* = 0.2$ with height 1.6, so a review cost of 1.6 or more closes the band and reviewing is never worth it.

k-nearest neighbours

What it is. Now we drop the fiction that the distribution is known and replace it by an i.i.d. sample. The most direct estimate is local: look at the k training points nearest to x and average their outputs, $\hat{f}_k(x) = \frac{1}{k}\sum_{i \in N_k(x)} y^{(i)}$ for regression and $\hat{p}_k(c \mid x) = \frac{1}{k}\sum_{i \in N_k(x)} \mathbf{1}(y^{(i)} = c)$ for classification [on your sheet]. There are no parameters to fit; the hyperparameters are k, the distance and the feature scales. With $k = 1$ the input space is cut into Voronoi cells whose boundaries are perpendicular bisectors [on your sheet], and the capacity of the method rises as k falls, roughly N/k regions.
Manipulate / measure. Fix the split and the distance and vary only k from 1 to N. The training error is 0 at $k = 1$, because every point is its own nearest neighbour, and it rises with k. The validation error is high at $k = 1$, falls to a minimum, and rises again toward the error of always predicting the global majority at $k = N$. That U is the first validation curve of the course. Give the method more data and the minimum moves to a larger k, because more neighbours can now be found close by.
Goes wrong. The tempting way to pick k is by training error, which always answers $k = 1$; the other tempting way is to tune the scales on the test set. Both corrupt the number you report upward. The fix is to choose them on validation rows or by cross-validation.
Example. Six points and the query $(2, 1.8)$: with $k = 1$ the answer is A; with $k = 3$ it is still A, with $\hat{p}(\text{A}) = 2/3$ and regression value 13.3; with $k = 5$ it flips to B, with 3/5 and value 16.4. The decision depends on k, which is why k must be chosen carefully.

Distances and scaling

What it is. The Minkowski distance $d_p(x, z) = (\sum_j |x_j - z_j|^p)^{1/p}$ [on your sheet] gives Manhattan for $p = 1$ and Euclidean for $p = 2$; the Hamming distance counts differing bits. A scaled and weighted version is $d_w = \sqrt{\sum_j w_j((x_j - z_j)/s_j)^2}$ [on your sheet]. I want you to see the distance as the inductive bias of kNN: it is the method's entire opinion about which points resemble each other.
Manipulate / measure. The scaling experiment. Two customers, $P = (30{,}000, 25)$ and $Q = (31{,}000, 60)$, income and age, and a query $(30{,}800, 25)$. With raw units the distance to P is 800 and to Q is 203, so the nearest neighbour is the 60-year-old. Standardise with $s = 1000$ for income and 10 for age: now the distances are 0.8 and 3.5 and the nearest is P. Nothing about the customers changed; the units chose the answer.
Goes wrong. Students standardise with μ_j, σ_j computed on all rows before splitting. That lets the test rows influence the representation they are graded on, which is leakage. Fit the scaler on the training rows and reuse the same numbers on everything else.

The curse of dimensionality

What it is. Here is why the local idea breaks. To cover $[0, 1]^d$ at 10 positions per axis you need 10^d cells [on your sheet]. Put n uniform points in the unit ball and call a point a neighbour if it is within $r = 0.1$ of the centre: the number of neighbours is $M_d \sim \text{Bin}(n, r^d)$ and $\mathbb{P}(M_d \geq 1) = 1 - (1 - r^d)^n$ [on your sheet]. The radius you need for a 95% chance of finding even one neighbour, $r_d = (1 - 0.05^{1/n})^{1/d}$, tends to 1 as d grows [on your sheet]: your "neighbour" ends up at the edge of the ball.
Manipulate / measure. Fix $n = 1000$ and vary only d. The probability of at least one neighbour within 0.1 is about 1 at $d = 1$, 0.99996 at $d = 2$, 0.632 at $d = 3$, 0.095 at $d = 4$, 0.010 at $d = 5$, 0.001 at $d = 6$, and 10^{-7} at $d = 10$; the expected count is $n10^{-d}$, which is 1 at $d = 3$. From about $d = 5$ the nearest point is far away and the local average is not local any more.
Goes wrong. The tempting plan is to run kNN on all 80 raw flow statistics. No stored flow is close to a new one in 80 dimensions, so the detector degrades into a crude rule on whichever features dominate the distance. Select or learn features first, or move to models that share parameters across inputs (Chapters 4 and 5). And remember the cost: $O(Nd)$ memory and $O(Nd)$ per query.

Parameterised models, nested families, the simplex

What it is. The alternative to storing the data is to choose a family $\mathcal{H} = \{f_\theta : \theta \in \Theta\}$ and fit its parameters. Families are nested when one contains the other, as lines sit inside quadratics. A probabilistic classifier outputs a point of the simplex, C nonnegative numbers summing to one, typically the softmax of some scores [on your sheet]. Parameters are set by the algorithm from the training rows; hyperparameters, such as k, the degree, λ or the scales, are set by the designer, and they must be chosen on data the fit did not see.

Goes wrong. If you let the training objective choose a hyperparameter, it always picks maximal capacity, as the next card proves. The hyperparameter is exactly the knob that training error cannot be trusted with.

Versus. Students ask what kNN and a parametric model have in common. Almost nothing: kNN learns nothing and stores everything, while a parametric model compresses all rows into θ and shares it across all inputs. That sharing is what survives high dimension, because a new input does not need a stored neighbour, only the parameters.

Population risk, empirical risk, ERM: honest versus optimistic

What it is. The number we want is the population risk $R(\theta) = \mathbb{E}\,\ell(Y, f_\theta(X))$, and it is unknowable. The number we can compute is the empirical risk $\widehat{R}(\theta) = \frac{1}{N}\sum_i \ell(y^{(i)}, f_\theta(x^{(i)}))$ [on your sheet], and empirical risk minimisation picks $\hat{\theta} = \arg\min_\theta \widehat{R}(\theta)$. Here is the subtle point. For a *fixed* θ, $\mathbb{E}[\widehat{R}(\theta)] = R(\theta)$: the training risk is an honest estimate. After minimisation it is not: $\mathbb{E}[\widehat{R}(\hat{\theta})] \le \min_\theta R \le \mathbb{E}[R(\hat{\theta})]$ [on your sheet], the training risk of the fitted model is optimistic. Two more facts: nested families can only fit better, $\mathcal{H}_1 \subseteq \mathcal{H}_2 \Rightarrow \min_{\mathcal{H}_2} \widehat{R} \le \min_{\mathcal{H}_1} \widehat{R}$ [on your sheet], and a polynomial of degree $N-1$ interpolates any N points, giving training risk 0.

Manipulate / measure. The honest-versus-optimistic experiment. Simulate a world, fix one line θ_0 in advance, and draw $N = 20$ points. Repeat the draw a thousand times and make three histograms: the training risk of the fixed line θ_0, the training risk of the line fitted to each sample, and the true risk of that fitted line. The first histogram is centred on $R(\theta_0)$: honest. The second sits below the best achievable risk, the third above it. The distance between the second and the third is the optimism, and it is there on every draw.

Goes wrong. The tempting comparison is between degrees by training risk. On the points $(0,1),(1,3),(2,2),(3,4)$ it goes 1.25 for the constant, 0.45 for the line $0.8x + 1.3$, and 0 for the cubic, and it says nothing whatsoever about a fifth point. You catch it by asking which rows produced the number.

Cross-entropy: the training loss that gives calibrated probabilities

What it is. Why not train on the zero–one loss we actually care about? Because as a function of θ it is piecewise constant: its gradient is zero or undefined, and it is blind to confidence. Cross-entropy, $\ell_{\text{CE}} = -\log \hat{p}_{\theta,y}(x)$ and $\widehat{R}_{\text{CE}} = -\frac{1}{N}\sum_i \log \hat{p}_{\theta,y^{(i)}}(x^{(i)})$ [on your sheet], in the binary case $-y\log\hat{p} - (1-y)\log(1-\hat{p})$ [on your sheet], fixes both problems. At each x its expectation is $H(p(x)) + \mathrm{KL}(p(x)\|\hat{p}_\theta(x))$ [on your sheet], which by Gibbs is minimised exactly by the true probabilities: that is calibration. And since $\log L(\theta) = -N\widehat{R}_{\text{CE}}$, minimising cross-entropy is maximum likelihood [on your sheet].

Manipulate / measure. Look at what the loss charges for the probability given to the true class: 0.9 costs 0.105, 0.6 costs 0.511, 0.5 costs 0.693, 0.1 costs 2.303, 0.01 costs 4.605 nats. Three examples at $0.9, 0.6, 0.1$ average 0.973: the one confident mistake dominates the sum. That asymmetry is deliberate; it is what pushes the model away from confident errors.

Goes wrong. Two temptations. Training on the zero–one loss, which cannot be optimised; and reading the training cross-entropy as something that should reach zero, when its floor is $H(Y \mid X)$, the problem's own noise.

Versus. The objective says which θ is preferred; the algorithm says how that θ is found, in closed form or by gradient steps. Keep them apart because they fail differently: a wrong objective finds the wrong thing perfectly, a weak algorithm finds the right thing badly.

Train, validation, test; selection; cross-validation

What it is. Training rows fit θ. Validation rows choose hyperparameters, $\hat{\lambda} = \arg\min_\lambda \widehat{R}_{\text{val}}(\hat{\theta}_\lambda)$ [on your sheet]. Test rows are used once, at the end. The held-out risk of a *fixed* model is honest, but the best of several is not: $\mathbb{E}[\min_h \widehat{R}_{\text{val}}(\hat{f}_h)] \le \min_h R(\hat{f}_h)$ [on your sheet], the winner's curse, because picking

the minimum of noisy estimates picks part luck. When data are scarce, K-fold cross-validation holds out each fold in turn, $\widehat{R}_{\mathrm{CV}}(\lambda) = \frac{1}{K}\sum_j \widehat{R}^{(j)}_{\mathrm{val}}(\lambda)$ [on your sheet], so every row validates once; afterwards you refit the chosen configuration on all development rows.

Manipulate / measure. Degrees $0, 1, 2, 3, 5$ on the same data: training risk $1.25, 0.45, 0.40, 0.10, 0.00$, falling as nested families must; validation risk $1.30, 0.50, 0.62, 1.90, 8.40$, a U. You keep the line. Now cross-validation with $K = 3$ for kNN: $k = 1$ gives fold errors $0.40, 0.20, 0.60$, mean 0.40; $k = 3$ gives $0.30, 0.30, 0.30$. You choose $k = 3$. Notice that fold 2 alone would have chosen $k = 1$; averaging the folds is what protected you from one lucky slice.

Goes wrong. Three temptations: fitting the scaler or selecting features on all rows before the folds, which leaks; reporting the winner's validation score as the final number, which is optimistic; deploying one of the fold models instead of refitting on all development rows.

Example. A fraud system: split by *time*, with 20 months for development and the last 4 months for test; train on cross-entropy; choose the configuration and the threshold by cost-weighted error on validation; run the test once.

Four losses, what each selects, and the two jobs

Loss	Optimal action at x	Used as	Watch out
squared $(y-a)^2$	conditional mean	decision and training loss (regression)	outliers drag the mean
absolute $\lvert y-a\rvert$, asymmetric	median, τ-quantile	decision loss	many medians possible
zero–one $\mathbf{1}(y \neq a)$	most probable class; Bayes error floor	decision loss only	cannot be optimised: flat in θ
cost-weighted	positive iff $p > C_{\mathrm{FP}}/(C_{\mathrm{FP}}+C_{\mathrm{FN}})$	decision loss	needs calibrated p
cross-entropy $-\log \hat{p}_y$	the true probabilities	training loss (classification)	floor $H(Y \mid X)$; = MLE

Squared loss selects the conditional mean

Write $Y - a = (Y - m) + (m - a)$ with $m = \mathbb{E}[Y \mid x]$; this is allowed because we are only adding and subtracting the same number.

Square and take the conditional expectation given x: the cross term is $2(m-a)\mathbb{E}[Y - m \mid x]$, and it vanishes because m is the conditional mean, so $Y - m$ has conditional mean zero.

What remains is $\mathbb{E}[(Y-a)^2 \mid x] = \mathrm{Var}(Y \mid x) + (m-a)^2$: the first term does not depend on a, the second is minimised at $a = m$.

What it buys: the same "add and subtract a mean" move, used twice, gives the bias–variance decomposition of Chapter 3.

Cost-sensitive threshold and the review band

Declaring positive is wrong exactly when $Y = 0$, so its conditional risk is $C_{\mathrm{FP}}(1-p)$; declaring negative is wrong exactly when $Y = 1$, so its risk is $C_{\mathrm{FN}}p$.

Positive is cheaper iff $C_{\mathrm{FP}}(1-p) < C_{\mathrm{FN}}p$, that is $C_{\mathrm{FP}} < (C_{\mathrm{FP}} + C_{\mathrm{FN}})p$, that is $p > C_{\mathrm{FP}}/(C_{\mathrm{FP}} + C_{\mathrm{FN}})$: three equivalent rearrangements.

A review action at flat cost C_R is a horizontal line on the same picture; it is the lowest of the three between $p = C_R/C_{\mathrm{FN}}$, where it crosses the "negative" line, and $p = 1 - C_R/C_{\mathrm{FP}}$, where it crosses the "positive" line.

The band is nonempty only if the horizontal line lies below the point where the two sloped lines cross, whose height is $C_{\mathrm{FP}}C_{\mathrm{FN}}/(C_{\mathrm{FP}} + C_{\mathrm{FN}})$.

Training risk is optimistic after minimisation

Let θ^* minimise the population risk R. Because $\hat{\theta}$ minimises the empirical risk, $\widehat{R}(\hat{\theta}) \leq \widehat{R}(\theta^*)$ on every sample.

Take expectations over the sample: θ^* is fixed before the sample is drawn, so $\mathbb{E}[\widehat{R}(\theta^*)] = R(\theta^*) = \min R$; an average of i.i.d. losses of a fixed function is unbiased.

On the other side, $R(\hat{\theta}) \geq \min R$ pointwise, because nothing beats the population minimiser.

Chain the three facts: $\mathbb{E}[\widehat{R}(\hat{\theta})] \le \min R \le \mathbb{E}[R(\hat{\theta})]$. The one word that matters is *fixed*: validation rows grade a model that was fixed before they were seen, so for them the honest line applies.

Cross-entropy = entropy + KL, and = maximum likelihood

At a fixed x, the expected loss is $\mathbb{E}[-\log \hat{p}_{\theta,Y}(x) \mid x] = -\sum_c p_c \log \hat{p}_{\theta,c}$, an average over the true class probabilities.
Add and subtract $\sum_c p_c \log p_c$: the expression becomes $H(p) + \sum_c p_c \log(p_c/\hat{p}_{\theta,c}) = H(p) + \mathrm{KL}(p\|\hat{p}_\theta)$.
By Gibbs the KL term is at least zero, with equality only if $\hat{p}_\theta = p$: the loss is minimised by the true probabilities, which is calibration.
For the likelihood: $L(\theta) = \prod_i \hat{p}_{\theta,y^{(i)}}(x^{(i)})$, so $\log L = -N\widehat{R}_{\mathrm{CE}}$, and maximising one is minimising the other.

How he asks it

Q. *Explain why a well-trained classifier cannot have zero error on a given representation, and what would reduce the floor.*
A. The floor is the Bayes error $\mathbb{E}[1 - \max_c \mathbb{P}(Y = c \mid X)]$, and it comes from outcomes that are genuinely uncertain given x. If the class probabilities at some x are $(0.5, 0.3, 0.2)$, the best possible rule declares the first class and is wrong half the time there, and no algorithm can do better because the uncertainty is in the world, not in the method. The only way to lower the floor is to enrich the representation with features that carry more information, because that changes the conditional distribution itself.

Q. *Why does kNN fail in high dimension even with a thousand points, and what are the remedies?*
A. Because closeness becomes rare. A ball of radius 0.1 occupies a fraction 0.1^d of the unit ball, so with $n = 1000$ points the chance that any point lies within 0.1 of the query is $1 - (1 - 10^{-d})^{1000}$: 0.63 at $d = 3$, 0.10 at $d = 4$, essentially 0 at $d = 6$. The "neighbours" are then far away and their average says nothing local about the query. The remedies are to select features, to learn a lower-dimensional representation, to reweight dimensions, or to move to a parametric model, which shares its parameters across all inputs and does not need a stored neighbour.

Q. *Why can the training objective not be used to choose the polynomial degree, and what is used instead?*
A. Because the families are nested: every polynomial of degree m is also a polynomial of degree $m + 1$, so the minimum training risk can only decrease with the degree, and it reaches 0 at degree $N - 1$, where the polynomial interpolates the points. Training risk therefore always answers "maximal degree". The degree is chosen on validation rows or by cross-validation, where each fitted model is a fixed object and its estimate is honest; the test set is kept untouched for the final number.

Q. *State what training, validation and test rows each decide, and why the validation score of the chosen model is still optimistic.*
A. Training rows set the parameters; validation rows choose hyperparameters, features and thresholds; test rows report once. Each candidate's validation score is honest on its own, because the candidate was fixed before the validation rows were looked at. But the chosen candidate is the minimum of several noisy estimates, and $\mathbb{E}[\min_h \widehat{R}_{\mathrm{val}}] \le \min_h R(\hat{f}_h)$: picking the minimum picks partly luck, so the winner's score is pulled below its true risk. Only rows that played no part in the choice, the test rows, estimate the winner honestly.

Five mistakes

- Declaring the most probable class under asymmetric costs; the threshold is $C_{\mathrm{FP}}/(C_{\mathrm{FP}} + C_{\mathrm{FN}})$, and it is small when misses are expensive.
- Choosing k, the degree or the scales on training error, which always answers maximal capacity, $k = 1$.
- Standardising or selecting features on all rows before splitting, or outside the folds.
- Treating the Bayes error as something a better algorithm could beat.
- Reporting the best validation or cross-validation score as the deployment estimate.